TEAM MEMBERS:

1. NUR AINI HIDAYAH BINTI ZAINURIN
2. NURUL FADZLIN BINTI MUSTAFA KAMAR
3. NUR'DIANA BINTI ABDUL TAHRIM
4. ISHAZA ARIANNA BINTI ISMADEE ASMARA

```
#from google.colab import drive
#drive.mount('/content/drive')
```

```
import numpy as np
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.optimizers import Adam, Adamax, SGD, RMSprop, Adagrad, Ftrl
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
```

```
#IMPORT DATASET PARKINSONS
path = "/content/parkinsons data.csv"
data = pd.read_csv(path)
data.head()
```

	name	MDVP:Fo(Hz)	MDVP:Fhi(Hz)	MDVP:Flo(Hz)	MDVP:Jitter(%)	MDVP:Jitter(Abs)	MDVP:RAP	MDVP:PPQ	Jitter:DDP	MDVP:Shimm
0	phon_R01_S01_1	119.992	157.302	74.997	0.00784	0.00007	0.00370	0.00554	0.01109	0.043
1	phon_R01_S01_2	122.400	148.650	113.819	0.00968	0.00008	0.00465	0.00696	0.01394	0.061
2	phon_R01_S01_3	116.682	131.111	111.555	0.01050	0.00009	0.00544	0.00781	0.01633	0.052
3	phon_R01_S01_4	116.676	137.871	111.366	0.00997	0.00009	0.00502	0.00698	0.01505	0.054
4	phon_R01_S01_5	116.014	141.781	110.655	0.01284	0.00011	0.00655	0.00908	0.01966	0.064

5 rows × 24 columns

```
print('Parkinsons Data Info : ', data.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 195 entries, 0 to 194
Data columns (total 24 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   name              195 non-null    object
 1   MDVP:Fo(Hz)       195 non-null    float64
 2   MDVP:Fhi(Hz)      195 non-null    float64
 3   MDVP:Flo(Hz)      195 non-null    float64
 4   MDVP:Jitter(%)    195 non-null    float64
 5   MDVP:Jitter(Abs)  195 non-null    float64
 6   MDVP:RAP          195 non-null    float64
 7   MDVP:PPQ          195 non-null    float64
 8   Jitter:DDP        195 non-null    float64
 9   MDVP:Shimmer      195 non-null    float64
 10  MDVP:Shimmer(dB)  195 non-null    float64
 11  Shimmer:APQ3      195 non-null    float64
 12  Shimmer:APQ5      195 non-null    float64
 13  MDVP:APQ          195 non-null    float64
 14  Shimmer:DDA       195 non-null    float64
 15  NHR               195 non-null    float64
 16  HNR               195 non-null    float64
 17  status            195 non-null    int64
 18  RPDE              195 non-null    float64
 19  DFA               195 non-null    float64
 20  spread1           195 non-null    float64
 21  spread2           195 non-null    float64
 22  D2                195 non-null    float64
 23  PPE               195 non-null    float64
dtypes: float64(22), int64(1), object(1)
memory usage: 36.7+ KB
Parkinsons Data Info :  None
```

```
print('Parkinsons Data Describe : ')
data.describe()
```

Parkinsons Data Describe :

	name	MDVP:Fo(Hz)	MDVP:Fhi(Hz)	MDVP:Flo(Hz)	MDVP:Jitter(%)	MDVP:Jitter(Abs)	MDVP:RAP	MDVP:PPQ	Jitter:DDP	MDVP:Shimmer
count	0.0	195.000000	195.000000	195.000000	195.000000	195.000000	195.000000	195.000000	195.000000	195.000000
mean	NaN	154.228641	197.104918	116.324631	0.006220	0.000044	0.003306	0.003446	0.009920	0.029709
std	NaN	41.390065	91.491548	43.521413	0.004848	0.000035	0.002968	0.002759	0.008903	0.018857
min	NaN	88.333000	102.145000	65.476000	0.001680	0.000007	0.000680	0.000920	0.002040	0.009540
25%	NaN	117.572000	134.862500	84.291000	0.003460	0.000020	0.001660	0.001860	0.004985	0.016505
50%	NaN	148.790000	175.829000	104.315000	0.004940	0.000030	0.002500	0.002690	0.007490	0.022970
75%	NaN	182.769000	224.205500	140.018500	0.007365	0.000060	0.003835	0.003955	0.011505	0.037885
max	NaN	260.105000	592.030000	239.170000	0.033160	0.000260	0.021440	0.019580	0.064330	0.119080

8 rows × 24 columns

```
print('Parkinsons Data Head : ')
data.head()
```

Parkinsons Data Head :

	name	MDVP:Fo(Hz)	MDVP:Fhi(Hz)	MDVP:Flo(Hz)	MDVP:Jitter(%)	MDVP:Jitter(Abs)	MDVP:RAP	MDVP:PPQ	Jitter:DDP	MDVP:Shimmer	...	Sh
0	NaN	119.992	157.302	74.997	0.00784	0.00007	0.00370	0.00554	0.01109	0.04374	...	
1	NaN	122.400	148.650	113.819	0.00968	0.00008	0.00465	0.00696	0.01394	0.06134	...	
2	NaN	116.682	131.111	111.555	0.01050	0.00009	0.00544	0.00781	0.01633	0.05233	...	
3	NaN	116.676	137.871	111.366	0.00997	0.00009	0.00502	0.00698	0.01505	0.05492	...	
4	NaN	116.014	141.781	110.655	0.01284	0.00011	0.00655	0.00908	0.01966	0.06425	...	

5 rows × 24 columns

```
print('Parkinsons Data Tail : ')
data.tail()
```

Parkinsons Data Tail :

	name	MDVP:Fo(Hz)	MDVP:Fhi(Hz)	MDVP:Flo(Hz)	MDVP:Jitter(%)	MDVP:Jitter(Abs)	MDVP:RAP	MDVP:PPQ	Jitter:DDP	MDVP:Shimmer	...
190	NaN	174.188	230.978	94.261	0.00459	0.00003	0.00263	0.00259	0.00790	0.04087	...
191	NaN	209.516	253.017	89.488	0.00564	0.00003	0.00331	0.00292	0.00994	0.02751	...
192	NaN	174.688	240.005	74.287	0.01360	0.00008	0.00624	0.00564	0.01873	0.02308	...
193	NaN	198.764	396.961	74.904	0.00740	0.00004	0.00370	0.00390	0.01109	0.02296	...
194	NaN	214.289	260.277	77.973	0.00567	0.00003	0.00295	0.00317	0.00885	0.01884	...

5 rows × 24 columns

*DATA PROCESSING *

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 195 entries, 0 to 194
Data columns (total 24 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   name              0 non-null      float64
 1   MDVP:Fo(Hz)       195 non-null    float64
 2   MDVP:Fhi(Hz)      195 non-null    float64
 3   MDVP:Flo(Hz)      195 non-null    float64
 4   MDVP:Jitter(%)    195 non-null    float64
 5   MDVP:Jitter(Abs)  195 non-null    float64
```

```
 6   MDVP:RAP          195 non-null    float64
 7   MDVP:PPQ          195 non-null    float64
 8   Jitter:DDP        195 non-null    float64
 9   MDVP:Shimmer      195 non-null    float64
 10  MDVP:Shimmer(dB)  195 non-null    float64
 11  Shimmer:APQ3      195 non-null    float64
 12  Shimmer:APQ5      195 non-null    float64
 13  MDVP:APQ          195 non-null    float64
 14  Shimmer:DDA       195 non-null    float64
 15  NHR               195 non-null    float64
 16  HNR               195 non-null    float64
 17  status            195 non-null    int64
 18  RPDE              195 non-null    float64
 19  DFA               195 non-null    float64
 20  spread1           195 non-null    float64
 21  spread2           195 non-null    float64
 22  D2                195 non-null    float64
 23  PPE               195 non-null    float64
dtypes: float64(23), int64(1)
memory usage: 36.7 KB
```

```
# find if there any missing value in the datasets
print("\nMissing Values in each column:")
data.isnull().sum()

#there's no missing value in the dataset
```

```
Missing Values in each column:
name                195
MDVP:Fo(Hz)           0
MDVP:Fhi(Hz)          0
MDVP:Flo(Hz)          0
MDVP:Jitter(%)        0
MDVP:Jitter(Abs)      0
MDVP:RAP              0
MDVP:PPQ              0
Jitter:DDP            0
MDVP:Shimmer          0
MDVP:Shimmer(dB)      0
Shimmer:APQ3          0
Shimmer:APQ5          0
MDVP:APQ              0
Shimmer:DDA           0
NHR                   0
HNR                   0
status                0
RPDE                  0
DFA                   0
spread1               0
spread2               0
D2                    0
PPE                   0
dtype: int64
```

```
data
```

	name	MDVP:Fo(Hz)	MDVP:Fhi(Hz)	MDVP:Flo(Hz)	MDVP:Jitter(%)	MDVP:Jitter(Abs)	MDVP:RAP	MDVP:PPQ	Jitter:DDP	MDVP:Shimmer	...
0	NaN	119.992	157.302	74.997	0.00784	0.00007	0.00370	0.00554	0.01109	0.04374	...
1	NaN	122.400	148.650	113.819	0.00968	0.00008	0.00465	0.00696	0.01394	0.06134	...
2	NaN	116.682	131.111	111.555	0.01050	0.00009	0.00544	0.00781	0.01633	0.05233	...
3	NaN	116.676	137.871	111.366	0.00997	0.00009	0.00502	0.00698	0.01505	0.05492	...
4	NaN	116.014	141.781	110.655	0.01284	0.00011	0.00655	0.00908	0.01966	0.06425	...
...	...	...	...	...	...	...	...	...	...	...	...
190	NaN	174.188	230.978	94.261	0.00459	0.00003	0.00263	0.00259	0.00790	0.04087	...
191	NaN	209.516	253.017	89.488	0.00564	0.00003	0.00331	0.00292	0.00994	0.02751	...
192	NaN	174.688	240.005	74.287	0.01360	0.00008	0.00624	0.00564	0.01873	0.02308	...
193	NaN	198.764	396.961	74.904	0.00740	0.00004	0.00370	0.00390	0.01109	0.02296	...
194	NaN	214.289	260.277	77.973	0.00567	0.00003	0.00295	0.00317	0.00885	0.01884	...

195 rows × 24 columns

```
#change the data object type to numeric

columns_to_convert = ['name']

# Convert object type columns to numeric
for column in columns_to_convert:
    data[column] = pd.to_numeric(data[column], errors='coerce')
```

```
data
```

	name	MDVP:Fo(Hz)	MDVP:Fhi(Hz)	MDVP:Flo(Hz)	MDVP:Jitter(%)	MDVP:Jitter(Abs)	MDVP:RAP	MDVP:PPQ	Jitter:DDP	MDVP:Shimmer	...
0	NaN	119.992	157.302	74.997	0.00784	0.00007	0.00370	0.00554	0.01109	0.04374	...
1	NaN	122.400	148.650	113.819	0.00968	0.00008	0.00465	0.00696	0.01394	0.06134	...
2	NaN	116.682	131.111	111.555	0.01050	0.00009	0.00544	0.00781	0.01633	0.05233	...
3	NaN	116.676	137.871	111.366	0.00997	0.00009	0.00502	0.00698	0.01505	0.05492	...
4	NaN	116.014	141.781	110.655	0.01284	0.00011	0.00655	0.00908	0.01966	0.06425	...
...	...	...	...	...	...	...	...	...	...	...	...
190	NaN	174.188	230.978	94.261	0.00459	0.00003	0.00263	0.00259	0.00790	0.04087	...
191	NaN	209.516	253.017	89.488	0.00564	0.00003	0.00331	0.00292	0.00994	0.02751	...
192	NaN	174.688	240.005	74.287	0.01360	0.00008	0.00624	0.00564	0.01873	0.02308	...
193	NaN	198.764	396.961	74.904	0.00740	0.00004	0.00370	0.00390	0.01109	0.02296	...
194	NaN	214.289	260.277	77.973	0.00567	0.00003	0.00295	0.00317	0.00885	0.01884	...

195 rows × 24 columns

```
#change the name data to numbering
data['name'] = range(1, len(data) + 1)

print("Updated data:")
data
```

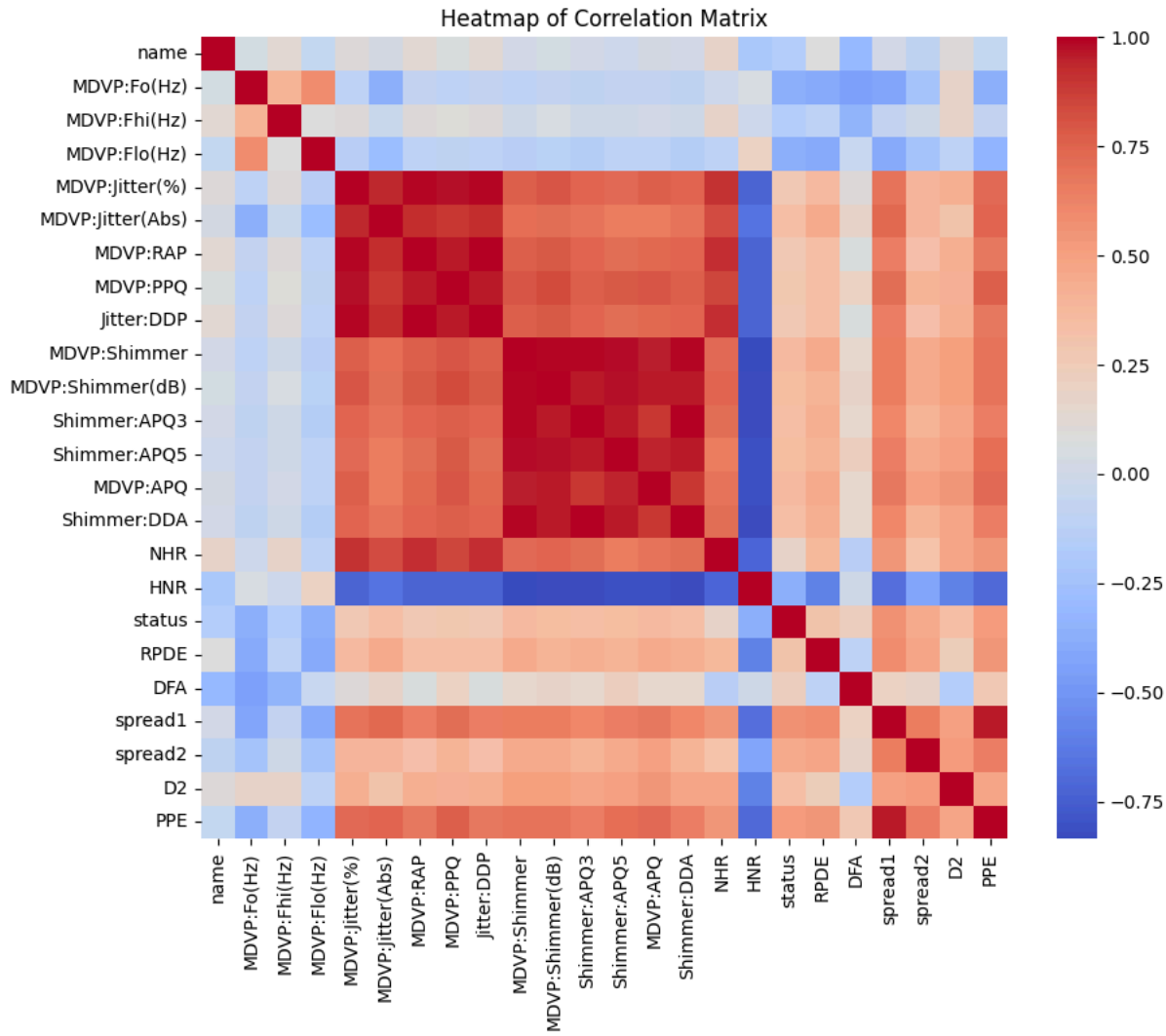
```
Updated data:
```

	name	MDVP:Fo(Hz)	MDVP:Fhi(Hz)	MDVP:Flo(Hz)	MDVP:Jitter(%)	MDVP:Jitter(Abs)	MDVP:RAP	MDVP:PPQ	Jitter:DDP	MDVP:Shimmer	...
0	1	119.992	157.302	74.997	0.00784	0.00007	0.00370	0.00554	0.01109	0.04374	...
1	2	122.400	148.650	113.819	0.00968	0.00008	0.00465	0.00696	0.01394	0.06134	...
2	3	116.682	131.111	111.555	0.01050	0.00009	0.00544	0.00781	0.01633	0.05233	...
3	4	116.676	137.871	111.366	0.00997	0.00009	0.00502	0.00698	0.01505	0.05492	...
4	5	116.014	141.781	110.655	0.01284	0.00011	0.00655	0.00908	0.01966	0.06425	...
...	...	...	...	...	...	...	...	...	...	...	...
190	191	174.188	230.978	94.261	0.00459	0.00003	0.00263	0.00259	0.00790	0.04087	...
191	192	209.516	253.017	89.488	0.00564	0.00003	0.00331	0.00292	0.00994	0.02751	...
192	193	174.688	240.005	74.287	0.01360	0.00008	0.00624	0.00564	0.01873	0.02308	...
193	194	198.764	396.961	74.904	0.00740	0.00004	0.00370	0.00390	0.01109	0.02296	...
194	195	214.289	260.277	77.973	0.00567	0.00003	0.00295	0.00317	0.00885	0.01884	...

195 rows × 24 columns

```
#HEATMAP BEFORE NORMALIZE
import seaborn as sb
# Calculate the correlation matrix
correlation_matrix = data.corr()

# Create the heatmap
plt.figure(figsize=(10, 8))
sb.heatmap(correlation_matrix, annot=False, fmt=".2f", cmap='coolwarm')
plt.title('Heatmap of Correlation Matrix')
```

```
plt.title('Heatmap of Correlation Matrix')
plt.show()
```



DATA PARTITION

```
x = data.drop('status', axis=1) #features
y = data['status'] #target
```

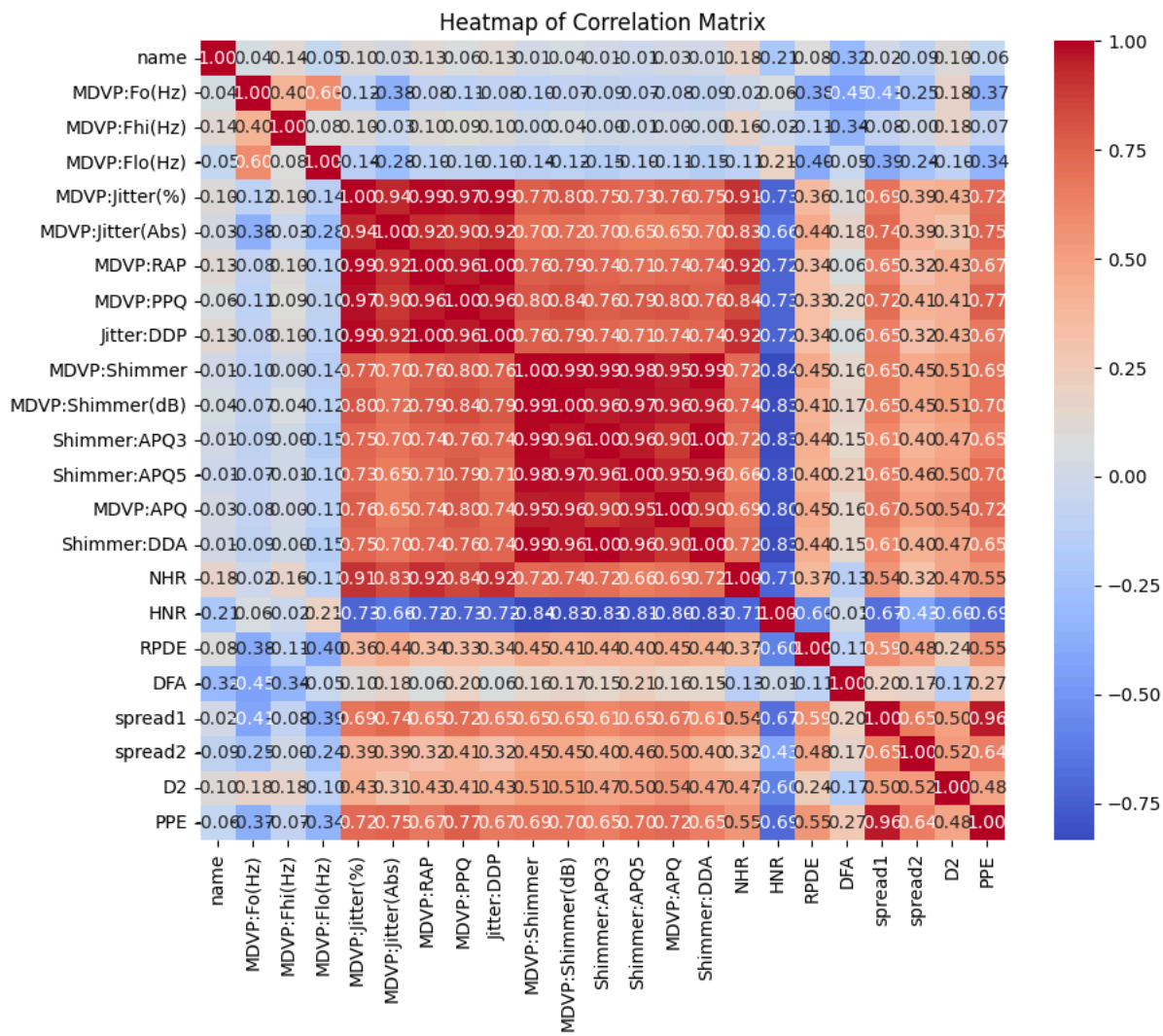
```
#Normalize the features using Min-Max Scaling
scaler = MinMaxScaler()
x_scaled = scaler.fit_transform(x)

#Convert the scaled data back to a DataFrame
x_scaled_data = pd.DataFrame(x_scaled, columns=x.columns)
```

```
x_scaled_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 195 entries, 0 to 194
Data columns (total 23 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   name              195 non-null    float64
 1   MDVP:Fo(Hz)       195 non-null    float64
 2   MDVP:Fhi(Hz)      195 non-null    float64
 3   MDVP:Flo(Hz)      195 non-null    float64
 4   MDVP:Jitter(%)    195 non-null    float64
 5   MDVP:Jitter(Abs)  195 non-null    float64
 6   MDVP:RAP          195 non-null    float64
 7   MDVP:PPQ          195 non-null    float64
 8   Jitter:DDP        195 non-null    float64
 9   MDVP:Shimmer      195 non-null    float64
```

```
 10  MDVP:Shimmer(dB)  195 non-null    float64
 11  Shimmer:APQ3      195 non-null    float64
 12  Shimmer:APQ5      195 non-null    float64
 13  MDVP:APQ          195 non-null    float64
 14  Shimmer:DDA       195 non-null    float64
 15  NHR               195 non-null    float64
 16  HNR               195 non-null    float64
 17  RPDE              195 non-null    float64
 18  DFA               195 non-null    float64
 19  spread1           195 non-null    float64
 20  spread2           195 non-null    float64
 21  D2                195 non-null    float64
 22  PPE               195 non-null    float64
dtypes: float64(23)
memory usage: 35.2 KB
```

```
#HEATMAP AFTER NORMALIZE

# Calculate the correlation matrix
correlation_matrix = x_scaled_data.corr()

plt.figure(figsize=(10, 8))
sb.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap='coolwarm')
plt.title('Heatmap of Correlation Matrix')
plt.show()
```




```
X_train, X_test, y_train, y_test = train_test_split(x_scaled_data, y, test_size=0.2, random_state=42)
```

```
print("x_train shape:", X_train.shape)
print("x_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

```
x_train shape: (156, 23)
x_test shape: (39, 23)
y_train shape: (156,)
y_test shape: (39,)
```

```
print('Y_Train Value counts')
y_train.value_counts()
```

```
Y_Train Value counts
status
1    115
0     41
Name: count, dtype: int64
```

```
print('Y_Test Value counts')
y_test.value_counts()
```

```
Y_Test Value counts
status
1    32
0     7
Name: count, dtype: int64
```

```
print('X train head: ')
print(X_train.head())
print('')
print('Y train head: ')
print(y_train.head())
```

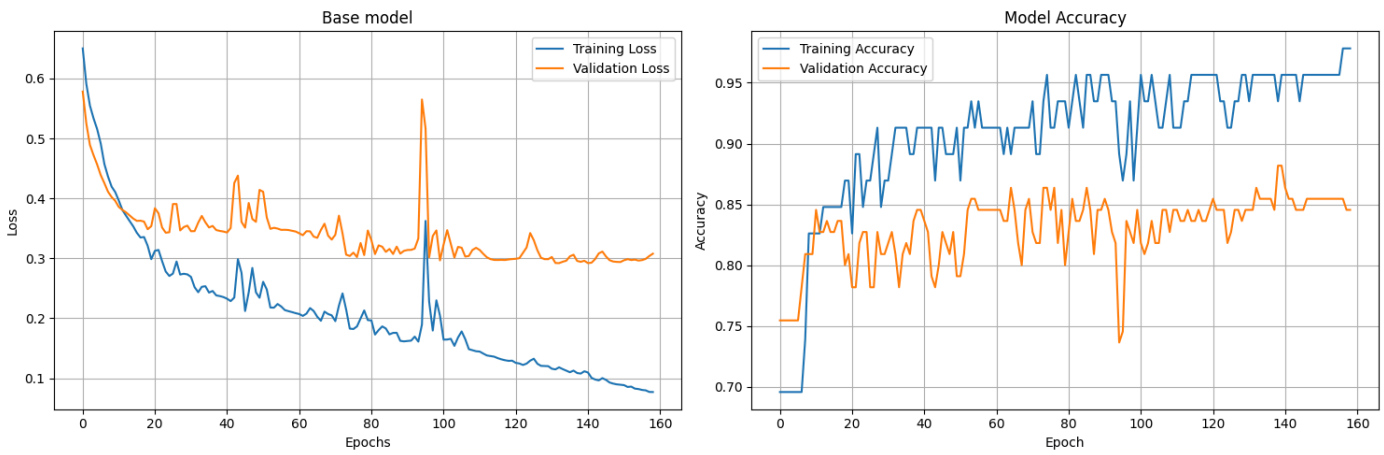
```
X train head:
          name  MDVP:Fo(Hz)  MDVP:Fhi(Hz)  MDVP:Flo(Hz)  MDVP:Jitter(%)  \
5    0.025773     0.187568      0.059232      0.278139        0.254130
135  0.695876     0.128775      0.051984      0.230739        0.103558
122  0.628866     0.290251      0.206940      0.102848        0.170267
167  0.860825     1.000000      0.332270      0.989251        0.054320
85   0.438144     0.539349      0.200006      0.518262        0.075604

     MDVP:Jitter(Abs)  MDVP:RAP  MDVP:PPQ  Jitter:DDP  MDVP:Shimmer  ...  \
5            0.288538  0.190270  0.352626    0.190079      0.342067  ...
135          0.130435  0.084778  0.119507    0.084604      0.289757  ...
122          0.169960  0.162813  0.163987    0.162787      0.321800  ...
167          0.011858  0.065992  0.050375    0.066142      0.098229  ...
85           0.051383  0.073218  0.081458    0.073045      0.264561  ...

     MDVP:APQ  Shimmer:DDA       NHR       HNR      RPDE       DFA   spread1  \
5    0.193277     0.360829  0.036827  0.525766  0.370978  0.999128  0.672961
135  0.198867     0.341764  0.078429  0.376575  0.925979  0.527497  0.475753
122  0.191515     0.410643  0.060572  0.400878  0.656693  0.666112  0.460354
167  0.049621     0.140775  0.026896  0.513777  0.430299  0.214242  0.080827
85   0.165250     0.318205  0.086482  0.314354  0.762992  0.611400  0.417123

      spread2        D2       PPE
5    0.659218  0.339999  0.648753
135  0.710843  0.537623  0.380632
122  0.346660  0.296793  0.383890
167  0.346991  0.203962  0.064304
85   0.697019  0.745137  0.322856

[5 rows x 23 columns]

Y train head:
5      1
135    1
122    1
167    0
85     1
Name: status, dtype: int64
```

BASE MODEL

```
# Define the base model function
def create_base_model(input_dim):
    model = Sequential()
    model.add(Dense(32, input_dim=input_dim, activation='relu')), Dropout(0.2),
    model.add(Dense(16, activation='relu')), Dropout(0.2),
    model.add(Dense(1, activation='sigmoid'))
    return model

# Initialize the model
model = create_base_model(X_train.shape[1])  # Assuming x_train is defined and has proper shape

# Compile the model
optimizer = Adam(learning_rate=0.004)
model.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model
history = model.fit(X_train, y_train, epochs=159, batch_size=15, validation_split=0.7, verbose=1)

# Evaluate the model on the test set
loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy: {accuracy*100:.2f} %")
```

```
4/4 [==============================] - 0s 18ms/step - loss: 0.1212 - accuracy: 0.9348 - val_loss: 0.2696 - val_accuracy: 0.8636
Epoch 132/159
4/4 [==============================] - 0s 16ms/step - loss: 0.1283 - accuracy: 0.9565 - val_loss: 0.2716 - val_accuracy: 0.8545
Epoch 133/159
4/4 [==============================] - 0s 18ms/step - loss: 0.1169 - accuracy: 0.9348 - val_loss: 0.2449 - val_accuracy: 0.8909
Epoch 134/159
4/4 [==============================] - 0s 24ms/step - loss: 0.1204 - accuracy: 0.9565 - val_loss: 0.2563 - val_accuracy: 0.8455
Epoch 135/159
4/4 [==============================] - 0s 40ms/step - loss: 0.1418 - accuracy: 0.9565 - val_loss: 0.2724 - val_accuracy: 0.8545
Epoch 136/159
4/4 [==============================] - 0s 18ms/step - loss: 0.1500 - accuracy: 0.9565 - val_loss: 0.2588 - val_accuracy: 0.8545
Epoch 137/159
4/4 [==============================] - 0s 16ms/step - loss: 0.1346 - accuracy: 0.9565 - val_loss: 0.2408 - val_accuracy: 0.8909
Epoch 138/159
4/4 [==============================] - 0s 18ms/step - loss: 0.1030 - accuracy: 0.9565 - val_loss: 0.2732 - val_accuracy: 0.8636
Epoch 139/159
4/4 [==============================] - 0s 15ms/step - loss: 0.1219 - accuracy: 0.9565 - val_loss: 0.3171 - val_accuracy: 0.8455
Epoch 140/159
4/4 [==============================] - 0s 19ms/step - loss: 0.1345 - accuracy: 0.9348 - val_loss: 0.2831 - val_accuracy: 0.8455
Epoch 141/159
4/4 [==============================] - 0s 19ms/step - loss: 0.1025 - accuracy: 0.9565 - val_loss: 0.2370 - val_accuracy: 0.8636
Epoch 142/159
4/4 [==============================] - 0s 19ms/step - loss: 0.1168 - accuracy: 0.9565 - val_loss: 0.2535 - val_accuracy: 0.8455
Epoch 143/159
4/4 [==============================] - 0s 16ms/step - loss: 0.1335 - accuracy: 0.9565 - val_loss: 0.2502 - val_accuracy: 0.8455
Epoch 144/159
4/4 [==============================] - 0s 16ms/step - loss: 0.1288 - accuracy: 0.9565 - val_loss: 0.2346 - val_accuracy: 0.8727
Epoch 145/159
4/4 [==============================] - 0s 19ms/step - loss: 0.1096 - accuracy: 0.9565 - val_loss: 0.2314 - val_accuracy: 0.8818
Epoch 146/159
4/4 [==============================] - 0s 19ms/step - loss: 0.0967 - accuracy: 0.9783 - val_loss: 0.2465 - val_accuracy: 0.8909
Epoch 147/159
4/4 [==============================] - 0s 17ms/step - loss: 0.0981 - accuracy: 0.9783 - val_loss: 0.2662 - val_accuracy: 0.8818
Epoch 148/159
4/4 [==============================] - 0s 23ms/step - loss: 0.0999 - accuracy: 0.9565 - val_loss: 0.2568 - val_accuracy: 0.8727
Epoch 149/159
4/4 [==============================] - 0s 25ms/step - loss: 0.0947 - accuracy: 0.9565 - val_loss: 0.2439 - val_accuracy: 0.8909
Epoch 150/159
4/4 [==============================] - 0s 39ms/step - loss: 0.0951 - accuracy: 0.9565 - val_loss: 0.2439 - val_accuracy: 0.8909
Epoch 151/159
4/4 [==============================] - 0s 16ms/step - loss: 0.0946 - accuracy: 0.9565 - val_loss: 0.2707 - val_accuracy: 0.8455
Epoch 152/159
4/4 [==============================] - 0s 15ms/step - loss: 0.0968 - accuracy: 0.9565 - val_loss: 0.2601 - val_accuracy: 0.8727
Epoch 153/159
4/4 [==============================] - 0s 17ms/step - loss: 0.0947 - accuracy: 0.9348 - val_loss: 0.2613 - val_accuracy: 0.8727
Epoch 154/159
4/4 [==============================] - 0s 17ms/step - loss: 0.0968 - accuracy: 0.9565 - val_loss: 0.2617 - val_accuracy: 0.8727
Epoch 155/159
4/4 [==============================] - 0s 17ms/step - loss: 0.0927 - accuracy: 0.9565 - val_loss: 0.2630 - val_accuracy: 0.8636
Epoch 156/159
4/4 [==============================] - 0s 18ms/step - loss: 0.0906 - accuracy: 0.9348 - val_loss: 0.2653 - val_accuracy: 0.8636
Epoch 157/159
4/4 [==============================] - 0s 16ms/step - loss: 0.0905 - accuracy: 0.9565 - val_loss: 0.2523 - val_accuracy: 0.8909
Epoch 158/159
4/4 [==============================] - 0s 19ms/step - loss: 0.0838 - accuracy: 0.9783 - val_loss: 0.2291 - val_accuracy: 0.8909
Epoch 159/159
4/4 [==============================] - 0s 19ms/step - loss: 0.0840 - accuracy: 0.9783 - val_loss: 0.2166 - val_accuracy: 0.9091
Test accuracy: 89.74 %
```

```
# Plot the results
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

axes[0].set_title(f'Base model')
axes[0].set_xlabel('Epochs')
axes[0].set_ylabel('Loss')
axes[0].plot(history.history['loss'], label='Training Loss')
axes[0].plot(history.history['val_loss'], label='Validation Loss')
axes[0].legend()
axes[0].grid(True)

axes[1].plot(history.history['accuracy'], label='Training Accuracy')
axes[1].plot(history.history['val_accuracy'], label='Validation Accuracy')
axes[1].set_title('Model Accuracy')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.show()
```



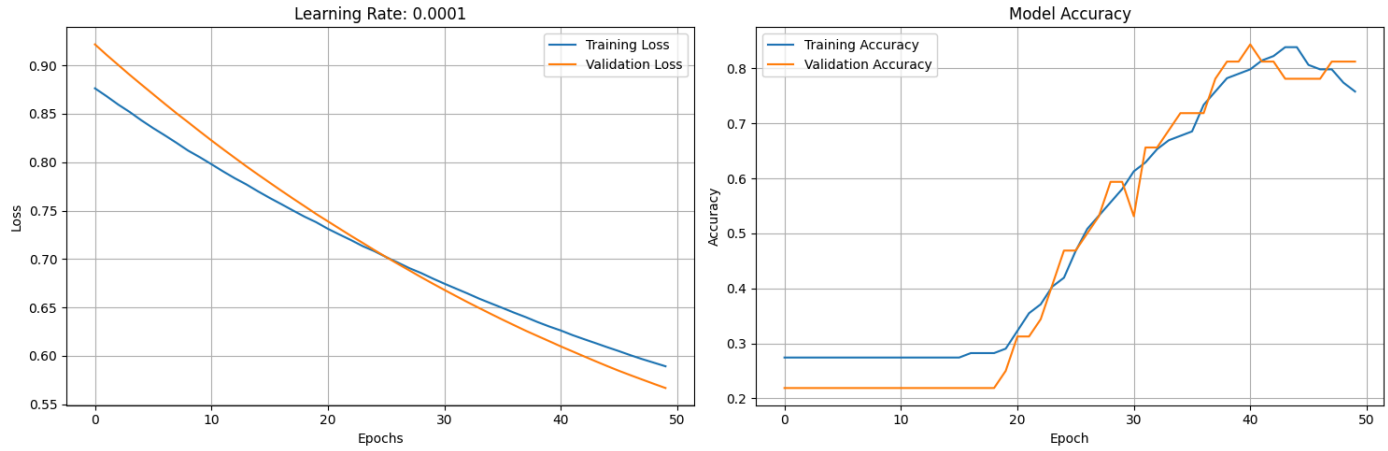
LEARNING RATE MODEL

```
# List of learning rates to evaluate
learning_rates = 0.0001
lr = learning_rates

print(f"Training with learning rate: {lr}")

# Initialize the model
model0 = create_base_model(X_train.shape[1])

# Compile the model with the current learning rate
optimizer = Adam(learning_rate=lr)
model0.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model
history0 = model0.fit(X_train, y_train, epochs=50, batch_size=32, validation_split=0.2, verbose=1)

# Evaluate the model
loss, accuracy = model0.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with learning rate {lr}: {accuracy:.4f}")

# Print all the results
print("\nResults:")
print("")
print(f"Learning rate:, {lr}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Training with learning rate: 0.0001
Epoch 1/50
4/4 [==============================] - 1s 75ms/step - loss: 0.8763 - accuracy: 0.2742 - val_loss: 0.9218 - val_accuracy: 0.2188
```

```
Epoch 2/50
4/4 [==============================] - 0s 12ms/step - loss: 0.8682 - accuracy: 0.2742 - val_loss: 0.9109 - val_accuracy: 0.2188
Epoch 3/50
4/4 [==============================] - 0s 11ms/step - loss: 0.8596 - accuracy: 0.2742 - val_loss: 0.9005 - val_accuracy: 0.2188
Epoch 4/50
4/4 [==============================] - 0s 12ms/step - loss: 0.8518 - accuracy: 0.2742 - val_loss: 0.8901 - val_accuracy: 0.2188
Epoch 5/50
4/4 [==============================] - 0s 12ms/step - loss: 0.8432 - accuracy: 0.2742 - val_loss: 0.8801 - val_accuracy: 0.2188
Epoch 6/50
4/4 [==============================] - 0s 19ms/step - loss: 0.8352 - accuracy: 0.2742 - val_loss: 0.8702 - val_accuracy: 0.2188
Epoch 7/50
4/4 [==============================] - 0s 17ms/step - loss: 0.8278 - accuracy: 0.2742 - val_loss: 0.8603 - val_accuracy: 0.2188
Epoch 8/50
4/4 [==============================] - 0s 14ms/step - loss: 0.8201 - accuracy: 0.2742 - val_loss: 0.8506 - val_accuracy: 0.2188
Epoch 9/50
4/4 [==============================] - 0s 18ms/step - loss: 0.8120 - accuracy: 0.2742 - val_loss: 0.8413 - val_accuracy: 0.2188
Epoch 10/50
4/4 [==============================] - 0s 11ms/step - loss: 0.8052 - accuracy: 0.2742 - val_loss: 0.8319 - val_accuracy: 0.2188
Epoch 11/50
4/4 [==============================] - 0s 13ms/step - loss: 0.7979 - accuracy: 0.2742 - val_loss: 0.8226 - val_accuracy: 0.2188
Epoch 12/50
4/4 [==============================] - 0s 12ms/step - loss: 0.7905 - accuracy: 0.2742 - val_loss: 0.8135 - val_accuracy: 0.2188
Epoch 13/50
4/4 [==============================] - 0s 12ms/step - loss: 0.7835 - accuracy: 0.2742 - val_loss: 0.8046 - val_accuracy: 0.2188
Epoch 14/50
4/4 [==============================] - 0s 18ms/step - loss: 0.7771 - accuracy: 0.2742 - val_loss: 0.7958 - val_accuracy: 0.2188
Epoch 15/50
4/4 [==============================] - 0s 20ms/step - loss: 0.7699 - accuracy: 0.2742 - val_loss: 0.7873 - val_accuracy: 0.2188
Epoch 16/50
4/4 [==============================] - 0s 17ms/step - loss: 0.7631 - accuracy: 0.2742 - val_loss: 0.7789 - val_accuracy: 0.2188
Epoch 17/50
4/4 [==============================] - 0s 17ms/step - loss: 0.7567 - accuracy: 0.2823 - val_loss: 0.7706 - val_accuracy: 0.2188
Epoch 18/50
4/4 [==============================] - 0s 18ms/step - loss: 0.7501 - accuracy: 0.2823 - val_loss: 0.7625 - val_accuracy: 0.2188
Epoch 19/50
4/4 [==============================] - 0s 11ms/step - loss: 0.7437 - accuracy: 0.2823 - val_loss: 0.7545 - val_accuracy: 0.2188
Epoch 20/50
4/4 [==============================] - 0s 18ms/step - loss: 0.7380 - accuracy: 0.2903 - val_loss: 0.7465 - val_accuracy: 0.2500
Epoch 21/50
4/4 [==============================] - 0s 18ms/step - loss: 0.7312 - accuracy: 0.3226 - val_loss: 0.7389 - val_accuracy: 0.3125
Epoch 22/50
4/4 [==============================] - 0s 18ms/step - loss: 0.7252 - accuracy: 0.3548 - val_loss: 0.7314 - val_accuracy: 0.3125
Epoch 23/50
4/4 [==============================] - 0s 13ms/step - loss: 0.7195 - accuracy: 0.3710 - val_loss: 0.7238 - val_accuracy: 0.3438
Epoch 24/50
4/4 [==============================] - 0s 21ms/step - loss: 0.7132 - accuracy: 0.4032 - val_loss: 0.7166 - val_accuracy: 0.4062
Epoch 25/50
4/4 [==============================] - 0s 22ms/step - loss: 0.7080 - accuracy: 0.4194 - val_loss: 0.7092 - val_accuracy: 0.4688
Epoch 26/50
4/4 [==============================] - 0s 21ms/step - loss: 0.7019 - accuracy: 0.4677 - val_loss: 0.7020 - val_accuracy: 0.4688
Epoch 27/50
4/4 [==============================] - 0s 21ms/step - loss: 0.6963 - accuracy: 0.5081 - val_loss: 0.6950 - val_accuracy: 0.5000
Epoch 28/50
4/4 [==============================] - 0s 16ms/step - loss: 0.6905 - accuracy: 0.5323 - val_loss: 0.6882 - val_accuracy: 0.5312
Epoch 29/50
```

```
# Plot the results
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

axes[0].set_title(f'Learning Rate: {lr}')
axes[0].set_xlabel('Epochs')
axes[0].set_ylabel('Loss')
axes[0].plot(history0.history['loss'], label='Training Loss')
axes[0].plot(history0.history['val_loss'], label='Validation Loss')
axes[0].legend()
axes[0].grid(True)

axes[1].plot(history0.history['accuracy'], label='Training Accuracy')
axes[1].plot(history0.history['val_accuracy'], label='Validation Accuracy')
axes[1].set_title('Model Accuracy')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.show()
```




```
# MODEL 1 : LEARNING RATE 0.01
# List of learning rates to evaluate
learning_rates = 0.01
lr = learning_rates

print(f"Training with learning rate: {lr}")

# Initialize the model
model_1 = create_base_model(X_train.shape[1])

# Compile the model with the current learning rate
optimizer = Adam(learning_rate=lr)
model_1.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model
history_1 = model_1.fit(X_train, y_train, epochs=50, batch_size=32, validation_split=0.2, verbose=1)

# Evaluate the model
loss, accuracy = model_1.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with learning rate {lr}: {accuracy:.4f}")

# Print all the results
print("\nResults:")
print("")
print(f"Learning rate:, {lr}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Training with learning rate: 0.01
Epoch 1/50
4/4 [==============================] - 1s 107ms/step - loss: 0.5178 - accuracy: 0.7339 - val_loss: 0.5082 - val_accuracy: 0.7812
Epoch 2/50
4/4 [==============================] - 0s 23ms/step - loss: 0.4582 - accuracy: 0.7661 - val_loss: 0.4300 - val_accuracy: 0.7188
Epoch 3/50
4/4 [==============================] - 0s 19ms/step - loss: 0.5078 - accuracy: 0.8306 - val_loss: 0.4333 - val_accuracy: 0.8438
Epoch 4/50
4/4 [==============================] - 0s 21ms/step - loss: 0.3294 - accuracy: 0.8387 - val_loss: 0.4558 - val_accuracy: 0.7188
Epoch 5/50
4/4 [==============================] - 0s 22ms/step - loss: 0.3275 - accuracy: 0.8629 - val_loss: 0.4152 - val_accuracy: 0.7500
Epoch 6/50
4/4 [==============================] - 0s 24ms/step - loss: 0.3036 - accuracy: 0.8468 - val_loss: 0.3887 - val_accuracy: 0.8438
Epoch 7/50
4/4 [==============================] - 0s 22ms/step - loss: 0.2841 - accuracy: 0.8548 - val_loss: 0.3850 - val_accuracy: 0.7500
Epoch 8/50
4/4 [==============================] - 0s 24ms/step - loss: 0.2831 - accuracy: 0.8710 - val_loss: 0.3638 - val_accuracy: 0.7812
Epoch 9/50
4/4 [==============================] - 0s 22ms/step - loss: 0.2706 - accuracy: 0.8790 - val_loss: 0.3241 - val_accuracy: 0.8438
Epoch 10/50
4/4 [==============================] - 0s 23ms/step - loss: 0.2601 - accuracy: 0.8629 - val_loss: 0.3231 - val_accuracy: 0.8438
Epoch 11/50
4/4 [==============================] - 0s 23ms/step - loss: 0.2438 - accuracy: 0.8952 - val_loss: 0.3199 - val_accuracy: 0.8125
Epoch 12/50
4/4 [==============================] - 0s 16ms/step - loss: 0.2392 - accuracy: 0.8952 - val_loss: 0.2968 - val_accuracy: 0.8438
Epoch 13/50
4/4 [==============================] - 0s 17ms/step - loss: 0.2289 - accuracy: 0.8871 - val_loss: 0.2839 - val_accuracy: 0.8438
Epoch 14/50
4/4 [==============================] - 0s 15ms/step - loss: 0.2231 - accuracy: 0.9032 - val_loss: 0.2628 - val_accuracy: 0.8750
```

```
Epoch 15/50
4/4 [==============================] - 0s 23ms/step - loss: 0.2130 - accuracy: 0.9032 - val_loss: 0.2770 - val_accuracy: 0.8438
Epoch 16/50
4/4 [==============================] - 0s 22ms/step - loss: 0.2052 - accuracy: 0.9032 - val_loss: 0.2686 - val_accuracy: 0.8438
Epoch 17/50
4/4 [==============================] - 0s 24ms/step - loss: 0.1985 - accuracy: 0.8952 - val_loss: 0.2407 - val_accuracy: 0.8438
Epoch 18/50
4/4 [==============================] - 0s 18ms/step - loss: 0.1872 - accuracy: 0.9113 - val_loss: 0.2404 - val_accuracy: 0.8438
Epoch 19/50
4/4 [==============================] - 0s 22ms/step - loss: 0.1770 - accuracy: 0.9032 - val_loss: 0.2344 - val_accuracy: 0.8438
Epoch 20/50
4/4 [==============================] - 0s 29ms/step - loss: 0.1739 - accuracy: 0.9032 - val_loss: 0.2136 - val_accuracy: 0.8438
Epoch 21/50
4/4 [==============================] - 0s 20ms/step - loss: 0.1733 - accuracy: 0.9032 - val_loss: 0.2281 - val_accuracy: 0.8750
Epoch 22/50
4/4 [==============================] - 0s 20ms/step - loss: 0.1746 - accuracy: 0.9194 - val_loss: 0.2036 - val_accuracy: 0.8750
Epoch 23/50
4/4 [==============================] - 0s 21ms/step - loss: 0.1403 - accuracy: 0.9194 - val_loss: 0.2301 - val_accuracy: 0.8750
Epoch 24/50
4/4 [==============================] - 0s 17ms/step - loss: 0.1386 - accuracy: 0.9355 - val_loss: 0.1513 - val_accuracy: 0.9375
Epoch 25/50
4/4 [==============================] - 0s 26ms/step - loss: 0.1273 - accuracy: 0.9516 - val_loss: 0.2362 - val_accuracy: 0.8750
Epoch 26/50
4/4 [==============================] - 0s 17ms/step - loss: 0.1459 - accuracy: 0.9194 - val_loss: 0.1929 - val_accuracy: 0.8750
Epoch 27/50
4/4 [==============================] - 0s 22ms/step - loss: 0.1239 - accuracy: 0.9597 - val_loss: 0.1319 - val_accuracy: 0.9375
Epoch 28/50
4/4 [==============================] - 0s 19ms/step - loss: 0.1150 - accuracy: 0.9435 - val_loss: 0.3142 - val_accuracy: 0.8750
Epoch 29/50
```

```
# MODEL 2 : LEARNING RATE 0.02
# List of learning rates to evaluate
learning_rates = 0.02
lr = learning_rates

print(f"Training with learning rate: {lr}")

# Initialize the model
model_2 = create_base_model(X_train.shape[1])

# Compile the model with the current learning rate
optimizer = Adam(learning_rate=lr)
model_2.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model
history_2 = model_2.fit(X_train, y_train, epochs=50, batch_size=32, validation_split=0.2, verbose=1)

# Evaluate the model
loss, accuracy = model_2.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with learning rate {lr}: {accuracy:.4f}")

# Print all the results
print("\nResults:")
print("")
print(f"Learning rate:, {lr}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Training with learning rate: 0.02
Epoch 1/50
4/4 [==============================] - 2s 71ms/step - loss: 8.7756 - accuracy: 0.2742 - val_loss: 0.8673 - val_accuracy: 0.3750
Epoch 2/50
4/4 [==============================] - 0s 12ms/step - loss: 0.5971 - accuracy: 0.6613 - val_loss: 0.4857 - val_accuracy: 0.7812
Epoch 3/50
4/4 [==============================] - 0s 17ms/step - loss: 0.4827 - accuracy: 0.7339 - val_loss: 0.4410 - val_accuracy: 0.7812
Epoch 4/50
4/4 [==============================] - 0s 18ms/step - loss: 0.3727 - accuracy: 0.8387 - val_loss: 0.4330 - val_accuracy: 0.7500
Epoch 5/50
4/4 [==============================] - 0s 12ms/step - loss: 0.3340 - accuracy: 0.8629 - val_loss: 0.4517 - val_accuracy: 0.7500
Epoch 6/50
4/4 [==============================] - 0s 13ms/step - loss: 0.3297 - accuracy: 0.8629 - val_loss: 0.4122 - val_accuracy: 0.7500
Epoch 7/50
4/4 [==============================] - 0s 13ms/step - loss: 0.3237 - accuracy: 0.8548 - val_loss: 0.4209 - val_accuracy: 0.7812
Epoch 8/50
4/4 [==============================] - 0s 17ms/step - loss: 0.2918 - accuracy: 0.8710 - val_loss: 0.3856 - val_accuracy: 0.8125
Epoch 9/50
4/4 [==============================] - 0s 12ms/step - loss: 0.2906 - accuracy: 0.8548 - val_loss: 0.3885 - val_accuracy: 0.7812
Epoch 10/50
4/4 [==============================] - 0s 25ms/step - loss: 0.2999 - accuracy: 0.8952 - val_loss: 0.3682 - val_accuracy: 0.7812
Epoch 11/50
4/4 [==============================] - 0s 24ms/step - loss: 0.2809 - accuracy: 0.8952 - val_loss: 0.3459 - val_accuracy: 0.8125
Epoch 12/50
4/4 [==============================] - 0s 29ms/step - loss: 0.2736 - accuracy: 0.8790 - val_loss: 0.3827 - val_accuracy: 0.8125
```

```
Epoch 13/50
4/4 [==============================] - 0s 26ms/step - loss: 0.2817 - accuracy: 0.9032 - val_loss: 0.3257 - val_accuracy: 0.8125
Epoch 14/50
4/4 [==============================] - 0s 21ms/step - loss: 0.2672 - accuracy: 0.8710 - val_loss: 0.3236 - val_accuracy: 0.8438
Epoch 15/50
4/4 [==============================] - 0s 17ms/step - loss: 0.2506 - accuracy: 0.8871 - val_loss: 0.3319 - val_accuracy: 0.8125
Epoch 16/50
4/4 [==============================] - 0s 16ms/step - loss: 0.2480 - accuracy: 0.9032 - val_loss: 0.3212 - val_accuracy: 0.8125
Epoch 17/50
4/4 [==============================] - 0s 18ms/step - loss: 0.2496 - accuracy: 0.8710 - val_loss: 0.3117 - val_accuracy: 0.8125
Epoch 18/50
4/4 [==============================] - 0s 26ms/step - loss: 0.2603 - accuracy: 0.8871 - val_loss: 0.3443 - val_accuracy: 0.8438
Epoch 19/50
4/4 [==============================] - 0s 22ms/step - loss: 0.2509 - accuracy: 0.8710 - val_loss: 0.2915 - val_accuracy: 0.8438
Epoch 20/50
4/4 [==============================] - 0s 23ms/step - loss: 0.2489 - accuracy: 0.8952 - val_loss: 0.3534 - val_accuracy: 0.8438
Epoch 21/50
4/4 [==============================] - 0s 20ms/step - loss: 0.2390 - accuracy: 0.8952 - val_loss: 0.2876 - val_accuracy: 0.8438
Epoch 22/50
4/4 [==============================] - 0s 30ms/step - loss: 0.2255 - accuracy: 0.8952 - val_loss: 0.3018 - val_accuracy: 0.8125
Epoch 23/50
4/4 [==============================] - 0s 20ms/step - loss: 0.2150 - accuracy: 0.9032 - val_loss: 0.2872 - val_accuracy: 0.8125
Epoch 24/50
4/4 [==============================] - 0s 25ms/step - loss: 0.2083 - accuracy: 0.9113 - val_loss: 0.2817 - val_accuracy: 0.8125
Epoch 25/50
4/4 [==============================] - 0s 15ms/step - loss: 0.2021 - accuracy: 0.9113 - val_loss: 0.2719 - val_accuracy: 0.8125
Epoch 26/50
4/4 [==============================] - 0s 24ms/step - loss: 0.1995 - accuracy: 0.8871 - val_loss: 0.2870 - val_accuracy: 0.7812
Epoch 27/50
4/4 [==============================] - 0s 21ms/step - loss: 0.1911 - accuracy: 0.9032 - val_loss: 0.3022 - val_accuracy: 0.7812
Epoch 28/50
4/4 [==============================] - 0s 24ms/step - loss: 0.1913 - accuracy: 0.9032 - val_loss: 0.2518 - val_accuracy: 0.8125
Epoch 29/50
```

```
# MODEL 3 : LEARNING RATE 0.001
# List of learning rates to evaluate
learning_rates = 0.001
lr = learning_rates

print(f"Training with learning rate: {lr}")

# Initialize the model
model_3 = create_base_model(X_train.shape[1])

# Compile the model with the current learning rate
optimizer = Adam(learning_rate=lr)
model_3.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model
history_3 = model_3.fit(X_train, y_train, epochs=50, batch_size=32, validation_split=0.2, verbose=1)

# Evaluate the model
loss, accuracy = model_3.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with learning rate {lr}: {accuracy:.4f}")

# Print all the results
print("\nResults:")
print("")
print(f"Learning rate:, {lr}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Training with learning rate: 0.001
Epoch 1/50
4/4 [==============================] - 1s 84ms/step - loss: 3.5050 - accuracy: 0.2984 - val_loss: 1.3543 - val_accuracy: 0.2812
Epoch 2/50
4/4 [==============================] - 0s 20ms/step - loss: 1.2601 - accuracy: 0.3629 - val_loss: 0.9921 - val_accuracy: 0.3750
Epoch 3/50
4/4 [==============================] - 0s 22ms/step - loss: 0.9222 - accuracy: 0.4274 - val_loss: 0.8233 - val_accuracy: 0.5312
Epoch 4/50
4/4 [==============================] - 0s 14ms/step - loss: 0.7805 - accuracy: 0.4839 - val_loss: 0.7237 - val_accuracy: 0.5312
Epoch 5/50
4/4 [==============================] - 0s 13ms/step - loss: 0.6958 - accuracy: 0.5403 - val_loss: 0.6585 - val_accuracy: 0.5938
Epoch 6/50
4/4 [==============================] - 0s 20ms/step - loss: 0.6391 - accuracy: 0.5806 - val_loss: 0.6145 - val_accuracy: 0.5625
Epoch 7/50
4/4 [==============================] - 0s 19ms/step - loss: 0.5973 - accuracy: 0.6532 - val_loss: 0.5839 - val_accuracy: 0.5938
Epoch 8/50
4/4 [==============================] - 0s 19ms/step - loss: 0.5696 - accuracy: 0.6935 - val_loss: 0.5610 - val_accuracy: 0.6250
Epoch 9/50
4/4 [==============================] - 0s 13ms/step - loss: 0.5456 - accuracy: 0.7258 - val_loss: 0.5433 - val_accuracy: 0.6875
Epoch 10/50
4/4 [==============================] - 0s 19ms/step - loss: 0.5276 - accuracy: 0.7661 - val_loss: 0.5296 - val_accuracy: 0.7500
```

```
Epoch 11/50
4/4 [==============================] - 0s 15ms/step - loss: 0.5119 - accuracy: 0.7823 - val_loss: 0.5188 - val_accuracy: 0.7812
Epoch 12/50
4/4 [==============================] - 0s 14ms/step - loss: 0.5000 - accuracy: 0.8145 - val_loss: 0.5102 - val_accuracy: 0.7500
Epoch 13/50
4/4 [==============================] - 0s 18ms/step - loss: 0.4865 - accuracy: 0.8226 - val_loss: 0.5041 - val_accuracy: 0.7500
Epoch 14/50
4/4 [==============================] - 0s 20ms/step - loss: 0.4763 - accuracy: 0.7903 - val_loss: 0.4989 - val_accuracy: 0.7500
Epoch 15/50
4/4 [==============================] - 0s 24ms/step - loss: 0.4670 - accuracy: 0.7823 - val_loss: 0.4948 - val_accuracy: 0.7500
Epoch 16/50
4/4 [==============================] - 0s 13ms/step - loss: 0.4583 - accuracy: 0.7903 - val_loss: 0.4910 - val_accuracy: 0.7500
Epoch 17/50
4/4 [==============================] - 0s 13ms/step - loss: 0.4499 - accuracy: 0.7984 - val_loss: 0.4881 - val_accuracy: 0.7500
Epoch 18/50
4/4 [==============================] - 0s 20ms/step - loss: 0.4423 - accuracy: 0.7984 - val_loss: 0.4859 - val_accuracy: 0.7188
Epoch 19/50
4/4 [==============================] - 0s 18ms/step - loss: 0.4358 - accuracy: 0.7984 - val_loss: 0.4845 - val_accuracy: 0.7188
Epoch 20/50
4/4 [==============================] - 0s 25ms/step - loss: 0.4277 - accuracy: 0.8065 - val_loss: 0.4833 - val_accuracy: 0.7188
Epoch 21/50
4/4 [==============================] - 0s 22ms/step - loss: 0.4213 - accuracy: 0.8065 - val_loss: 0.4820 - val_accuracy: 0.7188
Epoch 22/50
4/4 [==============================] - 0s 23ms/step - loss: 0.4153 - accuracy: 0.8145 - val_loss: 0.4811 - val_accuracy: 0.7188
Epoch 23/50
4/4 [==============================] - 0s 38ms/step - loss: 0.4081 - accuracy: 0.8145 - val_loss: 0.4804 - val_accuracy: 0.7188
Epoch 24/50
4/4 [==============================] - 0s 29ms/step - loss: 0.4019 - accuracy: 0.8145 - val_loss: 0.4803 - val_accuracy: 0.7188
Epoch 25/50
4/4 [==============================] - 0s 24ms/step - loss: 0.3957 - accuracy: 0.8145 - val_loss: 0.4795 - val_accuracy: 0.7188
Epoch 26/50
4/4 [==============================] - 0s 24ms/step - loss: 0.3892 - accuracy: 0.8226 - val_loss: 0.4786 - val_accuracy: 0.7188
Epoch 27/50
4/4 [==============================] - 0s 26ms/step - loss: 0.3837 - accuracy: 0.8226 - val_loss: 0.4770 - val_accuracy: 0.7188
Epoch 28/50
4/4 [==============================] - 0s 23ms/step - loss: 0.3789 - accuracy: 0.8226 - val_loss: 0.4753 - val_accuracy: 0.7188
Epoch 29/50
```

```
# MODEL 4 : LEARNING RATE 0.03
# List of learning rates to evaluate
learning_rates = 0.03
lr = learning_rates

print(f"Training with learning rate: {lr}")

# Initialize the model
model_4 = create_base_model(X_train.shape[1])

# Compile the model with the current learning rate
optimizer = Adam(learning_rate=lr)
model_4.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model
history_4 = model_4.fit(X_train, y_train, epochs=50, batch_size=32, validation_split=0.2, verbose=1)

# Evaluate the model
loss, accuracy = model_4.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with learning rate {lr}: {accuracy:.4f}")

# Print all the results
print("\nResults:")
print("")
print(f"Learning rate:, {lr}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Training with learning rate: 0.03
Epoch 1/50
4/4 [==============================] - 3s 210ms/step - loss: 1.7153 - accuracy: 0.5484 - val_loss: 0.6817 - val_accuracy: 0.7812
Epoch 2/50
4/4 [==============================] - 0s 20ms/step - loss: 0.6334 - accuracy: 0.7258 - val_loss: 0.4600 - val_accuracy: 0.7188
Epoch 3/50
4/4 [==============================] - 0s 25ms/step - loss: 0.4291 - accuracy: 0.7823 - val_loss: 0.5506 - val_accuracy: 0.7812
Epoch 4/50
4/4 [==============================] - 0s 23ms/step - loss: 0.4661 - accuracy: 0.8065 - val_loss: 0.5403 - val_accuracy: 0.7812
Epoch 5/50
4/4 [==============================] - 0s 24ms/step - loss: 0.3658 - accuracy: 0.8226 - val_loss: 0.4773 - val_accuracy: 0.7500
Epoch 6/50
4/4 [==============================] - 0s 31ms/step - loss: 0.4793 - accuracy: 0.7903 - val_loss: 0.4879 - val_accuracy: 0.7812
Epoch 7/50
4/4 [==============================] - 0s 28ms/step - loss: 0.3729 - accuracy: 0.8468 - val_loss: 0.4761 - val_accuracy: 0.7812
Epoch 8/50
```

```
4/4 [==============================] - 0s 29ms/step - loss: 0.3013 - accuracy: 0.8790 - val_loss: 0.4031 - val_accuracy: 0.8125
Epoch 9/50
4/4 [==============================] - 0s 24ms/step - loss: 0.3286 - accuracy: 0.7984 - val_loss: 0.3837 - val_accuracy: 0.7500
Epoch 10/50
4/4 [==============================] - 0s 24ms/step - loss: 0.3009 - accuracy: 0.8710 - val_loss: 0.4570 - val_accuracy: 0.8125
Epoch 11/50
4/4 [==============================] - 0s 27ms/step - loss: 0.3030 - accuracy: 0.8871 - val_loss: 0.3431 - val_accuracy: 0.8125
Epoch 12/50
4/4 [==============================] - 0s 24ms/step - loss: 0.2812 - accuracy: 0.8710 - val_loss: 0.3279 - val_accuracy: 0.8438
Epoch 13/50
4/4 [==============================] - 0s 23ms/step - loss: 0.2688 - accuracy: 0.8790 - val_loss: 0.3652 - val_accuracy: 0.8125
Epoch 14/50
4/4 [==============================] - 0s 24ms/step - loss: 0.2603 - accuracy: 0.9032 - val_loss: 0.3261 - val_accuracy: 0.8125
Epoch 15/50
4/4 [==============================] - 0s 26ms/step - loss: 0.2402 - accuracy: 0.8790 - val_loss: 0.3067 - val_accuracy: 0.8438
Epoch 16/50
4/4 [==============================] - 0s 59ms/step - loss: 0.2469 - accuracy: 0.8629 - val_loss: 0.3068 - val_accuracy: 0.8125
Epoch 17/50
4/4 [==============================] - 0s 27ms/step - loss: 0.2307 - accuracy: 0.9032 - val_loss: 0.3061 - val_accuracy: 0.8438
Epoch 18/50
4/4 [==============================] - 0s 14ms/step - loss: 0.2262 - accuracy: 0.9194 - val_loss: 0.2908 - val_accuracy: 0.8125
Epoch 19/50
4/4 [==============================] - 0s 11ms/step - loss: 0.2109 - accuracy: 0.8952 - val_loss: 0.2840 - val_accuracy: 0.8125
Epoch 20/50
4/4 [==============================] - 0s 11ms/step - loss: 0.1996 - accuracy: 0.8790 - val_loss: 0.2613 - val_accuracy: 0.8438
Epoch 21/50
4/4 [==============================] - 0s 17ms/step - loss: 0.1860 - accuracy: 0.9274 - val_loss: 0.2658 - val_accuracy: 0.8125
Epoch 22/50
4/4 [==============================] - 0s 13ms/step - loss: 0.1775 - accuracy: 0.9355 - val_loss: 0.2114 - val_accuracy: 0.9062
Epoch 23/50
4/4 [==============================] - 0s 21ms/step - loss: 0.1780 - accuracy: 0.9194 - val_loss: 0.3253 - val_accuracy: 0.8125
Epoch 24/50
4/4 [==============================] - 0s 16ms/step - loss: 0.1978 - accuracy: 0.9032 - val_loss: 0.2326 - val_accuracy: 0.8438
Epoch 25/50
4/4 [==============================] - 0s 14ms/step - loss: 0.1671 - accuracy: 0.9355 - val_loss: 0.1900 - val_accuracy: 0.9062
Epoch 26/50
4/4 [==============================] - 0s 14ms/step - loss: 0.1450 - accuracy: 0.9435 - val_loss: 0.2584 - val_accuracy: 0.8438
Epoch 27/50
4/4 [==============================] - 0s 14ms/step - loss: 0.1247 - accuracy: 0.9597 - val_loss: 0.1822 - val_accuracy: 0.9375
Epoch 28/50
4/4 [==============================] - 0s 19ms/step - loss: 0.1288 - accuracy: 0.9435 - val_loss: 0.2567 - val_accuracy: 0.8750
```

```
# MODEL 5 : LEARNING RATE 0.002
# List of learning rates to evaluate
learning_rates = 0.002
lr = learning_rates

print(f"Training with learning rate: {lr}")

# Initialize the model
model_5 = create_base_model(X_train.shape[1])

# Compile the model with the current learning rate
optimizer = Adam(learning_rate=lr)
model_5.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model
history_5 = model_5.fit(X_train, y_train, epochs=50, batch_size=32, validation_split=0.2, verbose=1)

# Evaluate the model
loss, accuracy = model_5.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with learning rate {lr}: {accuracy:.4f}")

# Print all the results
print("\nResults:")
print("")
print(f"Learning rate:, {lr}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Training with learning rate: 0.002
Epoch 1/50
4/4 [==============================] - 3s 160ms/step - loss: 0.5463 - accuracy: 0.7581 - val_loss: 0.5411 - val_accuracy: 0.6562
Epoch 2/50
4/4 [==============================] - 0s 57ms/step - loss: 0.4664 - accuracy: 0.8065 - val_loss: 0.5199 - val_accuracy: 0.7812
Epoch 3/50
4/4 [==============================] - 0s 35ms/step - loss: 0.4462 - accuracy: 0.8226 - val_loss: 0.5208 - val_accuracy: 0.7812
Epoch 4/50
4/4 [==============================] - 0s 40ms/step - loss: 0.4369 - accuracy: 0.8306 - val_loss: 0.5148 - val_accuracy: 0.7812
Epoch 5/50
4/4 [==============================] - 0s 45ms/step - loss: 0.4157 - accuracy: 0.8468 - val_loss: 0.4902 - val_accuracy: 0.7812
Epoch 6/50
```

```
4/4 [==============================] - 0s 69ms/step - loss: 0.3818 - accuracy: 0.8548 - val_loss: 0.4673 - val_accuracy: 0.7812
Epoch 7/50
4/4 [==============================] - 0s 34ms/step - loss: 0.3614 - accuracy: 0.8548 - val_loss: 0.4471 - val_accuracy: 0.7188
Epoch 8/50
4/4 [==============================] - 0s 22ms/step - loss: 0.3455 - accuracy: 0.8387 - val_loss: 0.4313 - val_accuracy: 0.7188
Epoch 9/50
4/4 [==============================] - 0s 14ms/step - loss: 0.3315 - accuracy: 0.8306 - val_loss: 0.4204 - val_accuracy: 0.7188
Epoch 10/50
4/4 [==============================] - 0s 19ms/step - loss: 0.3194 - accuracy: 0.8306 - val_loss: 0.4131 - val_accuracy: 0.7188
Epoch 11/50
4/4 [==============================] - 0s 14ms/step - loss: 0.3085 - accuracy: 0.8468 - val_loss: 0.4105 - val_accuracy: 0.7188
Epoch 12/50
4/4 [==============================] - 0s 15ms/step - loss: 0.3011 - accuracy: 0.8710 - val_loss: 0.4047 - val_accuracy: 0.7188
Epoch 13/50
4/4 [==============================] - 0s 14ms/step - loss: 0.2947 - accuracy: 0.8710 - val_loss: 0.3892 - val_accuracy: 0.7500
Epoch 14/50
4/4 [==============================] - 0s 20ms/step - loss: 0.2860 - accuracy: 0.8790 - val_loss: 0.3843 - val_accuracy: 0.7500
Epoch 15/50
4/4 [==============================] - 0s 21ms/step - loss: 0.2814 - accuracy: 0.8790 - val_loss: 0.3779 - val_accuracy: 0.7500
Epoch 16/50
4/4 [==============================] - 0s 20ms/step - loss: 0.2758 - accuracy: 0.8790 - val_loss: 0.3696 - val_accuracy: 0.7500
Epoch 17/50
4/4 [==============================] - 0s 14ms/step - loss: 0.2705 - accuracy: 0.8952 - val_loss: 0.3548 - val_accuracy: 0.7500
Epoch 18/50
4/4 [==============================] - 0s 14ms/step - loss: 0.2660 - accuracy: 0.8952 - val_loss: 0.3478 - val_accuracy: 0.7500
Epoch 19/50
4/4 [==============================] - 0s 29ms/step - loss: 0.2615 - accuracy: 0.8952 - val_loss: 0.3400 - val_accuracy: 0.7812
Epoch 20/50
4/4 [==============================] - 0s 31ms/step - loss: 0.2575 - accuracy: 0.8952 - val_loss: 0.3277 - val_accuracy: 0.8125
Epoch 21/50
4/4 [==============================] - 0s 29ms/step - loss: 0.2507 - accuracy: 0.8871 - val_loss: 0.3288 - val_accuracy: 0.8125
Epoch 22/50
4/4 [==============================] - 0s 31ms/step - loss: 0.2468 - accuracy: 0.9032 - val_loss: 0.3247 - val_accuracy: 0.8125
Epoch 23/50
4/4 [==============================] - 0s 27ms/step - loss: 0.2431 - accuracy: 0.9032 - val_loss: 0.3206 - val_accuracy: 0.8125
Epoch 24/50
4/4 [==============================] - 0s 24ms/step - loss: 0.2386 - accuracy: 0.8952 - val_loss: 0.3070 - val_accuracy: 0.8125
Epoch 25/50
4/4 [==============================] - 0s 25ms/step - loss: 0.2348 - accuracy: 0.8952 - val_loss: 0.3013 - val_accuracy: 0.8125
Epoch 26/50
4/4 [==============================] - 0s 20ms/step - loss: 0.2313 - accuracy: 0.9032 - val_loss: 0.3054 - val_accuracy: 0.8438
Epoch 27/50
4/4 [==============================] - 0s 25ms/step - loss: 0.2337 - accuracy: 0.9032 - val_loss: 0.2942 - val_accuracy: 0.8125
Epoch 28/50
4/4 [==============================] - 0s 22ms/step - loss: 0.2256 - accuracy: 0.8952 - val_loss: 0.3013 - val_accuracy: 0.8438
Epoch 29/50
```

```
# MODEL 6 : LEARNING RATE 0.12
# List of learning rates to evaluate
learning_rates = 0.12
lr = learning_rates

print(f"Training with learning rate: {lr}")

# Initialize the model
model_6 = create_base_model(X_train.shape[1])

# Compile the model with the current learning rate
optimizer = Adam(learning_rate=lr)
model_6.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model
history_6 = model_6.fit(X_train, y_train, epochs=50, batch_size=32, validation_split=0.2, verbose=1)

# Evaluate the model
loss, accuracy = model_2.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with learning rate {lr}: {accuracy:.4f}")

# Print all the results
print("\nResults:")
print("")
print(f"Learning rate:, {lr}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Training with learning rate: 0.12
Epoch 1/50
4/4 [==============================] - 3s 275ms/step - loss: 11.1955 - accuracy: 0.2742 - val_loss: 12.0507 - val_accuracy: 0.2188
Epoch 2/50
4/4 [==============================] - 0s 23ms/step - loss: 11.1955 - accuracy: 0.2742 - val_loss: 12.0507 - val_accuracy: 0.2188
Epoch 3/50
4/4 [==============================] - 0s 25ms/step - loss: 11.1955 - accuracy: 0.2742 - val_loss: 12.0507 - val_accuracy: 0.2188
Epoch 4/50
```

```
4/4 [==============================] - 0s 23ms/step - loss: 11.1955 - accuracy: 0.2742 - val_loss: 12.0507 - val_accuracy: 0.2188
Epoch 5/50
4/4 [==============================] - 0s 28ms/step - loss: 11.1955 - accuracy: 0.2742 - val_loss: 12.0507 - val_accuracy: 0.2188
Epoch 6/50
4/4 [==============================] - 0s 12ms/step - loss: 11.1955 - accuracy: 0.2742 - val_loss: 12.0507 - val_accuracy: 0.2188
Epoch 7/50
4/4 [==============================] - 0s 11ms/step - loss: 11.1955 - accuracy: 0.2742 - val_loss: 12.0507 - val_accuracy: 0.2188
Epoch 8/50
4/4 [==============================] - 0s 12ms/step - loss: 11.1955 - accuracy: 0.2742 - val_loss: 12.0507 - val_accuracy: 0.2188
Epoch 9/50
4/4 [==============================] - 0s 11ms/step - loss: 11.1955 - accuracy: 0.2742 - val_loss: 12.0507 - val_accuracy: 0.2188
Epoch 10/50
4/4 [==============================] - 0s 12ms/step - loss: 11.1955 - accuracy: 0.2742 - val_loss: 12.0507 - val_accuracy: 0.2188
Epoch 11/50
4/4 [==============================] - 0s 19ms/step - loss: 11.1955 - accuracy: 0.2742 - val_loss: 12.0507 - val_accuracy: 0.2188
Epoch 12/50
4/4 [==============================] - 0s 11ms/step - loss: 11.1955 - accuracy: 0.2742 - val_loss: 12.0507 - val_accuracy: 0.2188
Epoch 13/50
4/4 [==============================] - 0s 15ms/step - loss: 11.1955 - accuracy: 0.2742 - val_loss: 12.0507 - val_accuracy: 0.2188
Epoch 14/50
4/4 [==============================] - 0s 14ms/step - loss: 11.1955 - accuracy: 0.2742 - val_loss: 12.0507 - val_accuracy: 0.2188
Epoch 15/50
4/4 [==============================] - 0s 17ms/step - loss: 11.1955 - accuracy: 0.2742 - val_loss: 12.0507 - val_accuracy: 0.2188
Epoch 16/50
4/4 [==============================] - 0s 18ms/step - loss: 11.1955 - accuracy: 0.2742 - val_loss: 12.0507 - val_accuracy: 0.2188
Epoch 17/50
4/4 [==============================] - 0s 14ms/step - loss: 11.1955 - accuracy: 0.2742 - val_loss: 12.0507 - val_accuracy: 0.2188
Epoch 18/50
4/4 [==============================] - 0s 14ms/step - loss: 11.1955 - accuracy: 0.2742 - val_loss: 12.0507 - val_accuracy: 0.2188
Epoch 19/50
4/4 [==============================] - 0s 12ms/step - loss: 11.1955 - accuracy: 0.2742 - val_loss: 12.0507 - val_accuracy: 0.2188
Epoch 20/50
4/4 [==============================] - 0s 13ms/step - loss: 11.1955 - accuracy: 0.2742 - val_loss: 12.0507 - val_accuracy: 0.2188
Epoch 21/50
4/4 [==============================] - 0s 17ms/step - loss: 11.1955 - accuracy: 0.2742 - val_loss: 12.0507 - val_accuracy: 0.2188
Epoch 22/50
4/4 [==============================] - 0s 12ms/step - loss: 11.1955 - accuracy: 0.2742 - val_loss: 12.0507 - val_accuracy: 0.2188
Epoch 23/50
4/4 [==============================] - 0s 18ms/step - loss: 11.1955 - accuracy: 0.2742 - val_loss: 12.0507 - val_accuracy: 0.2188
Epoch 24/50
4/4 [==============================] - 0s 12ms/step - loss: 11.1955 - accuracy: 0.2742 - val_loss: 12.0507 - val_accuracy: 0.2188
Epoch 25/50
4/4 [==============================] - 0s 12ms/step - loss: 11.1955 - accuracy: 0.2742 - val_loss: 12.0507 - val_accuracy: 0.2188
Epoch 26/50
4/4 [==============================] - 0s 12ms/step - loss: 11.1955 - accuracy: 0.2742 - val_loss: 12.0507 - val_accuracy: 0.2188
Epoch 27/50
4/4 [==============================] - 0s 14ms/step - loss: 11.1955 - accuracy: 0.2742 - val_loss: 12.0507 - val_accuracy: 0.2188
Epoch 28/50
4/4 [==============================] - 0s 12ms/step - loss: 11.1955 - accuracy: 0.2742 - val_loss: 12.0507 - val_accuracy: 0.2188
Epoch 29/50
```

```
# MODEL 7 : LEARNING RATE 0.012
# List of learning rates to evaluate
learning_rates = 0.012
lr = learning_rates

print(f"Training with learning rate: {lr}")

# Initialize the model
model_7 = create_base_model(X_train.shape[1])

# Compile the model with the current learning rate
optimizer = Adam(learning_rate=lr)
model_7.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model
history_7 = model_7.fit(X_train, y_train, epochs=50, batch_size=32, validation_split=0.2, verbose=1)

# Evaluate the model
loss, accuracy = model_7.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with learning rate {lr}: {accuracy:.4f}")

# Print all the results
print("\nResults:")
print("")
print(f"Learning rate:, {lr}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Training with learning rate: 0.012
Epoch 1/50
4/4 [==============================] - 1s 69ms/step - loss: 7.0066 - accuracy: 0.2742 - val_loss: 0.9273 - val_accuracy: 0.3125
Epoch 2/50
```

```
4/4 [==============================] - 0s 11ms/step - loss: 0.6675 - accuracy: 0.5565 - val_loss: 0.4557 - val_accuracy: 0.7812
Epoch 3/50
4/4 [==============================] - 0s 11ms/step - loss: 0.4894 - accuracy: 0.7258 - val_loss: 0.5067 - val_accuracy: 0.7812
Epoch 4/50
4/4 [==============================] - 0s 11ms/step - loss: 0.4777 - accuracy: 0.7258 - val_loss: 0.4386 - val_accuracy: 0.7500
Epoch 5/50
4/4 [==============================] - 0s 11ms/step - loss: 0.3594 - accuracy: 0.8468 - val_loss: 0.4476 - val_accuracy: 0.7500
Epoch 6/50
4/4 [==============================] - 0s 22ms/step - loss: 0.3790 - accuracy: 0.8145 - val_loss: 0.4200 - val_accuracy: 0.7500
Epoch 7/50
4/4 [==============================] - 0s 19ms/step - loss: 0.3299 - accuracy: 0.8629 - val_loss: 0.4443 - val_accuracy: 0.7188
Epoch 8/50
4/4 [==============================] - 0s 12ms/step - loss: 0.3091 - accuracy: 0.8548 - val_loss: 0.4229 - val_accuracy: 0.7500
Epoch 9/50
4/4 [==============================] - 0s 17ms/step - loss: 0.3080 - accuracy: 0.8548 - val_loss: 0.4169 - val_accuracy: 0.7500
Epoch 10/50
4/4 [==============================] - 0s 17ms/step - loss: 0.2935 - accuracy: 0.8548 - val_loss: 0.4166 - val_accuracy: 0.7500
Epoch 11/50
4/4 [==============================] - 0s 12ms/step - loss: 0.2899 - accuracy: 0.8629 - val_loss: 0.3943 - val_accuracy: 0.7500
Epoch 12/50
4/4 [==============================] - 0s 11ms/step - loss: 0.2859 - accuracy: 0.8468 - val_loss: 0.3665 - val_accuracy: 0.7812
Epoch 13/50
4/4 [==============================] - 0s 11ms/step - loss: 0.2726 - accuracy: 0.8710 - val_loss: 0.3640 - val_accuracy: 0.7812
Epoch 14/50
4/4 [==============================] - 0s 12ms/step - loss: 0.2658 - accuracy: 0.8790 - val_loss: 0.3415 - val_accuracy: 0.8125
Epoch 15/50
4/4 [==============================] - 0s 12ms/step - loss: 0.2599 - accuracy: 0.8871 - val_loss: 0.3277 - val_accuracy: 0.8125
Epoch 16/50
4/4 [==============================] - 0s 12ms/step - loss: 0.2487 - accuracy: 0.8871 - val_loss: 0.3144 - val_accuracy: 0.8125
Epoch 17/50
4/4 [==============================] - 0s 12ms/step - loss: 0.2476 - accuracy: 0.8710 - val_loss: 0.3090 - val_accuracy: 0.8125
Epoch 18/50
4/4 [==============================] - 0s 11ms/step - loss: 0.2412 - accuracy: 0.8871 - val_loss: 0.2974 - val_accuracy: 0.8125
Epoch 19/50
4/4 [==============================] - 0s 11ms/step - loss: 0.2342 - accuracy: 0.9032 - val_loss: 0.2988 - val_accuracy: 0.8125
Epoch 20/50
4/4 [==============================] - 0s 11ms/step - loss: 0.2259 - accuracy: 0.9032 - val_loss: 0.2906 - val_accuracy: 0.8438
Epoch 21/50
4/4 [==============================] - 0s 12ms/step - loss: 0.2264 - accuracy: 0.8871 - val_loss: 0.2923 - val_accuracy: 0.8438
Epoch 22/50
4/4 [==============================] - 0s 11ms/step - loss: 0.2246 - accuracy: 0.9032 - val_loss: 0.3125 - val_accuracy: 0.7812
Epoch 23/50
4/4 [==============================] - 0s 11ms/step - loss: 0.2141 - accuracy: 0.9113 - val_loss: 0.2850 - val_accuracy: 0.8125
Epoch 24/50
4/4 [==============================] - 0s 18ms/step - loss: 0.2113 - accuracy: 0.8790 - val_loss: 0.2884 - val_accuracy: 0.7812
Epoch 25/50
4/4 [==============================] - 0s 16ms/step - loss: 0.2033 - accuracy: 0.9032 - val_loss: 0.2885 - val_accuracy: 0.7812
Epoch 26/50
4/4 [==============================] - 0s 26ms/step - loss: 0.2020 - accuracy: 0.8710 - val_loss: 0.2861 - val_accuracy: 0.8125
Epoch 27/50
4/4 [==============================] - 0s 16ms/step - loss: 0.1993 - accuracy: 0.9032 - val_loss: 0.3193 - val_accuracy: 0.8438
Epoch 28/50
4/4 [==============================] - 0s 16ms/step - loss: 0.1871 - accuracy: 0.9274 - val_loss: 0.2660 - val_accuracy: 0.7812
Epoch 29/50
```

```
# MODEL 8 : LEARNING RATE 0.019
# List of learning rates to evaluate
learning_rates = 0.019
lr = learning_rates

print(f"Training with learning rate: {lr}")

# Initialize the model
model_8 = create_base_model(X_train.shape[1])

# Compile the model with the current learning rate
optimizer = Adam(learning_rate=lr)
model_8.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model
history_8 = model_8.fit(X_train, y_train, epochs=50, batch_size=32, validation_split=0.2, verbose=1)

# Evaluate the model
loss, accuracy = model_8.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with learning rate {lr}: {accuracy:.4f}")

# Print all the results
print("\nResults:")
print("")
print(f"Learning rate:, {lr}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Training with learning rate: 0.019
Epoch 1/50
4/4 [==============================] - 1s 79ms/step - loss: 1.5465 - accuracy: 0.5323 - val_loss: 0.5096 - val_accuracy: 0.7812
Epoch 2/50
4/4 [==============================] - 0s 13ms/step - loss: 0.5744 - accuracy: 0.7258 - val_loss: 0.4646 - val_accuracy: 0.7812
Epoch 3/50
4/4 [==============================] - 0s 14ms/step - loss: 0.4449 - accuracy: 0.7419 - val_loss: 0.4586 - val_accuracy: 0.7500
Epoch 4/50
4/4 [==============================] - 0s 19ms/step - loss: 0.3832 - accuracy: 0.8145 - val_loss: 0.4532 - val_accuracy: 0.7188
Epoch 5/50
4/4 [==============================] - 0s 20ms/step - loss: 0.3600 - accuracy: 0.8468 - val_loss: 0.4472 - val_accuracy: 0.7500
Epoch 6/50
4/4 [==============================] - 0s 12ms/step - loss: 0.3285 - accuracy: 0.8468 - val_loss: 0.4511 - val_accuracy: 0.7500
Epoch 7/50
4/4 [==============================] - 0s 11ms/step - loss: 0.3181 - accuracy: 0.8629 - val_loss: 0.4366 - val_accuracy: 0.7500
Epoch 8/50
4/4 [==============================] - 0s 13ms/step - loss: 0.3161 - accuracy: 0.8548 - val_loss: 0.3871 - val_accuracy: 0.7500
Epoch 9/50
4/4 [==============================] - 0s 19ms/step - loss: 0.3090 - accuracy: 0.8710 - val_loss: 0.3979 - val_accuracy: 0.7812
Epoch 10/50
4/4 [==============================] - 0s 21ms/step - loss: 0.2792 - accuracy: 0.8871 - val_loss: 0.3415 - val_accuracy: 0.8438
Epoch 11/50
4/4 [==============================] - 0s 15ms/step - loss: 0.2875 - accuracy: 0.8468 - val_loss: 0.3412 - val_accuracy: 0.8125
Epoch 12/50
4/4 [==============================] - 0s 15ms/step - loss: 0.2754 - accuracy: 0.9032 - val_loss: 0.3360 - val_accuracy: 0.8125
Epoch 13/50
4/4 [==============================] - 0s 22ms/step - loss: 0.2657 - accuracy: 0.9032 - val_loss: 0.3070 - val_accuracy: 0.8438
Epoch 14/50
4/4 [==============================] - 0s 18ms/step - loss: 0.2636 - accuracy: 0.8629 - val_loss: 0.3128 - val_accuracy: 0.8438
Epoch 15/50
4/4 [==============================] - 0s 18ms/step - loss: 0.2575 - accuracy: 0.8952 - val_loss: 0.3470 - val_accuracy: 0.7812
Epoch 16/50
4/4 [==============================] - 0s 12ms/step - loss: 0.2509 - accuracy: 0.8790 - val_loss: 0.3130 - val_accuracy: 0.8125
Epoch 17/50
4/4 [==============================] - 0s 18ms/step - loss: 0.2592 - accuracy: 0.8629 - val_loss: 0.3258 - val_accuracy: 0.8125
Epoch 18/50
4/4 [==============================] - 0s 18ms/step - loss: 0.2468 - accuracy: 0.8952 - val_loss: 0.2792 - val_accuracy: 0.8438
Epoch 19/50
4/4 [==============================] - 0s 19ms/step - loss: 0.2424 - accuracy: 0.8871 - val_loss: 0.3327 - val_accuracy: 0.8125
Epoch 20/50
4/4 [==============================] - 0s 11ms/step - loss: 0.2359 - accuracy: 0.8952 - val_loss: 0.2652 - val_accuracy: 0.8438
Epoch 21/50
4/4 [==============================] - 0s 11ms/step - loss: 0.2234 - accuracy: 0.8871 - val_loss: 0.3303 - val_accuracy: 0.8438
Epoch 22/50
4/4 [==============================] - 0s 15ms/step - loss: 0.2309 - accuracy: 0.9113 - val_loss: 0.2634 - val_accuracy: 0.7812
Epoch 23/50
4/4 [==============================] - 0s 12ms/step - loss: 0.2128 - accuracy: 0.9032 - val_loss: 0.2784 - val_accuracy: 0.7812
Epoch 24/50
4/4 [==============================] - 0s 12ms/step - loss: 0.2024 - accuracy: 0.8871 - val_loss: 0.3328 - val_accuracy: 0.8438
Epoch 25/50
4/4 [==============================] - 0s 12ms/step - loss: 0.1999 - accuracy: 0.8790 - val_loss: 0.2317 - val_accuracy: 0.8750
Epoch 26/50
4/4 [==============================] - 0s 18ms/step - loss: 0.2063 - accuracy: 0.8871 - val_loss: 0.3142 - val_accuracy: 0.8438
Epoch 27/50
4/4 [==============================] - 0s 18ms/step - loss: 0.2102 - accuracy: 0.9032 - val_loss: 0.1960 - val_accuracy: 0.8750
Epoch 28/50
4/4 [==============================] - 0s 22ms/step - loss: 0.2121 - accuracy: 0.8710 - val_loss: 0.3161 - val_accuracy: 0.8750
Epoch 29/50
```

```
# MODEL 9 : LEARNING RATE 0.011
# List of learning rates to evaluate
learning_rates = 0.011
lr = learning_rates

print(f"Training with learning rate: {lr}")

# Initialize the model
model_9 = create_base_model(X_train.shape[1])

# Compile the model with the current learning rate
optimizer = Adam(learning_rate=lr)
model_9.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model
history_9 = model_9.fit(X_train, y_train, epochs=50, batch_size=32, validation_split=0.2, verbose=1)

# Evaluate the model
loss, accuracy = model_9.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with learning rate {lr}: {accuracy:.4f}")

# Print all the results
print("\nResults:")
print("")
print(f"Learning rate:, {lr}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Training with learning rate: 0.011
Epoch 1/50
4/4 [==============================] - 2s 76ms/step - loss: 3.6603 - accuracy: 0.4516 - val_loss: 0.4366 - val_accuracy: 0.7812
Epoch 2/50
4/4 [==============================] - 0s 16ms/step - loss: 0.4891 - accuracy: 0.7419 - val_loss: 0.5397 - val_accuracy: 0.7812
Epoch 3/50
4/4 [==============================] - 0s 11ms/step - loss: 0.5312 - accuracy: 0.7339 - val_loss: 0.5063 - val_accuracy: 0.7812
Epoch 4/50
4/4 [==============================] - 0s 11ms/step - loss: 0.4126 - accuracy: 0.8145 - val_loss: 0.4439 - val_accuracy: 0.7500
Epoch 5/50
4/4 [==============================] - 0s 13ms/step - loss: 0.3650 - accuracy: 0.8306 - val_loss: 0.4540 - val_accuracy: 0.7188
Epoch 6/50
4/4 [==============================] - 0s 12ms/step - loss: 0.4625 - accuracy: 0.8065 - val_loss: 0.4531 - val_accuracy: 0.7188
Epoch 7/50
4/4 [==============================] - 0s 18ms/step - loss: 0.3391 - accuracy: 0.8629 - val_loss: 0.5052 - val_accuracy: 0.7812
Epoch 8/50
4/4 [==============================] - 0s 14ms/step - loss: 0.3413 - accuracy: 0.8629 - val_loss: 0.4509 - val_accuracy: 0.7188
Epoch 9/50
4/4 [==============================] - 0s 15ms/step - loss: 0.4223 - accuracy: 0.8629 - val_loss: 0.3988 - val_accuracy: 0.7500
Epoch 10/50
4/4 [==============================] - 0s 26ms/step - loss: 0.3323 - accuracy: 0.8468 - val_loss: 0.4164 - val_accuracy: 0.7812
Epoch 11/50
4/4 [==============================] - 0s 20ms/step - loss: 0.3875 - accuracy: 0.8629 - val_loss: 0.5953 - val_accuracy: 0.7812
Epoch 12/50
4/4 [==============================] - 0s 16ms/step - loss: 0.5219 - accuracy: 0.7581 - val_loss: 0.5600 - val_accuracy: 0.7812
Epoch 13/50
4/4 [==============================] - 0s 14ms/step - loss: 0.4776 - accuracy: 0.7581 - val_loss: 0.4183 - val_accuracy: 0.7812
Epoch 14/50
4/4 [==============================] - 0s 14ms/step - loss: 0.3800 - accuracy: 0.8145 - val_loss: 0.3401 - val_accuracy: 0.8438
Epoch 15/50
4/4 [==============================] - 0s 21ms/step - loss: 0.3397 - accuracy: 0.8952 - val_loss: 0.3380 - val_accuracy: 0.8125
Epoch 16/50
4/4 [==============================] - 0s 24ms/step - loss: 0.3152 - accuracy: 0.8790 - val_loss: 0.3393 - val_accuracy: 0.7812
Epoch 17/50
4/4 [==============================] - 0s 27ms/step - loss: 0.2909 - accuracy: 0.8871 - val_loss: 0.3389 - val_accuracy: 0.7812
Epoch 18/50
4/4 [==============================] - 0s 35ms/step - loss: 0.2776 - accuracy: 0.8871 - val_loss: 0.3346 - val_accuracy: 0.8125
Epoch 19/50
4/4 [==============================] - 0s 20ms/step - loss: 0.2641 - accuracy: 0.8710 - val_loss: 0.3537 - val_accuracy: 0.7812
Epoch 20/50
4/4 [==============================] - 0s 22ms/step - loss: 0.2584 - accuracy: 0.8710 - val_loss: 0.3476 - val_accuracy: 0.7812
Epoch 21/50
4/4 [==============================] - 0s 25ms/step - loss: 0.2530 - accuracy: 0.8710 - val_loss: 0.3218 - val_accuracy: 0.8125
Epoch 22/50
4/4 [==============================] - 0s 27ms/step - loss: 0.2437 - accuracy: 0.8710 - val_loss: 0.3179 - val_accuracy: 0.8125
Epoch 23/50
4/4 [==============================] - 0s 24ms/step - loss: 0.2433 - accuracy: 0.8790 - val_loss: 0.3018 - val_accuracy: 0.8125
Epoch 24/50
4/4 [==============================] - 0s 24ms/step - loss: 0.2342 - accuracy: 0.8952 - val_loss: 0.3002 - val_accuracy: 0.8438
Epoch 25/50
4/4 [==============================] - 0s 27ms/step - loss: 0.2346 - accuracy: 0.8629 - val_loss: 0.2816 - val_accuracy: 0.8125
Epoch 26/50
4/4 [==============================] - 0s 26ms/step - loss: 0.2228 - accuracy: 0.8871 - val_loss: 0.3105 - val_accuracy: 0.8125
Epoch 27/50
4/4 [==============================] - 0s 28ms/step - loss: 0.2179 - accuracy: 0.8952 - val_loss: 0.2822 - val_accuracy: 0.8125
```

```
Epoch 28/50
4/4 [==============================] - 0s 21ms/step - loss: 0.2246 - accuracy: 0.8468 - val_loss: 0.2619 - val_accuracy: 0.8125
Epoch 29/50
```

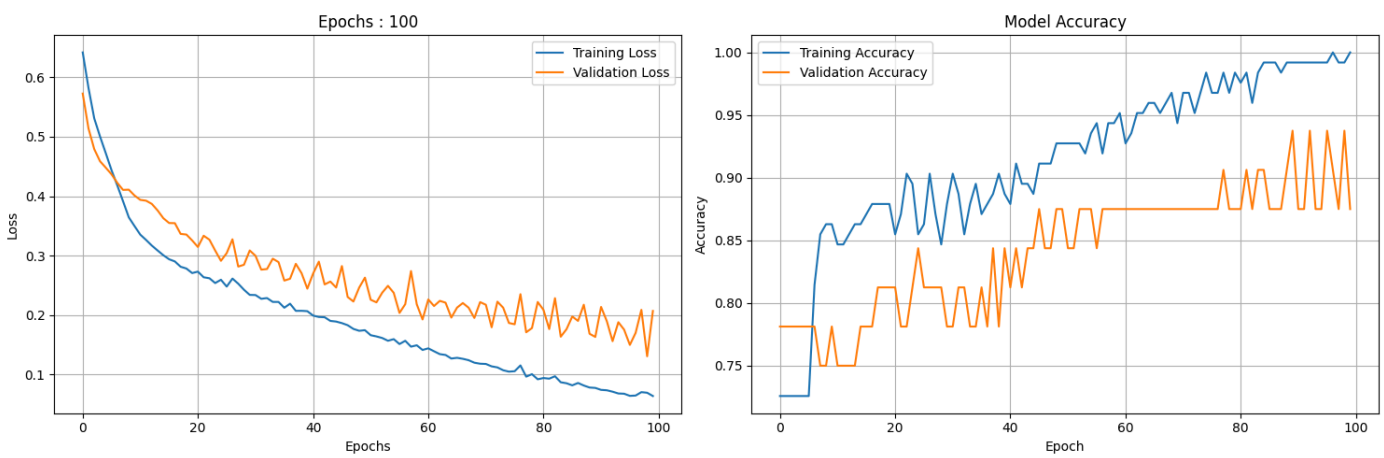
EPOCHS MODEL

```
# MODEL 1
# EPOCH : 100
# learning rate : 0.001
# determine the epoch to evaluate
ep = 100
lr = 0.001
#for ep in epochs:
print(f"Training with epochs: {ep}")

# Initialize the model1
model1 = create_base_model(X_train.shape[1])

# Compile the model with the current epoch
optimizer = Adam(learning_rate=0.001)
model1.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model1
history1 = model1.fit(X_train, y_train, epochs=ep, batch_size=32, validation_split=0.2, verbose=1)

# Evaluate the model1
loss, accuracy = model1.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with epochs {ep}: {accuracy:.4f}")

# Print all the results
print("\nResults Model 1 :")

print(f"epochs: {ep}")
print(f"learning rate: {lr}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Training with epochs: 100
Epoch 1/100
4/4 [==============================] - 1s 67ms/step - loss: 0.8107 - accuracy: 0.1774 - val_loss: 0.7338 - val_accuracy: 0.3750
Epoch 2/100
4/4 [==============================] - 0s 11ms/step - loss: 0.7414 - accuracy: 0.3226 - val_loss: 0.6705 - val_accuracy: 0.5625
Epoch 3/100
4/4 [==============================] - 0s 12ms/step - loss: 0.6868 - accuracy: 0.5484 - val_loss: 0.6173 - val_accuracy: 0.7812
Epoch 4/100
4/4 [==============================] - 0s 12ms/step - loss: 0.6432 - accuracy: 0.7097 - val_loss: 0.5752 - val_accuracy: 0.8438
Epoch 5/100
4/4 [==============================] - 0s 23ms/step - loss: 0.6081 - accuracy: 0.7339 - val_loss: 0.5394 - val_accuracy: 0.8125
Epoch 6/100
4/4 [==============================] - 0s 15ms/step - loss: 0.5775 - accuracy: 0.7258 - val_loss: 0.5080 - val_accuracy: 0.8125
Epoch 7/100
4/4 [==============================] - 0s 13ms/step - loss: 0.5486 - accuracy: 0.7258 - val_loss: 0.4835 - val_accuracy: 0.7812
Epoch 8/100
4/4 [==============================] - 0s 13ms/step - loss: 0.5249 - accuracy: 0.7258 - val_loss: 0.4653 - val_accuracy: 0.7812
Epoch 9/100
4/4 [==============================] - 0s 11ms/step - loss: 0.5073 - accuracy: 0.7258 - val_loss: 0.4512 - val_accuracy: 0.7812
Epoch 10/100
4/4 [==============================] - 0s 11ms/step - loss: 0.4913 - accuracy: 0.7258 - val_loss: 0.4416 - val_accuracy: 0.7812
Epoch 11/100
4/4 [==============================] - 0s 12ms/step - loss: 0.4775 - accuracy: 0.7258 - val_loss: 0.4344 - val_accuracy: 0.7812
Epoch 12/100
4/4 [==============================] - 0s 11ms/step - loss: 0.4649 - accuracy: 0.7258 - val_loss: 0.4283 - val_accuracy: 0.7812
Epoch 13/100
4/4 [==============================] - 0s 11ms/step - loss: 0.4543 - accuracy: 0.7339 - val_loss: 0.4226 - val_accuracy: 0.7812
Epoch 14/100
4/4 [==============================] - 0s 12ms/step - loss: 0.4432 - accuracy: 0.7419 - val_loss: 0.4174 - val_accuracy: 0.7812
Epoch 15/100
4/4 [==============================] - 0s 12ms/step - loss: 0.4346 - accuracy: 0.7581 - val_loss: 0.4128 - val_accuracy: 0.7812
Epoch 16/100
4/4 [==============================] - 0s 11ms/step - loss: 0.4233 - accuracy: 0.7823 - val_loss: 0.4085 - val_accuracy: 0.7812
Epoch 17/100
4/4 [==============================] - 0s 11ms/step - loss: 0.4136 - accuracy: 0.8065 - val_loss: 0.4045 - val_accuracy: 0.7812
Epoch 18/100
4/4 [==============================] - 0s 11ms/step - loss: 0.4038 - accuracy: 0.8306 - val_loss: 0.4006 - val_accuracy: 0.7812
Epoch 19/100
4/4 [==============================] - 0s 13ms/step - loss: 0.3944 - accuracy: 0.8468 - val_loss: 0.3966 - val_accuracy: 0.8125
Epoch 20/100
4/4 [==============================] - 0s 13ms/step - loss: 0.3856 - accuracy: 0.8629 - val_loss: 0.3928 - val_accuracy: 0.7812
Epoch 21/100
4/4 [==============================] - 0s 14ms/step - loss: 0.3771 - accuracy: 0.8548 - val_loss: 0.3893 - val_accuracy: 0.7812
```

```
Epoch 22/100
4/4 [==============================] - 0s 12ms/step - loss: 0.3703 - accuracy: 0.8548 - val_loss: 0.3862 - val_accuracy: 0.7812
Epoch 23/100
4/4 [==============================] - 0s 11ms/step - loss: 0.3632 - accuracy: 0.8629 - val_loss: 0.3834 - val_accuracy: 0.7812
Epoch 24/100
4/4 [==============================] - 0s 11ms/step - loss: 0.3569 - accuracy: 0.8629 - val_loss: 0.3812 - val_accuracy: 0.7812
Epoch 25/100
4/4 [==============================] - 0s 11ms/step - loss: 0.3520 - accuracy: 0.8548 - val_loss: 0.3798 - val_accuracy: 0.7812
Epoch 26/100
4/4 [==============================] - 0s 20ms/step - loss: 0.3466 - accuracy: 0.8548 - val_loss: 0.3784 - val_accuracy: 0.7812
Epoch 27/100
4/4 [==============================] - 0s 19ms/step - loss: 0.3420 - accuracy: 0.8548 - val_loss: 0.3769 - val_accuracy: 0.8125
Epoch 28/100
4/4 [==============================] - 0s 15ms/step - loss: 0.3374 - accuracy: 0.8629 - val_loss: 0.3759 - val_accuracy: 0.8125
```

```
# Plot the results
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

axes[0].set_title(f'Epochs : {ep}')
axes[0].set_xlabel('Epochs')
axes[0].set_ylabel('Loss')
axes[0].plot(history1.history['loss'], label='Training Loss')
axes[0].plot(history1.history['val_loss'], label='Validation Loss')
axes[0].legend()
axes[0].grid(True)

axes[1].plot(history1.history['accuracy'], label='Training Accuracy')
axes[1].plot(history1.history['val_accuracy'], label='Validation Accuracy')
axes[1].set_title('Model Accuracy')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.show()
```




```
# MODEL 2
# EPOCHS : 200
# learning rate : 0.001
# determine the epoch to evaluate
ep = 135
lr = 0.001

#for ep in epochs:
print(f"Training with epochs: {ep}")

# Initialize the model1
modelE2 = create_base_model(X_train.shape[1])

# Compile the model with the current epoch
optimizer = Adam(learning_rate=0.001)
modelE2.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model1
```

```
historyE2 = modelE2.fit(X_train, y_train, epochs=ep, batch_size=40, validation_split=0.2, verbose=1)

# Evaluate the model1
loss, accuracy = modelE2.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with epochs {ep}: {accuracy:.4f}")

# Print all the results
print("\nResults Model 2:")

print(f"epochs: {ep}")
print(f"learning rate: {lr}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Training with epochs: 135
Epoch 1/135
4/4 [==============================] - 3s 281ms/step - loss: 0.7315 - accuracy: 0.2903 - val_loss: 0.7113 - val_accuracy: 0.5312
Epoch 2/135
4/4 [==============================] - 0s 58ms/step - loss: 0.6966 - accuracy: 0.4677 - val_loss: 0.6858 - val_accuracy: 0.5625
Epoch 3/135
4/4 [==============================] - 0s 41ms/step - loss: 0.6749 - accuracy: 0.6694 - val_loss: 0.6685 - val_accuracy: 0.6875
Epoch 4/135
4/4 [==============================] - 0s 39ms/step - loss: 0.6609 - accuracy: 0.7581 - val_loss: 0.6540 - val_accuracy: 0.8125
Epoch 5/135
4/4 [==============================] - 0s 45ms/step - loss: 0.6484 - accuracy: 0.7903 - val_loss: 0.6424 - val_accuracy: 0.8125
Epoch 6/135
4/4 [==============================] - 0s 34ms/step - loss: 0.6369 - accuracy: 0.8065 - val_loss: 0.6319 - val_accuracy: 0.8125
Epoch 7/135
4/4 [==============================] - 0s 48ms/step - loss: 0.6270 - accuracy: 0.7903 - val_loss: 0.6214 - val_accuracy: 0.7812
Epoch 8/135
4/4 [==============================] - 0s 36ms/step - loss: 0.6163 - accuracy: 0.7984 - val_loss: 0.6110 - val_accuracy: 0.7812
Epoch 9/135
4/4 [==============================] - 0s 41ms/step - loss: 0.6056 - accuracy: 0.7903 - val_loss: 0.6012 - val_accuracy: 0.7812
Epoch 10/135
4/4 [==============================] - 0s 36ms/step - loss: 0.5945 - accuracy: 0.8065 - val_loss: 0.5904 - val_accuracy: 0.7812
Epoch 11/135
4/4 [==============================] - 0s 50ms/step - loss: 0.5828 - accuracy: 0.8387 - val_loss: 0.5796 - val_accuracy: 0.7812
Epoch 12/135
4/4 [==============================] - 0s 61ms/step - loss: 0.5712 - accuracy: 0.8548 - val_loss: 0.5690 - val_accuracy: 0.7812
Epoch 13/135
4/4 [==============================] - 0s 37ms/step - loss: 0.5595 - accuracy: 0.8548 - val_loss: 0.5586 - val_accuracy: 0.7812
Epoch 14/135
4/4 [==============================] - 0s 32ms/step - loss: 0.5481 - accuracy: 0.8548 - val_loss: 0.5480 - val_accuracy: 0.7812
Epoch 15/135
4/4 [==============================] - 0s 40ms/step - loss: 0.5365 - accuracy: 0.8548 - val_loss: 0.5380 - val_accuracy: 0.7812
Epoch 16/135
4/4 [==============================] - 0s 53ms/step - loss: 0.5258 - accuracy: 0.8710 - val_loss: 0.5294 - val_accuracy: 0.7812
Epoch 17/135
4/4 [==============================] - 0s 65ms/step - loss: 0.5157 - accuracy: 0.8790 - val_loss: 0.5216 - val_accuracy: 0.7812
Epoch 18/135
4/4 [==============================] - 0s 71ms/step - loss: 0.5056 - accuracy: 0.8790 - val_loss: 0.5132 - val_accuracy: 0.7812
Epoch 19/135
4/4 [==============================] - 0s 86ms/step - loss: 0.4955 - accuracy: 0.8790 - val_loss: 0.5056 - val_accuracy: 0.7812
Epoch 20/135
4/4 [==============================] - 0s 55ms/step - loss: 0.4850 - accuracy: 0.8871 - val_loss: 0.4987 - val_accuracy: 0.7812
Epoch 21/135
4/4 [==============================] - 0s 66ms/step - loss: 0.4761 - accuracy: 0.8790 - val_loss: 0.4916 - val_accuracy: 0.7812
Epoch 22/135
4/4 [==============================] - 0s 53ms/step - loss: 0.4660 - accuracy: 0.8790 - val_loss: 0.4850 - val_accuracy: 0.7812
Epoch 23/135
4/4 [==============================] - 0s 61ms/step - loss: 0.4570 - accuracy: 0.8790 - val_loss: 0.4789 - val_accuracy: 0.7812
Epoch 24/135
4/4 [==============================] - 0s 61ms/step - loss: 0.4475 - accuracy: 0.8790 - val_loss: 0.4735 - val_accuracy: 0.7812
Epoch 25/135
4/4 [==============================] - 0s 53ms/step - loss: 0.4392 - accuracy: 0.8790 - val_loss: 0.4689 - val_accuracy: 0.7812
Epoch 26/135
4/4 [==============================] - 0s 38ms/step - loss: 0.4311 - accuracy: 0.8790 - val_loss: 0.4644 - val_accuracy: 0.7812
Epoch 27/135
4/4 [==============================] - 0s 33ms/step - loss: 0.4220 - accuracy: 0.8790 - val_loss: 0.4600 - val_accuracy: 0.7812
Epoch 28/135
4/4 [==============================] - 0s 34ms/step - loss: 0.4151 - accuracy: 0.8790 - val_loss: 0.4550 - val_accuracy: 0.7812
Epoch 29/135
```

```
# MODEL 3
# EPOCHS : 350
# learning rate : 0.3
# determine the epoch to evaluate
ep = 350
lr = 0.3

#for ep in epochs:
print(f"Training with epochs: {ep}")

# Initialize the model1
modelE3 = create_base_model(X_train.shape[1])

# Compile the model with the current epoch
optimizer = Adam(learning_rate=0.3)
modelE3.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model1
historyE3 = modelE3.fit(X_train, y_train, epochs=ep, batch_size=32, validation_split=0.2, verbose=1)

# Evaluate the model1
loss, accuracy = modelE3.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with epochs {ep}: {accuracy:.4f}")

# Print all the results
print("\nResults Model 3:")

print(f"epochs: {ep}")
print(f"learning rate: {lr}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Training with epochs: 350
Epoch 1/350
4/4 [==============================] - 3s 220ms/step - loss: 0.7325 - accuracy: 0.6613 - val_loss: 0.5832 - val_accuracy: 0.7812
Epoch 2/350
4/4 [==============================] - 0s 60ms/step - loss: 0.5993 - accuracy: 0.7258 - val_loss: 0.5315 - val_accuracy: 0.7812
Epoch 3/350
4/4 [==============================] - 0s 48ms/step - loss: 0.5876 - accuracy: 0.7258 - val_loss: 0.5254 - val_accuracy: 0.7812
Epoch 4/350
4/4 [==============================] - 0s 47ms/step - loss: 0.6008 - accuracy: 0.7258 - val_loss: 0.5254 - val_accuracy: 0.7812
Epoch 5/350
4/4 [==============================] - 0s 54ms/step - loss: 0.5948 - accuracy: 0.7258 - val_loss: 0.5341 - val_accuracy: 0.7812
Epoch 6/350
4/4 [==============================] - 0s 57ms/step - loss: 0.5891 - accuracy: 0.7258 - val_loss: 0.5471 - val_accuracy: 0.7812
Epoch 7/350
4/4 [==============================] - 0s 31ms/step - loss: 0.5917 - accuracy: 0.7258 - val_loss: 0.5451 - val_accuracy: 0.7812
Epoch 8/350
4/4 [==============================] - 0s 40ms/step - loss: 0.5883 - accuracy: 0.7258 - val_loss: 0.5334 - val_accuracy: 0.7812
Epoch 9/350
4/4 [==============================] - 0s 56ms/step - loss: 0.5865 - accuracy: 0.7258 - val_loss: 0.5287 - val_accuracy: 0.7812
Epoch 10/350
4/4 [==============================] - 0s 51ms/step - loss: 0.5900 - accuracy: 0.7258 - val_loss: 0.5271 - val_accuracy: 0.7812
Epoch 11/350
4/4 [==============================] - 0s 49ms/step - loss: 0.5905 - accuracy: 0.7258 - val_loss: 0.5320 - val_accuracy: 0.7812
Epoch 12/350
4/4 [==============================] - 0s 50ms/step - loss: 0.5875 - accuracy: 0.7258 - val_loss: 0.5344 - val_accuracy: 0.7812
Epoch 13/350
4/4 [==============================] - 0s 59ms/step - loss: 0.5873 - accuracy: 0.7258 - val_loss: 0.5383 - val_accuracy: 0.7812
Epoch 14/350
4/4 [==============================] - 0s 43ms/step - loss: 0.5895 - accuracy: 0.7258 - val_loss: 0.5362 - val_accuracy: 0.7812
Epoch 15/350
4/4 [==============================] - 0s 60ms/step - loss: 0.5878 - accuracy: 0.7258 - val_loss: 0.5340 - val_accuracy: 0.7812
Epoch 16/350
4/4 [==============================] - 0s 76ms/step - loss: 0.5881 - accuracy: 0.7258 - val_loss: 0.5352 - val_accuracy: 0.7812
Epoch 17/350
4/4 [==============================] - 0s 65ms/step - loss: 0.5873 - accuracy: 0.7258 - val_loss: 0.5325 - val_accuracy: 0.7812
Epoch 18/350
4/4 [==============================] - 0s 45ms/step - loss: 0.5903 - accuracy: 0.7258 - val_loss: 0.5338 - val_accuracy: 0.7812
Epoch 19/350
4/4 [==============================] - 0s 49ms/step - loss: 0.5874 - accuracy: 0.7258 - val_loss: 0.5307 - val_accuracy: 0.7812
Epoch 20/350
4/4 [==============================] - 0s 51ms/step - loss: 0.5892 - accuracy: 0.7258 - val_loss: 0.5321 - val_accuracy: 0.7812
Epoch 21/350
4/4 [==============================] - 0s 52ms/step - loss: 0.5886 - accuracy: 0.7258 - val_loss: 0.5327 - val_accuracy: 0.7812
Epoch 22/350
4/4 [==============================] - 0s 52ms/step - loss: 0.5882 - accuracy: 0.7258 - val_loss: 0.5316 - val_accuracy: 0.7812
Epoch 23/350
4/4 [==============================] - 0s 55ms/step - loss: 0.5885 - accuracy: 0.7258 - val_loss: 0.5325 - val_accuracy: 0.7812
Epoch 24/350
4/4 [==============================] - 0s 40ms/step - loss: 0.5886 - accuracy: 0.7258 - val_loss: 0.5325 - val_accuracy: 0.7812
Epoch 25/350
```

```
4/4 [==============================] - 0s 47ms/step - loss: 0.5883 - accuracy: 0.7258 - val_loss: 0.5321 - val_accuracy: 0.7812
Epoch 26/350
4/4 [==============================] - 0s 53ms/step - loss: 0.5881 - accuracy: 0.7258 - val_loss: 0.5344 - val_accuracy: 0.7812
Epoch 27/350
4/4 [==============================] - 0s 38ms/step - loss: 0.5885 - accuracy: 0.7258 - val_loss: 0.5331 - val_accuracy: 0.7812
Epoch 28/350
4/4 [==============================] - 0s 49ms/step - loss: 0.5887 - accuracy: 0.7258 - val_loss: 0.5330 - val_accuracy: 0.7812
```

```
# MODEL 4
# EPOCHS : 410
# learning rate : 0.5
# determine the epoch to evaluate
ep = 148
lr = 0.5

#for ep in epochs:
print(f"Training with epochs: {ep}")

# Initialize the model1
modelE4 = create_base_model(X_train.shape[1])

# Compile the model with the current epoch
optimizer = Adam(learning_rate=0.5)
modelE4.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model1
historyE4 = modelE4.fit(X_train, y_train, epochs=ep, batch_size=32, validation_split=0.2, verbose=1)

# Evaluate the model1
loss, accuracy = modelE4.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with epochs {ep}: {accuracy:.4f}")

# Print all the results
print("\nResults Model 4:")

print(f"epochs: {ep}")
print(f"learning rate: {lr}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Training with epochs: 148
Epoch 1/148
4/4 [==============================] - 2s 118ms/step - loss: 4.5550 - accuracy: 0.5484 - val_loss: 4.6412 - val_accuracy: 0.7812
Epoch 2/148
4/4 [==============================] - 0s 23ms/step - loss: 2.3135 - accuracy: 0.6694 - val_loss: 0.6248 - val_accuracy: 0.7812
Epoch 3/148
4/4 [==============================] - 0s 15ms/step - loss: 0.6328 - accuracy: 0.7258 - val_loss: 0.5819 - val_accuracy: 0.7812
Epoch 4/148
4/4 [==============================] - 0s 12ms/step - loss: 0.5949 - accuracy: 0.7258 - val_loss: 0.5354 - val_accuracy: 0.7812
Epoch 5/148
4/4 [==============================] - 0s 20ms/step - loss: 0.5993 - accuracy: 0.7258 - val_loss: 0.5256 - val_accuracy: 0.7812
Epoch 6/148
4/4 [==============================] - 0s 16ms/step - loss: 0.6007 - accuracy: 0.7258 - val_loss: 0.5257 - val_accuracy: 0.7812
Epoch 7/148
4/4 [==============================] - 0s 14ms/step - loss: 0.5993 - accuracy: 0.7258 - val_loss: 0.5279 - val_accuracy: 0.7812
Epoch 8/148
4/4 [==============================] - 0s 18ms/step - loss: 0.5870 - accuracy: 0.7258 - val_loss: 0.5374 - val_accuracy: 0.7812
Epoch 9/148
4/4 [==============================] - 0s 12ms/step - loss: 0.5890 - accuracy: 0.7258 - val_loss: 0.5448 - val_accuracy: 0.7812
Epoch 10/148
4/4 [==============================] - 0s 11ms/step - loss: 0.5902 - accuracy: 0.7258 - val_loss: 0.5397 - val_accuracy: 0.7812
Epoch 11/148
4/4 [==============================] - 0s 17ms/step - loss: 0.5882 - accuracy: 0.7258 - val_loss: 0.5369 - val_accuracy: 0.7812
Epoch 12/148
4/4 [==============================] - 0s 14ms/step - loss: 0.5966 - accuracy: 0.7258 - val_loss: 0.5276 - val_accuracy: 0.7812
Epoch 13/148
4/4 [==============================] - 0s 21ms/step - loss: 0.5890 - accuracy: 0.7258 - val_loss: 0.5314 - val_accuracy: 0.7812
Epoch 14/148
4/4 [==============================] - 0s 21ms/step - loss: 0.5878 - accuracy: 0.7258 - val_loss: 0.5349 - val_accuracy: 0.7812
Epoch 15/148
4/4 [==============================] - 0s 20ms/step - loss: 0.5908 - accuracy: 0.7258 - val_loss: 0.5405 - val_accuracy: 0.7812
Epoch 16/148
4/4 [==============================] - 0s 13ms/step - loss: 0.5880 - accuracy: 0.7258 - val_loss: 0.5339 - val_accuracy: 0.7812
Epoch 17/148
4/4 [==============================] - 0s 19ms/step - loss: 0.5897 - accuracy: 0.7258 - val_loss: 0.5286 - val_accuracy: 0.7812
Epoch 18/148
4/4 [==============================] - 0s 17ms/step - loss: 0.5882 - accuracy: 0.7258 - val_loss: 0.5308 - val_accuracy: 0.7812
Epoch 19/148
4/4 [==============================] - 0s 11ms/step - loss: 0.5880 - accuracy: 0.7258 - val_loss: 0.5326 - val_accuracy: 0.7812
Epoch 20/148
4/4 [==============================] - 0s 12ms/step - loss: 0.5915 - accuracy: 0.7258 - val_loss: 0.5436 - val_accuracy: 0.7812
Epoch 21/148
```

```
4/4 [==============================] - 0s 11ms/step - loss: 0.5894 - accuracy: 0.7258 - val_loss: 0.5389 - val_accuracy: 0.7812
Epoch 22/148
4/4 [==============================] - 0s 12ms/step - loss: 0.5905 - accuracy: 0.7258 - val_loss: 0.5288 - val_accuracy: 0.7812
Epoch 23/148
4/4 [==============================] - 0s 18ms/step - loss: 0.5908 - accuracy: 0.7258 - val_loss: 0.5278 - val_accuracy: 0.7812
Epoch 24/148
4/4 [==============================] - 0s 15ms/step - loss: 0.5898 - accuracy: 0.7258 - val_loss: 0.5369 - val_accuracy: 0.7812
Epoch 25/148
4/4 [==============================] - 0s 11ms/step - loss: 0.5893 - accuracy: 0.7258 - val_loss: 0.5412 - val_accuracy: 0.7812
Epoch 26/148
4/4 [==============================] - 0s 13ms/step - loss: 0.5892 - accuracy: 0.7258 - val_loss: 0.5337 - val_accuracy: 0.7812
Epoch 27/148
4/4 [==============================] - 0s 11ms/step - loss: 0.5875 - accuracy: 0.7258 - val_loss: 0.5310 - val_accuracy: 0.7812
Epoch 28/148
4/4 [==============================] - 0s 11ms/step - loss: 0.5880 - accuracy: 0.7258 - val_loss: 0.5320 - val_accuracy: 0.7812
Epoch 29/148
```

```
# MODEL 5
# EPOCHS : 490
# learning rate : 0.6
# determine the epoch to evaluate
ep = 157
lr = 0.6

#for ep in epochs:
print(f"Training with epochs: {ep}")

# Initialize the model1
modelE5 = create_base_model(X_train.shape[1])

# Compile the model with the current epoch
optimizer = Adam(learning_rate=0.6)
modelE5.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model1
historyE5 = modelE5.fit(X_train, y_train, epochs=ep, batch_size=32, validation_split=0.2, verbose=1)

# Evaluate the model1
loss, accuracy = modelE5.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with epochs {ep}: {accuracy:.4f}")

# Print all the results
print("\nResults Model 5:")

print(f"epochs: {ep}")
print(f"learning rate: {lr}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Training with epochs: 157
Epoch 1/157
4/4 [==============================] - 4s 781ms/step - loss: 7.8956 - accuracy: 0.4839 - val_loss: 17.1143 - val_accuracy: 0.2188
Epoch 2/157
4/4 [==============================] - 0s 21ms/step - loss: 6.1534 - accuracy: 0.5968 - val_loss: 1.2751 - val_accuracy: 0.7812
Epoch 3/157
4/4 [==============================] - 0s 31ms/step - loss: 1.3386 - accuracy: 0.6452 - val_loss: 4.1877 - val_accuracy: 0.2188
Epoch 4/157
4/4 [==============================] - 0s 29ms/step - loss: 2.3280 - accuracy: 0.5081 - val_loss: 0.5376 - val_accuracy: 0.6250
Epoch 5/157
4/4 [==============================] - 0s 26ms/step - loss: 0.7562 - accuracy: 0.6452 - val_loss: 0.4935 - val_accuracy: 0.7812
Epoch 6/157
4/4 [==============================] - 0s 26ms/step - loss: 0.5931 - accuracy: 0.7258 - val_loss: 0.4546 - val_accuracy: 0.7812
Epoch 7/157
4/4 [==============================] - 0s 26ms/step - loss: 0.5401 - accuracy: 0.7258 - val_loss: 0.4541 - val_accuracy: 0.7812
Epoch 8/157
4/4 [==============================] - 0s 45ms/step - loss: 0.4828 - accuracy: 0.7258 - val_loss: 0.3786 - val_accuracy: 0.7812
Epoch 9/157
4/4 [==============================] - 0s 27ms/step - loss: 0.4038 - accuracy: 0.7258 - val_loss: 0.4075 - val_accuracy: 0.6562
Epoch 10/157
4/4 [==============================] - 0s 27ms/step - loss: 0.4351 - accuracy: 0.7581 - val_loss: 0.6572 - val_accuracy: 0.6875
Epoch 11/157
4/4 [==============================] - 0s 26ms/step - loss: 0.4063 - accuracy: 0.7419 - val_loss: 0.3877 - val_accuracy: 0.7188
Epoch 12/157
4/4 [==============================] - 0s 26ms/step - loss: 0.4807 - accuracy: 0.6452 - val_loss: 0.4250 - val_accuracy: 0.6562
Epoch 13/157
4/4 [==============================] - 0s 27ms/step - loss: 0.4860 - accuracy: 0.6210 - val_loss: 0.3836 - val_accuracy: 0.7812
Epoch 14/157
4/4 [==============================] - 0s 21ms/step - loss: 0.4135 - accuracy: 0.7258 - val_loss: 0.7499 - val_accuracy: 0.7812
Epoch 15/157
4/4 [==============================] - 0s 16ms/step - loss: 0.4886 - accuracy: 0.7258 - val_loss: 0.3609 - val_accuracy: 0.7812
Epoch 16/157
4/4 [==============================] - 0s 17ms/step - loss: 0.4447 - accuracy: 0.7258 - val_loss: 0.4017 - val_accuracy: 0.7812
```

```
Epoch 17/157
4/4 [==============================] - 0s 14ms/step - loss: 0.4697 - accuracy: 0.7258 - val_loss: 0.3816 - val_accuracy: 0.7812
Epoch 18/157
4/4 [==============================] - 0s 14ms/step - loss: 0.4530 - accuracy: 0.7258 - val_loss: 0.3790 - val_accuracy: 0.7812
Epoch 19/157
4/4 [==============================] - 0s 14ms/step - loss: 0.4401 - accuracy: 0.7258 - val_loss: 0.3775 - val_accuracy: 0.7812
Epoch 20/157
4/4 [==============================] - 0s 26ms/step - loss: 0.4358 - accuracy: 0.7258 - val_loss: 0.3764 - val_accuracy: 0.7812
Epoch 21/157
4/4 [==============================] - 0s 20ms/step - loss: 0.4281 - accuracy: 0.7258 - val_loss: 0.3766 - val_accuracy: 0.7812
Epoch 22/157
4/4 [==============================] - 0s 20ms/step - loss: 0.4237 - accuracy: 0.7258 - val_loss: 0.3604 - val_accuracy: 0.7812
Epoch 23/157
4/4 [==============================] - 0s 15ms/step - loss: 0.4126 - accuracy: 0.7258 - val_loss: 0.3630 - val_accuracy: 0.7812
Epoch 24/157
4/4 [==============================] - 0s 16ms/step - loss: 0.4029 - accuracy: 0.7258 - val_loss: 0.3690 - val_accuracy: 0.6875
Epoch 25/157
4/4 [==============================] - 0s 16ms/step - loss: 0.4034 - accuracy: 0.6613 - val_loss: 0.3673 - val_accuracy: 0.7812
Epoch 26/157
4/4 [==============================] - 0s 15ms/step - loss: 0.3932 - accuracy: 0.7258 - val_loss: 0.6478 - val_accuracy: 0.7812
Epoch 27/157
4/4 [==============================] - 0s 15ms/step - loss: 0.4044 - accuracy: 0.7258 - val_loss: 0.3679 - val_accuracy: 0.7812
Epoch 28/157
4/4 [==============================] - 0s 15ms/step - loss: 0.4529 - accuracy: 0.6532 - val_loss: 0.4065 - val_accuracy: 0.7812
Epoch 29/157
```

```
# MODEL 6
# EPOCHS : 550
# learning rate : 0.1
# determine the epoch to evaluate
ep = 131
lr = 0.1

#for ep in epochs:
print(f"Training with epochs: {ep}")

# Initialize the model1
modelE6 = create_base_model(X_train.shape[1])

# Compile the model with the current epoch
optimizer = Adam(learning_rate=0.1)
modelE6.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model1
historyE6 = modelE6.fit(X_train, y_train, epochs=ep, batch_size=32, validation_split=0.2, verbose=1)

# Evaluate the model1
loss, accuracy = modelE6.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with epochs {ep}: {accuracy:.4f}")

# Print all the results
print("\nResults Model 6:")

print(f"epochs: {ep}")
print(f"learning rate: {lr}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Training with epochs: 131
Epoch 1/131
4/4 [==============================] - 1s 82ms/step - loss: 1.1774 - accuracy: 0.3145 - val_loss: 0.6491 - val_accuracy: 0.7812
Epoch 2/131
4/4 [==============================] - 0s 18ms/step - loss: 0.6284 - accuracy: 0.7258 - val_loss: 0.5064 - val_accuracy: 0.7812
Epoch 3/131
4/4 [==============================] - 0s 12ms/step - loss: 0.4328 - accuracy: 0.8065 - val_loss: 0.4872 - val_accuracy: 0.6875
Epoch 4/131
4/4 [==============================] - 0s 18ms/step - loss: 0.4146 - accuracy: 0.7742 - val_loss: 0.4607 - val_accuracy: 0.7500
Epoch 5/131
4/4 [==============================] - 0s 18ms/step - loss: 0.3413 - accuracy: 0.8387 - val_loss: 0.4088 - val_accuracy: 0.7812
Epoch 6/131
4/4 [==============================] - 0s 17ms/step - loss: 0.3032 - accuracy: 0.8468 - val_loss: 0.3557 - val_accuracy: 0.7812
Epoch 7/131
4/4 [==============================] - 0s 21ms/step - loss: 0.2653 - accuracy: 0.8710 - val_loss: 0.3589 - val_accuracy: 0.7812
Epoch 8/131
4/4 [==============================] - 0s 12ms/step - loss: 0.2363 - accuracy: 0.8952 - val_loss: 0.3286 - val_accuracy: 0.8438
Epoch 9/131
4/4 [==============================] - 0s 13ms/step - loss: 0.2070 - accuracy: 0.9113 - val_loss: 0.2785 - val_accuracy: 0.8750
Epoch 10/131
4/4 [==============================] - 0s 12ms/step - loss: 0.2142 - accuracy: 0.9113 - val_loss: 0.3276 - val_accuracy: 0.8125
Epoch 11/131
4/4 [==============================] - 0s 12ms/step - loss: 0.2231 - accuracy: 0.9032 - val_loss: 0.4042 - val_accuracy: 0.8125
Epoch 12/131
4/4 [==============================] - 0s 12ms/step - loss: 0.2248 - accuracy: 0.9194 - val_loss: 0.3317 - val_accuracy: 0.8125
```

```
Epoch 13/131
4/4 [==============================] - 0s 12ms/step - loss: 0.2667 - accuracy: 0.9113 - val_loss: 0.3555 - val_accuracy: 0.8125
Epoch 14/131
4/4 [==============================] - 0s 12ms/step - loss: 0.2612 - accuracy: 0.9032 - val_loss: 0.3716 - val_accuracy: 0.7812
Epoch 15/131
4/4 [==============================] - 0s 13ms/step - loss: 0.2557 - accuracy: 0.8952 - val_loss: 0.3399 - val_accuracy: 0.8125
Epoch 16/131
4/4 [==============================] - 0s 12ms/step - loss: 0.2442 - accuracy: 0.8952 - val_loss: 0.4509 - val_accuracy: 0.8125
Epoch 17/131
4/4 [==============================] - 0s 16ms/step - loss: 0.2748 - accuracy: 0.8871 - val_loss: 0.3062 - val_accuracy: 0.8438
Epoch 18/131
4/4 [==============================] - 0s 12ms/step - loss: 0.1995 - accuracy: 0.9032 - val_loss: 0.3890 - val_accuracy: 0.8125
Epoch 19/131
4/4 [==============================] - 0s 14ms/step - loss: 0.2058 - accuracy: 0.8871 - val_loss: 0.3488 - val_accuracy: 0.8438
Epoch 20/131
4/4 [==============================] - 0s 13ms/step - loss: 0.2618 - accuracy: 0.8871 - val_loss: 0.4034 - val_accuracy: 0.8438
Epoch 21/131
4/4 [==============================] - 0s 13ms/step - loss: 0.2160 - accuracy: 0.9032 - val_loss: 0.3882 - val_accuracy: 0.7812
Epoch 22/131
4/4 [==============================] - 0s 12ms/step - loss: 0.2854 - accuracy: 0.8387 - val_loss: 0.3406 - val_accuracy: 0.8125
Epoch 23/131
4/4 [==============================] - 0s 12ms/step - loss: 0.2597 - accuracy: 0.8629 - val_loss: 0.3485 - val_accuracy: 0.7812
Epoch 24/131
4/4 [==============================] - 0s 12ms/step - loss: 0.2412 - accuracy: 0.8790 - val_loss: 0.5669 - val_accuracy: 0.7812
Epoch 25/131
4/4 [==============================] - 0s 20ms/step - loss: 0.2269 - accuracy: 0.9032 - val_loss: 0.3112 - val_accuracy: 0.8125
Epoch 26/131
4/4 [==============================] - 0s 16ms/step - loss: 0.2140 - accuracy: 0.9113 - val_loss: 0.3409 - val_accuracy: 0.8125
Epoch 27/131
4/4 [==============================] - 0s 22ms/step - loss: 0.2321 - accuracy: 0.9113 - val_loss: 0.3056 - val_accuracy: 0.8125
Epoch 28/131
4/4 [==============================] - 0s 21ms/step - loss: 0.1898 - accuracy: 0.9113 - val_loss: 0.5687 - val_accuracy: 0.8125
Epoch 29/131
```

```
# MODEL 7
# EPOCHS : 600
# learning rate : 0.001
# determine the epoch to evaluate
ep = 125
lr = 0.001

#for ep in epochs:
print(f"Training with epochs: {ep}")

# Initialize the model1
modelE7 = create_base_model(X_train.shape[1])

# Compile the model with the current epoch
optimizer = Adam(learning_rate=0.001)
modelE7.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model1
historyE7 = modelE7.fit(X_train, y_train, epochs=ep, batch_size=32, validation_split=0.7, verbose=1)

# Evaluate the model1
loss, accuracy = modelE7.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with epochs {ep}: {accuracy:.4f}")

# Print all the results
print("\nResults Model 7:")

print(f"epochs: {ep}")
print(f"learning rate: {lr}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Training with epochs: 125
Epoch 1/125
2/2 [==============================] - 1s 240ms/step - loss: 0.6747 - accuracy: 0.5652 - val_loss: 0.6600 - val_accuracy: 0.6545
Epoch 2/125
2/2 [==============================] - 0s 54ms/step - loss: 0.6530 - accuracy: 0.6739 - val_loss: 0.6382 - val_accuracy: 0.7455
Epoch 3/125
2/2 [==============================] - 0s 37ms/step - loss: 0.6325 - accuracy: 0.7609 - val_loss: 0.6189 - val_accuracy: 0.7455
Epoch 4/125
2/2 [==============================] - 0s 43ms/step - loss: 0.6173 - accuracy: 0.7609 - val_loss: 0.6014 - val_accuracy: 0.7636
Epoch 5/125
2/2 [==============================] - 0s 54ms/step - loss: 0.6025 - accuracy: 0.6739 - val_loss: 0.5861 - val_accuracy: 0.7636
Epoch 6/125
2/2 [==============================] - 0s 49ms/step - loss: 0.5915 - accuracy: 0.6957 - val_loss: 0.5725 - val_accuracy: 0.7545
Epoch 7/125
2/2 [==============================] - 0s 56ms/step - loss: 0.5794 - accuracy: 0.6957 - val_loss: 0.5604 - val_accuracy: 0.7545
Epoch 8/125
```

```
2/2 [==============================] - 0s 51ms/step - loss: 0.5690 - accuracy: 0.6957 - val_loss: 0.5492 - val_accuracy: 0.7545
Epoch 9/125
2/2 [==============================] - 0s 37ms/step - loss: 0.5589 - accuracy: 0.6957 - val_loss: 0.5386 - val_accuracy: 0.7545
Epoch 10/125
2/2 [==============================] - 0s 50ms/step - loss: 0.5507 - accuracy: 0.6957 - val_loss: 0.5286 - val_accuracy: 0.7545
Epoch 11/125
2/2 [==============================] - 0s 51ms/step - loss: 0.5435 - accuracy: 0.6957 - val_loss: 0.5197 - val_accuracy: 0.7545
Epoch 12/125
2/2 [==============================] - 0s 51ms/step - loss: 0.5340 - accuracy: 0.6957 - val_loss: 0.5120 - val_accuracy: 0.7545
Epoch 13/125
2/2 [==============================] - 0s 52ms/step - loss: 0.5266 - accuracy: 0.6957 - val_loss: 0.5046 - val_accuracy: 0.7545
Epoch 14/125
2/2 [==============================] - 0s 52ms/step - loss: 0.5201 - accuracy: 0.6957 - val_loss: 0.4975 - val_accuracy: 0.7545
Epoch 15/125
2/2 [==============================] - 0s 51ms/step - loss: 0.5135 - accuracy: 0.6957 - val_loss: 0.4911 - val_accuracy: 0.7545
Epoch 16/125
2/2 [==============================] - 0s 51ms/step - loss: 0.5072 - accuracy: 0.6957 - val_loss: 0.4852 - val_accuracy: 0.7545
Epoch 17/125
2/2 [==============================] - 0s 52ms/step - loss: 0.5022 - accuracy: 0.6957 - val_loss: 0.4798 - val_accuracy: 0.7545
Epoch 18/125
2/2 [==============================] - 0s 43ms/step - loss: 0.4961 - accuracy: 0.6957 - val_loss: 0.4751 - val_accuracy: 0.7545
Epoch 19/125
2/2 [==============================] - 0s 51ms/step - loss: 0.4917 - accuracy: 0.6957 - val_loss: 0.4707 - val_accuracy: 0.7636
Epoch 20/125
2/2 [==============================] - 0s 38ms/step - loss: 0.4866 - accuracy: 0.6957 - val_loss: 0.4667 - val_accuracy: 0.7727
Epoch 21/125
2/2 [==============================] - 0s 56ms/step - loss: 0.4821 - accuracy: 0.6957 - val_loss: 0.4629 - val_accuracy: 0.7727
Epoch 22/125
2/2 [==============================] - 0s 68ms/step - loss: 0.4770 - accuracy: 0.6957 - val_loss: 0.4598 - val_accuracy: 0.7727
Epoch 23/125
2/2 [==============================] - 0s 48ms/step - loss: 0.4719 - accuracy: 0.7174 - val_loss: 0.4570 - val_accuracy: 0.7727
Epoch 24/125
2/2 [==============================] - 0s 62ms/step - loss: 0.4675 - accuracy: 0.7174 - val_loss: 0.4546 - val_accuracy: 0.7727
Epoch 25/125
2/2 [==============================] - 0s 48ms/step - loss: 0.4627 - accuracy: 0.7174 - val_loss: 0.4521 - val_accuracy: 0.7909
Epoch 26/125
2/2 [==============================] - 0s 54ms/step - loss: 0.4585 - accuracy: 0.7174 - val_loss: 0.4498 - val_accuracy: 0.7909
Epoch 27/125
2/2 [==============================] - 0s 88ms/step - loss: 0.4542 - accuracy: 0.7609 - val_loss: 0.4472 - val_accuracy: 0.7909
Epoch 28/125
2/2 [==============================] - 0s 67ms/step - loss: 0.4491 - accuracy: 0.7826 - val_loss: 0.4443 - val_accuracy: 0.7909
Epoch 29/125
```

```
# MODEL 8
# EPOCHS : 670
# learning rate : 0.005
# determine the epoch to evaluate
ep = 144
lr = 0.005

#for ep in epochs:
print(f"Training with epochs: {ep}")

# Initialize the model1
modelE8 = create_base_model(X_train.shape[1])

# Compile the model with the current epoch
optimizer = Adam(learning_rate=0.005)
modelE8.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model1
historyE8 = modelE8.fit(X_train, y_train, epochs=ep, batch_size=32, validation_split=0.2, verbose=1)

# Evaluate the model1
loss, accuracy = modelE8.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with epochs {ep}: {accuracy:.4f}")

# Print all the results
print("\nResults Model 8:")

print(f"epochs: {ep}")
print(f"learning rate: {lr}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Training with epochs: 144
Epoch 1/144
4/4 [==============================] - 2s 192ms/step - loss: 0.5590 - accuracy: 0.7419 - val_loss: 0.4681 - val_accuracy: 0.7812
Epoch 2/144
4/4 [==============================] - 0s 39ms/step - loss: 0.4892 - accuracy: 0.7258 - val_loss: 0.4561 - val_accuracy: 0.7812
Epoch 3/144
4/4 [==============================] - 0s 12ms/step - loss: 0.4386 - accuracy: 0.7500 - val_loss: 0.4277 - val_accuracy: 0.7812
```

```
Epoch 4/144
4/4 [==============================] - 0s 14ms/step - loss: 0.4021 - accuracy: 0.8629 - val_loss: 0.4069 - val_accuracy: 0.7812
Epoch 5/144
4/4 [==============================] - 0s 12ms/step - loss: 0.3831 - accuracy: 0.8306 - val_loss: 0.3981 - val_accuracy: 0.7500
Epoch 6/144
4/4 [==============================] - 0s 12ms/step - loss: 0.3594 - accuracy: 0.8387 - val_loss: 0.4133 - val_accuracy: 0.7812
Epoch 7/144
4/4 [==============================] - 0s 14ms/step - loss: 0.3490 - accuracy: 0.8387 - val_loss: 0.4070 - val_accuracy: 0.7812
Epoch 8/144
4/4 [==============================] - 0s 18ms/step - loss: 0.3276 - accuracy: 0.8629 - val_loss: 0.3868 - val_accuracy: 0.7812
Epoch 9/144
4/4 [==============================] - 0s 12ms/step - loss: 0.3351 - accuracy: 0.8065 - val_loss: 0.3770 - val_accuracy: 0.8125
Epoch 10/144
4/4 [==============================] - 0s 12ms/step - loss: 0.3205 - accuracy: 0.8226 - val_loss: 0.3830 - val_accuracy: 0.7812
Epoch 11/144
4/4 [==============================] - 0s 12ms/step - loss: 0.3150 - accuracy: 0.8629 - val_loss: 0.3964 - val_accuracy: 0.7500
Epoch 12/144
4/4 [==============================] - 0s 13ms/step - loss: 0.3098 - accuracy: 0.8629 - val_loss: 0.3726 - val_accuracy: 0.7812
Epoch 13/144
4/4 [==============================] - 0s 19ms/step - loss: 0.3181 - accuracy: 0.8468 - val_loss: 0.3510 - val_accuracy: 0.8125
Epoch 14/144
4/4 [==============================] - 0s 18ms/step - loss: 0.2997 - accuracy: 0.8548 - val_loss: 0.3595 - val_accuracy: 0.7812
Epoch 15/144
4/4 [==============================] - 0s 20ms/step - loss: 0.2959 - accuracy: 0.8710 - val_loss: 0.3634 - val_accuracy: 0.7812
Epoch 16/144
4/4 [==============================] - 0s 15ms/step - loss: 0.2930 - accuracy: 0.8548 - val_loss: 0.3343 - val_accuracy: 0.8125
Epoch 17/144
4/4 [==============================] - 0s 21ms/step - loss: 0.2851 - accuracy: 0.8548 - val_loss: 0.3333 - val_accuracy: 0.8125
Epoch 18/144
4/4 [==============================] - 0s 21ms/step - loss: 0.2812 - accuracy: 0.8548 - val_loss: 0.3262 - val_accuracy: 0.8125
Epoch 19/144
4/4 [==============================] - 0s 14ms/step - loss: 0.2762 - accuracy: 0.8548 - val_loss: 0.3306 - val_accuracy: 0.7812
Epoch 20/144
4/4 [==============================] - 0s 18ms/step - loss: 0.2743 - accuracy: 0.8629 - val_loss: 0.3274 - val_accuracy: 0.7812
Epoch 21/144
4/4 [==============================] - 0s 18ms/step - loss: 0.2726 - accuracy: 0.8629 - val_loss: 0.3087 - val_accuracy: 0.8125
Epoch 22/144
4/4 [==============================] - 0s 12ms/step - loss: 0.2684 - accuracy: 0.8629 - val_loss: 0.3243 - val_accuracy: 0.7812
Epoch 23/144
4/4 [==============================] - 0s 18ms/step - loss: 0.2633 - accuracy: 0.8548 - val_loss: 0.3171 - val_accuracy: 0.7812
Epoch 24/144
4/4 [==============================] - 0s 12ms/step - loss: 0.2607 - accuracy: 0.8548 - val_loss: 0.3144 - val_accuracy: 0.7812
Epoch 25/144
4/4 [==============================] - 0s 13ms/step - loss: 0.2727 - accuracy: 0.8468 - val_loss: 0.2923 - val_accuracy: 0.8125
Epoch 26/144
4/4 [==============================] - 0s 18ms/step - loss: 0.2497 - accuracy: 0.8629 - val_loss: 0.3274 - val_accuracy: 0.7812
Epoch 27/144
4/4 [==============================] - 0s 12ms/step - loss: 0.2600 - accuracy: 0.8871 - val_loss: 0.3316 - val_accuracy: 0.7812
Epoch 28/144
4/4 [==============================] - 0s 17ms/step - loss: 0.2443 - accuracy: 0.8871 - val_loss: 0.2831 - val_accuracy: 0.7812
```

```
# MODEL 9
# EPOCHS : 101
# learning rate : 0.11
# determine the epoch to evaluate
ep9 = 101
lr = 0.11

#for ep in epochs:
print(f"Training with epochs: {ep}")

# Initialize the model1
modelE9 = create_base_model(X_train.shape[1])

# Compile the model with the current epoch
optimizer = Adam(learning_rate=0.11)
modelE9.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model1
historyE9 = modelE9.fit(X_train, y_train, epochs=ep9, batch_size=32, validation_split=0.2, verbose=1)

# Evaluate the model1
loss, accuracy = modelE9.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with epochs {ep9}: {accuracy:.4f}")

# Print all the results
print("\nResults Model 9:")

print(f"epochs: {ep9}")
print(f"learning rate: {lr}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Training with epochs: 144
Epoch 1/101
4/4 [==============================] - 2s 142ms/step - loss: 0.8385 - accuracy: 0.4839 - val_loss: 0.7231 - val_accuracy: 0.7812
Epoch 2/101
4/4 [==============================] - 0s 18ms/step - loss: 0.6960 - accuracy: 0.7177 - val_loss: 0.4948 - val_accuracy: 0.7812
Epoch 3/101
4/4 [==============================] - 0s 24ms/step - loss: 0.5123 - accuracy: 0.7258 - val_loss: 0.4784 - val_accuracy: 0.7812
Epoch 4/101
4/4 [==============================] - 0s 12ms/step - loss: 0.4599 - accuracy: 0.7581 - val_loss: 0.4014 - val_accuracy: 0.7812
Epoch 5/101
4/4 [==============================] - 0s 12ms/step - loss: 0.3791 - accuracy: 0.8387 - val_loss: 0.4348 - val_accuracy: 0.6875
Epoch 6/101
4/4 [==============================] - 0s 13ms/step - loss: 0.3997 - accuracy: 0.7823 - val_loss: 0.4019 - val_accuracy: 0.7188
Epoch 7/101
4/4 [==============================] - 0s 13ms/step - loss: 0.3422 - accuracy: 0.7984 - val_loss: 0.4742 - val_accuracy: 0.7812
Epoch 8/101
4/4 [==============================] - 0s 12ms/step - loss: 0.3554 - accuracy: 0.8548 - val_loss: 0.3542 - val_accuracy: 0.7812
Epoch 9/101
4/4 [==============================] - 0s 12ms/step - loss: 0.3299 - accuracy: 0.8548 - val_loss: 0.3510 - val_accuracy: 0.7812
Epoch 10/101
4/4 [==============================] - 0s 15ms/step - loss: 0.2972 - accuracy: 0.8548 - val_loss: 0.3747 - val_accuracy: 0.7812
Epoch 11/101
4/4 [==============================] - 0s 15ms/step - loss: 0.2686 - accuracy: 0.8548 - val_loss: 0.3653 - val_accuracy: 0.7812
Epoch 12/101
4/4 [==============================] - 0s 19ms/step - loss: 0.2599 - accuracy: 0.8548 - val_loss: 0.3431 - val_accuracy: 0.8125
Epoch 13/101
4/4 [==============================] - 0s 21ms/step - loss: 0.2607 - accuracy: 0.8629 - val_loss: 0.3993 - val_accuracy: 0.7812
Epoch 14/101
4/4 [==============================] - 0s 15ms/step - loss: 0.2130 - accuracy: 0.8790 - val_loss: 0.3390 - val_accuracy: 0.7812
Epoch 15/101
4/4 [==============================] - 0s 21ms/step - loss: 0.2121 - accuracy: 0.8710 - val_loss: 0.4864 - val_accuracy: 0.8125
Epoch 16/101
4/4 [==============================] - 0s 15ms/step - loss: 0.2225 - accuracy: 0.8952 - val_loss: 0.3097 - val_accuracy: 0.8125
Epoch 17/101
4/4 [==============================] - 0s 12ms/step - loss: 0.2217 - accuracy: 0.9032 - val_loss: 0.3287 - val_accuracy: 0.8125
Epoch 18/101
4/4 [==============================] - 0s 19ms/step - loss: 0.1873 - accuracy: 0.9113 - val_loss: 0.4459 - val_accuracy: 0.8125
Epoch 19/101
4/4 [==============================] - 0s 12ms/step - loss: 0.1859 - accuracy: 0.8952 - val_loss: 0.3593 - val_accuracy: 0.8750
Epoch 20/101
4/4 [==============================] - 0s 19ms/step - loss: 0.1798 - accuracy: 0.9113 - val_loss: 0.3234 - val_accuracy: 0.8125
Epoch 21/101
4/4 [==============================] - 0s 17ms/step - loss: 0.1864 - accuracy: 0.9113 - val_loss: 0.2544 - val_accuracy: 0.8750
Epoch 22/101
4/4 [==============================] - 0s 12ms/step - loss: 0.1523 - accuracy: 0.9194 - val_loss: 0.5238 - val_accuracy: 0.8125
Epoch 23/101
4/4 [==============================] - 0s 13ms/step - loss: 0.1594 - accuracy: 0.9113 - val_loss: 0.2033 - val_accuracy: 0.9062
Epoch 24/101
4/4 [==============================] - 0s 18ms/step - loss: 0.1729 - accuracy: 0.9113 - val_loss: 0.3711 - val_accuracy: 0.8125
Epoch 25/101
```

```
4/4 [==============================] - 0s 19ms/step - loss: 0.1674 - accuracy: 0.9032 - val_loss: 0.2738 - val_accuracy: 0.8438
Epoch 26/101
4/4 [==============================] - 0s 19ms/step - loss: 0.1527 - accuracy: 0.9032 - val_loss: 0.2023 - val_accuracy: 0.8750
Epoch 27/101
4/4 [==============================] - 0s 12ms/step - loss: 0.1426 - accuracy: 0.9113 - val_loss: 0.4926 - val_accuracy: 0.8125
Epoch 28/101
4/4 [==============================] - 0s 13ms/step - loss: 0.1538 - accuracy: 0.9113 - val_loss: 0.2282 - val_accuracy: 0.8438
```

```
# MODEL 10
# EPOCHS : 816
# learning rate : 0.0006
# determine the epoch to evaluate
ep10 = 816
lr = 0.0006

#for ep in epochs:
print(f"Training with epochs: {ep}")

# Initialize the model1
modelE10 = create_base_model(X_train.shape[1])

# Compile the model with the current epoch
optimizer = Adam(learning_rate=0.0006)
modelE10.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model1
historyE10 = modelE10.fit(X_train, y_train, epochs=ep9, batch_size=32, validation_split=0.2, verbose=1)

# Evaluate the model1
loss, accuracy = modelE10.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with epochs {ep9}: {accuracy:.4f}")

# Print all the results
print("\nResults Model 10:")

print(f"epochs: {ep9}")
print(f"learning rate: {lr}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Training with epochs: 144
Epoch 1/101
4/4 [==============================] - 2s 123ms/step - loss: 0.7288 - accuracy: 0.3226 - val_loss: 0.6829 - val_accuracy: 0.5312
Epoch 2/101
4/4 [==============================] - 0s 18ms/step - loss: 0.7022 - accuracy: 0.5161 - val_loss: 0.6525 - val_accuracy: 0.7812
Epoch 3/101
4/4 [==============================] - 0s 27ms/step - loss: 0.6765 - accuracy: 0.6452 - val_loss: 0.6271 - val_accuracy: 0.8125
Epoch 4/101
4/4 [==============================] - 0s 19ms/step - loss: 0.6531 - accuracy: 0.7177 - val_loss: 0.6072 - val_accuracy: 0.7812
Epoch 5/101
4/4 [==============================] - 0s 23ms/step - loss: 0.6342 - accuracy: 0.7258 - val_loss: 0.5920 - val_accuracy: 0.7812
Epoch 6/101
4/4 [==============================] - 0s 22ms/step - loss: 0.6169 - accuracy: 0.7258 - val_loss: 0.5793 - val_accuracy: 0.7812
Epoch 7/101
4/4 [==============================] - 0s 18ms/step - loss: 0.6033 - accuracy: 0.7258 - val_loss: 0.5665 - val_accuracy: 0.7812
Epoch 8/101
4/4 [==============================] - 0s 23ms/step - loss: 0.5898 - accuracy: 0.7258 - val_loss: 0.5538 - val_accuracy: 0.7812
Epoch 9/101
4/4 [==============================] - 0s 24ms/step - loss: 0.5787 - accuracy: 0.7258 - val_loss: 0.5407 - val_accuracy: 0.7812
Epoch 10/101
4/4 [==============================] - 0s 27ms/step - loss: 0.5645 - accuracy: 0.7258 - val_loss: 0.5303 - val_accuracy: 0.7812
Epoch 11/101
4/4 [==============================] - 0s 24ms/step - loss: 0.5538 - accuracy: 0.7258 - val_loss: 0.5202 - val_accuracy: 0.7812
Epoch 12/101
4/4 [==============================] - 0s 23ms/step - loss: 0.5422 - accuracy: 0.7258 - val_loss: 0.5104 - val_accuracy: 0.7812
Epoch 13/101
4/4 [==============================] - 0s 21ms/step - loss: 0.5321 - accuracy: 0.7258 - val_loss: 0.5013 - val_accuracy: 0.7812
Epoch 14/101
4/4 [==============================] - 0s 23ms/step - loss: 0.5232 - accuracy: 0.7258 - val_loss: 0.4936 - val_accuracy: 0.7812
Epoch 15/101
4/4 [==============================] - 0s 24ms/step - loss: 0.5161 - accuracy: 0.7258 - val_loss: 0.4871 - val_accuracy: 0.7812
Epoch 16/101
4/4 [==============================] - 0s 32ms/step - loss: 0.5076 - accuracy: 0.7258 - val_loss: 0.4820 - val_accuracy: 0.7812
Epoch 17/101
4/4 [==============================] - 0s 30ms/step - loss: 0.5008 - accuracy: 0.7258 - val_loss: 0.4775 - val_accuracy: 0.7812
Epoch 18/101
4/4 [==============================] - 0s 23ms/step - loss: 0.4935 - accuracy: 0.7258 - val_loss: 0.4732 - val_accuracy: 0.7812
Epoch 19/101
4/4 [==============================] - 0s 15ms/step - loss: 0.4866 - accuracy: 0.7339 - val_loss: 0.4694 - val_accuracy: 0.7812
Epoch 20/101
4/4 [==============================] - 0s 12ms/step - loss: 0.4795 - accuracy: 0.7661 - val_loss: 0.4656 - val_accuracy: 0.7812
Epoch 21/101
```

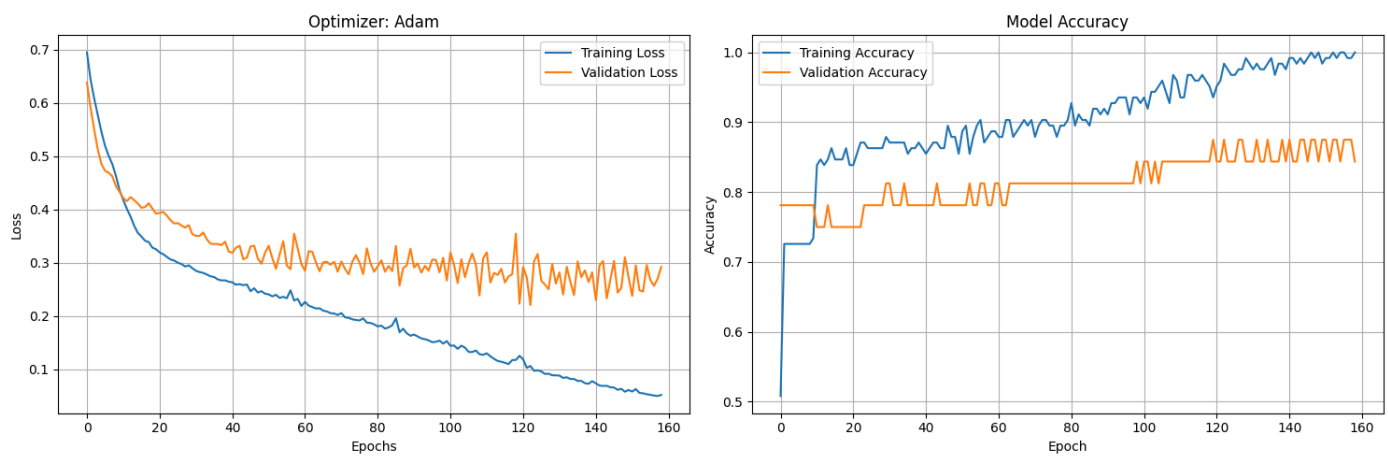
```
4/4 [==============================] - 0s 20ms/step - loss: 0.4727 - accuracy: 0.7823 - val_loss: 0.4621 - val_accuracy: 0.7812
Epoch 22/101
4/4 [==============================] - 0s 12ms/step - loss: 0.4653 - accuracy: 0.7984 - val_loss: 0.4583 - val_accuracy: 0.7812
Epoch 23/101
4/4 [==============================] - 0s 14ms/step - loss: 0.4584 - accuracy: 0.7984 - val_loss: 0.4546 - val_accuracy: 0.7812
Epoch 24/101
4/4 [==============================] - 0s 12ms/step - loss: 0.4526 - accuracy: 0.8226 - val_loss: 0.4512 - val_accuracy: 0.7812
Epoch 25/101
4/4 [==============================] - 0s 12ms/step - loss: 0.4464 - accuracy: 0.8306 - val_loss: 0.4479 - val_accuracy: 0.7812
Epoch 26/101
4/4 [==============================] - 0s 12ms/step - loss: 0.4417 - accuracy: 0.8306 - val_loss: 0.4446 - val_accuracy: 0.7812
Epoch 27/101
4/4 [==============================] - 0s 12ms/step - loss: 0.4356 - accuracy: 0.8387 - val_loss: 0.4417 - val_accuracy: 0.7812
Epoch 28/101
4/4 [==============================] - 0s 13ms/step - loss: 0.4301 - accuracy: 0.8548 - val_loss: 0.4389 - val_accuracy: 0.7812
Epoch 29/101
```

OPTIMIZER MODEL

```
# MODEL 1
# Optimizer :  Adam

# determine the optimizer to evaluate
optimizers = tf.keras.optimizers.Adam
op = optimizers

#for ep in optimizer:
print(f"Training with optimizer: {op.__name__}")

# Initialize the model1
modelO1 = create_base_model(X_train.shape[1])

# Compile the model with the current optimizer
optimizer = op(learning_rate=0.004)
modelO1.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model1
historyO1 = modelO1.fit(X_train, y_train, epochs=159, batch_size=32, validation_split=0.2, verbose=1)

# Evaluate the model1
loss, accuracy = modelO1.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with epochs {op.__name__}: {accuracy:.4f}")

# Print all the results
print("\nResults:")

print(f"Optimizer: {op.__name__}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
4/4 [==============================] - 0s 19ms/step - loss: 0.0636 - accuracy: 0.9919 - val_loss: 0.2528 - val_accuracy: 0.8750
Epoch 149/159
4/4 [==============================] - 0s 13ms/step - loss: 0.0582 - accuracy: 1.0000 - val_loss: 0.3102 - val_accuracy: 0.8438
Epoch 150/159
4/4 [==============================] - 0s 14ms/step - loss: 0.0614 - accuracy: 0.9839 - val_loss: 0.2730 - val_accuracy: 0.8750
Epoch 151/159
4/4 [==============================] - 0s 13ms/step - loss: 0.0584 - accuracy: 0.9919 - val_loss: 0.2379 - val_accuracy: 0.8750
Epoch 152/159
4/4 [==============================] - 0s 19ms/step - loss: 0.0631 - accuracy: 0.9919 - val_loss: 0.2950 - val_accuracy: 0.8438
Epoch 153/159
4/4 [==============================] - 0s 22ms/step - loss: 0.0559 - accuracy: 1.0000 - val_loss: 0.2486 - val_accuracy: 0.8750
Epoch 154/159
4/4 [==============================] - 0s 19ms/step - loss: 0.0552 - accuracy: 0.9919 - val_loss: 0.2460 - val_accuracy: 0.8750
Epoch 155/159
4/4 [==============================] - 0s 16ms/step - loss: 0.0534 - accuracy: 1.0000 - val_loss: 0.2959 - val_accuracy: 0.8438
Epoch 156/159
4/4 [==============================] - 0s 16ms/step - loss: 0.0524 - accuracy: 1.0000 - val_loss: 0.2679 - val_accuracy: 0.8750
Epoch 157/159
4/4 [==============================] - 0s 16ms/step - loss: 0.0510 - accuracy: 0.9919 - val_loss: 0.2568 - val_accuracy: 0.8750
Epoch 158/159
4/4 [==============================] - 0s 22ms/step - loss: 0.0501 - accuracy: 0.9919 - val_loss: 0.2699 - val_accuracy: 0.8750
Epoch 159/159
4/4 [==============================] - 0s 15ms/step - loss: 0.0521 - accuracy: 1.0000 - val_loss: 0.2924 - val_accuracy: 0.8438
Test accuracy with epochs Adam: 0.8974

Results:
Optimizer: Adam
Test accuracy: 0.8974
```

```
# Plot the results
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

axes[0].set_title(f'Optimizer: Adam')
axes[0].set_xlabel('Epochs')
axes[0].set_ylabel('Loss')
axes[0].plot(historyO1.history['loss'], label='Training Loss')
axes[0].plot(historyO1.history['val_loss'], label='Validation Loss')
axes[0].legend()
axes[0].grid(True)

axes[1].plot(historyO1.history['accuracy'], label='Training Accuracy')
axes[1].plot(historyO1.history['val_accuracy'], label='Validation Accuracy')
axes[1].set_title('Model Accuracy')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.show()
```



```
# Model 2
# Optimizer : Nadam

# determine the optimizer to evaluate
optimizers = tf.keras.optimizers.Nadam
op = optimizers

#for ep in optimizer:
print(f"Training with optimizer: {op.__name__}")

# Initialize the model1
model02 = create_base_model(X_train.shape[1])

# Compile the model with the current optimizer
optimizer = op(learning_rate=0.004)
model02.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model1
history02 = model02.fit(X_train, y_train, epochs=159, batch_size=32, validation_split=0.2, verbose=1)

# Evaluate the model1
loss, accuracy = model02.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with epochs {op.__name__}: {accuracy:.4f}")

# Print all the results
print("\nResults:")

print(f"Optimizer: {op.__name__}")
print(f"Test accuracy: {accuracy:.4f}")
```

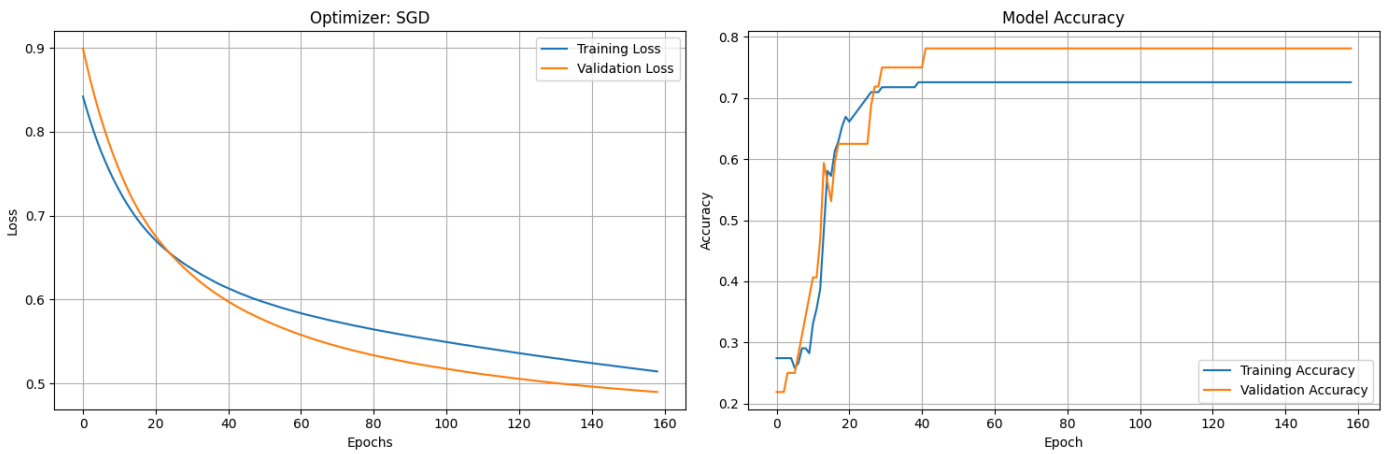
```
4/4 [==============================] - 0s 13ms/step - loss: 0.0757 - accuracy: 0.9758 - val_loss: 0.2333 - val_accuracy: 0.9375
Epoch 148/159
4/4 [==============================] - 0s 12ms/step - loss: 0.0717 - accuracy: 0.9839 - val_loss: 0.2589 - val_accuracy: 0.9062
Epoch 149/159
4/4 [==============================] - 0s 13ms/step - loss: 0.0720 - accuracy: 0.9839 - val_loss: 0.2466 - val_accuracy: 0.9375
Epoch 150/159
4/4 [==============================] - 0s 13ms/step - loss: 0.0716 - accuracy: 0.9758 - val_loss: 0.2597 - val_accuracy: 0.9062
```

```
# Plot the results
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

axes[0].set_title(f'Optimizer: Nadam')
axes[0].set_xlabel('Epochs')
axes[0].set_ylabel('Loss')
axes[0].plot(historyO2.history['loss'], label='Training Loss')
axes[0].plot(historyO2.history['val_loss'], label='Validation Loss')
axes[0].legend()
axes[0].grid(True)

axes[1].plot(historyO2.history['accuracy'], label='Training Accuracy')
axes[1].plot(historyO2.history['val_accuracy'], label='Validation Accuracy')
axes[1].set_title('Model Accuracy')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.show()
```

```
# Model 3
# Optimizer : SGD

# determine the optimizer to evaluate
optimizers = tf.keras.optimizers.SGD
op = optimizers

#for ep in optimizer:
print(f"Training with optimizer: {op.__name__}")

# Initialize the model1
model03 = create_base_model(X_train.shape[1])

# Compile the model with the current optimizer
optimizer = op(learning_rate=0.004)
model03.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model1
history03 = model03.fit(X_train, y_train, epochs=159, batch_size=32, validation_split=0.2, verbose=1)

# Evaluate the model1
loss, accuracy = model03.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with epochs {op.__name__}: {accuracy:.4f}")

# Print all the results
print("\nResults:")

print(f"Optimizer: {op.__name__}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Epoch 159/159
4/4 [==============================] - 0s 14ms/step - loss: 0.5145 - accuracy: 0.7258 - val_loss: 0.4899 - val_accuracy: 0.7812
Test accuracy with epochs SGD: 0.8205

Results:
Optimizer: SGD
Test accuracy: 0.8205
```

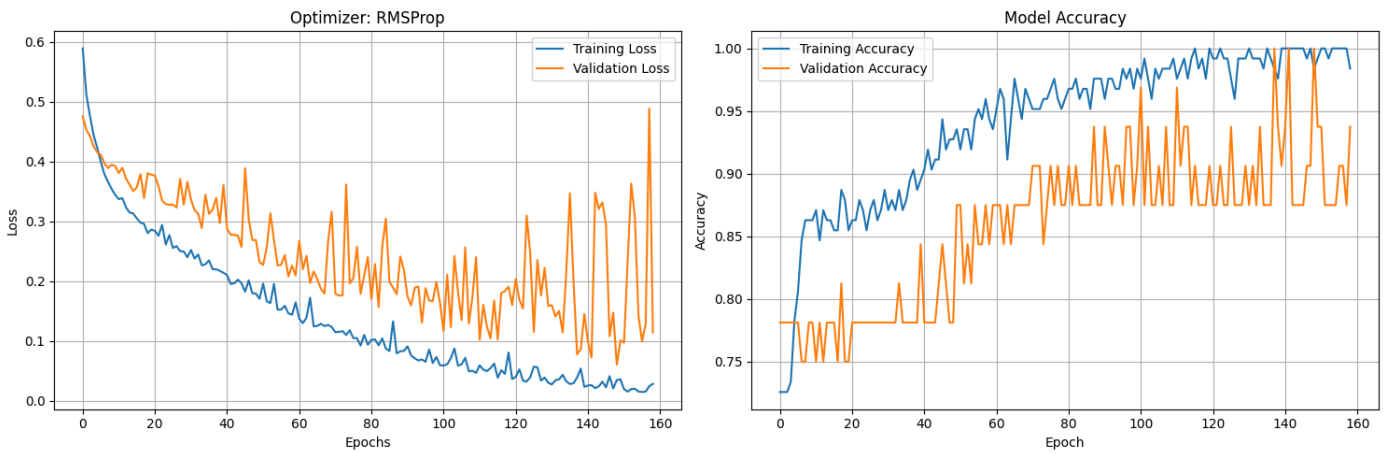
```
# Plot the results
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

axes[0].set_title(f'Optimizer: SGD')
axes[0].set_xlabel('Epochs')
axes[0].set_ylabel('Loss')
axes[0].plot(historyO3.history['loss'], label='Training Loss')
axes[0].plot(historyO3.history['val_loss'], label='Validation Loss')
axes[0].legend()
axes[0].grid(True)

axes[1].plot(historyO3.history['accuracy'], label='Training Accuracy')
axes[1].plot(historyO3.history['val_accuracy'], label='Validation Accuracy')
axes[1].set_title('Model Accuracy')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.show()
```



```
# Model 4
# Optimizer : Rmsprop

# determine the optimizer to evaluate
optimizers = tf.keras.optimizers.RMSprop
op = optimizers

#for ep in optimizer:
print(f"Training with optimizer: {op.__name__}")

# Initialize the model1
model04 = create_base_model(X_train.shape[1])

# Compile the model with the current optimizer
optimizer = op(learning_rate=0.004)
model04.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model1
history04 = model04.fit(X_train, y_train, epochs=159, batch_size=32, validation_split=0.2, verbose=1)

# Evaluate the model1
loss, accuracy = model04.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with epochs {op.__name__}: {accuracy:.4f}")

# Print all the results
print("\nResults:")

print(f"Optimizer: {op.__name__}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Epoch 159/159
4/4 [==============================] - 0s 20ms/step - loss: 0.0280 - accuracy: 0.9839 - val_loss: 0.1141 - val_accuracy: 0.9375
Test accuracy with epochs RMSprop: 0.8974

Results:
Optimizer: RMSprop
Test accuracy: 0.8974
```

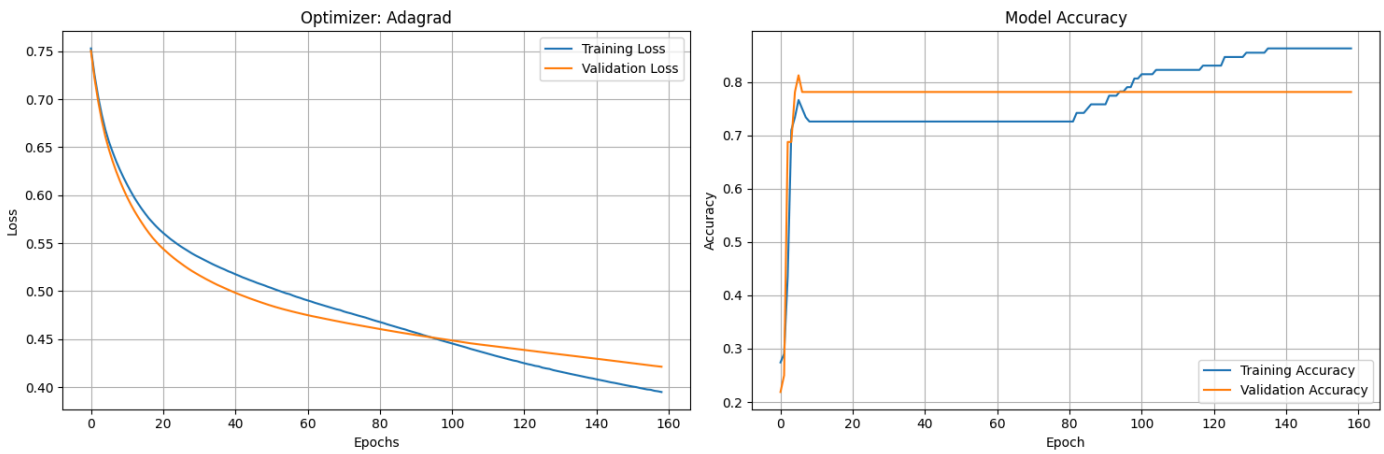
```
# Plot the results
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

axes[0].set_title(f'Optimizer: RMSProp')
axes[0].set_xlabel('Epochs')
axes[0].set_ylabel('Loss')
axes[0].plot(historyO4.history['loss'], label='Training Loss')
axes[0].plot(historyO4.history['val_loss'], label='Validation Loss')
axes[0].legend()
axes[0].grid(True)

axes[1].plot(historyO4.history['accuracy'], label='Training Accuracy')
axes[1].plot(historyO4.history['val_accuracy'], label='Validation Accuracy')
axes[1].set_title('Model Accuracy')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.show()
```



```
# Model 5
# Optimizer : Adagrad

# determine the optimizer to evaluate
optimizers = tf.keras.optimizers.Adagrad
op = optimizers

#for ep in optimizer:
print(f"Training with optimizer: {op.__name__}")

# Initialize the model1
model05 = create_base_model(X_train.shape[1])

# Compile the model with the current optimizer
optimizer = op(learning_rate=0.004)
model05.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model1
history05 = model05.fit(X_train, y_train, epochs=159, batch_size=32, validation_split=0.2, verbose=1)

# Evaluate the model1
loss, accuracy = model05.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with epochs {op.__name__}: {accuracy:.4f}")

# Print all the results
print("\nResults:")

print(f"Optimizer: {op.__name__}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Epoch 159/159
4/4 [==============================] - 0s 24ms/step - loss: 0.3949 - accuracy: 0.8629 - val_loss: 0.4213 - val_accuracy: 0.7812
Test accuracy with epochs Adagrad: 0.8974

Results:
Optimizer: Adagrad
Test accuracy: 0.8974
```

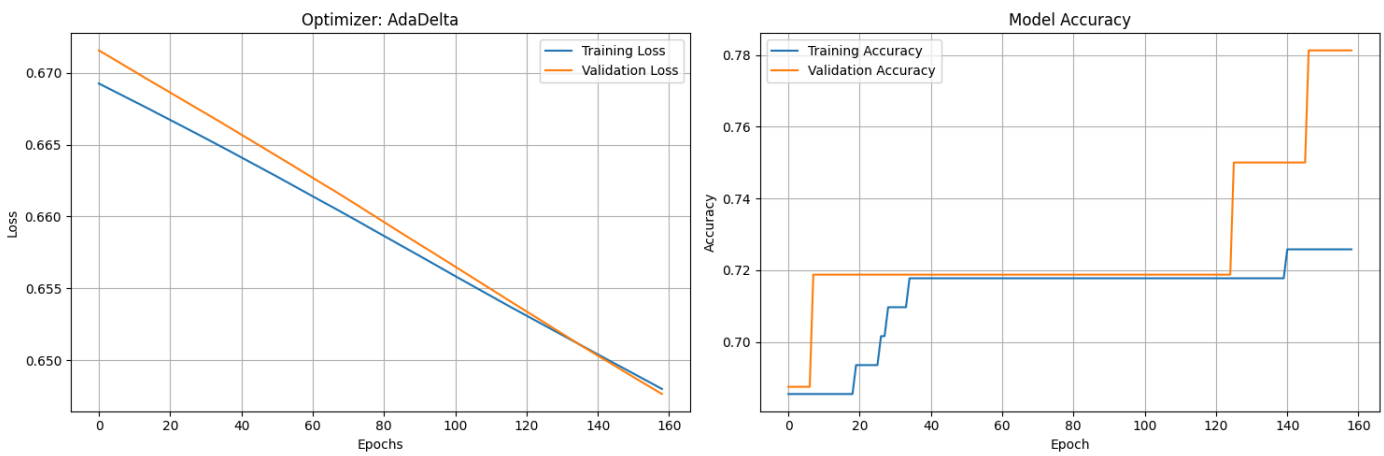
```
# Plot the results
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

axes[0].set_title(f'Optimizer: Adagrad')
axes[0].set_xlabel('Epochs')
axes[0].set_ylabel('Loss')
axes[0].plot(historyO5.history['loss'], label='Training Loss')
axes[0].plot(historyO5.history['val_loss'], label='Validation Loss')
axes[0].legend()
axes[0].grid(True)

axes[1].plot(historyO5.history['accuracy'], label='Training Accuracy')
axes[1].plot(historyO5.history['val_accuracy'], label='Validation Accuracy')
axes[1].set_title('Model Accuracy')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.show()
```



```
# Model 6
# Optimzer : Adadelta

# determine the optimizer to evaluate
optimizers = tf.keras.optimizers.Adadelta
op = optimizers

#for ep in optimizer:
print(f"Training with optimizer: {op.__name__}")

# Initialize the model1
model06 = create_base_model(X_train.shape[1])

# Compile the model with the current optimizer
optimizer = op(learning_rate=0.004)
model06.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model1
history06 = model06.fit(X_train, y_train, epochs=159, batch_size=32, validation_split=0.2, verbose=1)

# Evaluate the model1
loss, accuracy = model06.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with epochs {op.__name__}: {accuracy:.4f}")

# Print all the results
print("\nResults:")

print(f"Optimizer: {op.__name__}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
4/4 [==============================] - 0s 13ms/step - loss: 0.6482 - accuracy: 0.7258 - val_loss: 0.6478 - val_accuracy: 0.7812
Epoch 159/159
4/4 [==============================] - 0s 12ms/step - loss: 0.6480 - accuracy: 0.7258 - val_loss: 0.6477 - val_accuracy: 0.7812
Test accuracy with epochs Adadelta: 0.8205

Results:
Optimizer: Adadelta
Test accuracy: 0.8205
```

```
# Plot the results
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

axes[0].set_title(f'Optimizer: AdaDelta')
axes[0].set_xlabel('Epochs')
axes[0].set_ylabel('Loss')
axes[0].plot(historyO6.history['loss'], label='Training Loss')
axes[0].plot(historyO6.history['val_loss'], label='Validation Loss')
axes[0].legend()
axes[0].grid(True)

axes[1].plot(historyO6.history['accuracy'], label='Training Accuracy')
axes[1].plot(historyO6.history['val_accuracy'], label='Validation Accuracy')
axes[1].set_title('Model Accuracy')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.show()
```



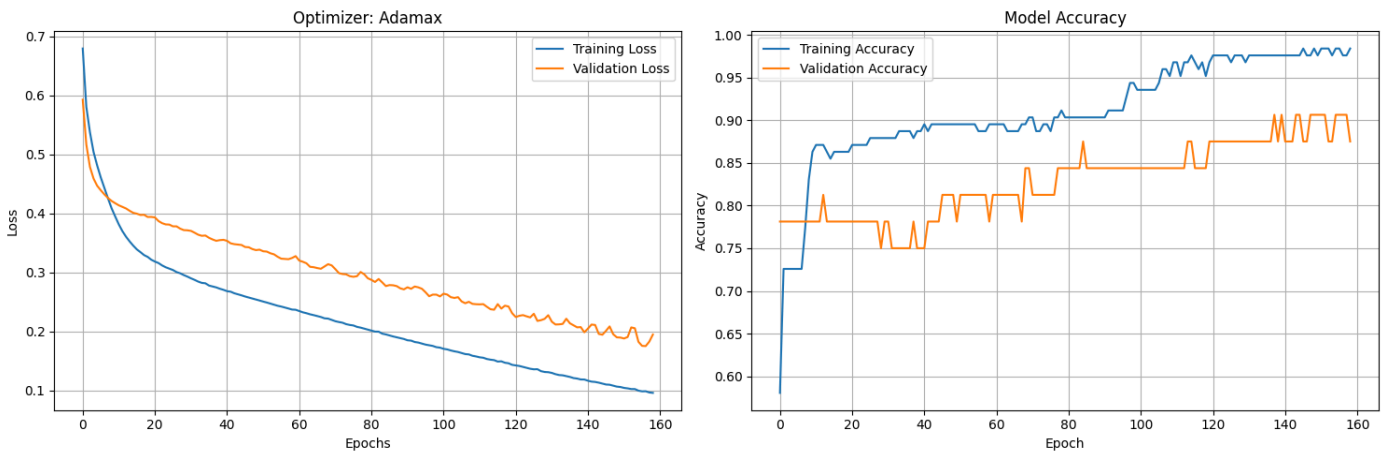

```
# Model 7
# Optimizer : Adamax

# determine the optimizer to evaluate
optimizers = tf.keras.optimizers.Adamax
op = optimizers

#for ep in optimizer:
print(f"Training with optimizer: {op.__name__}")

# Initialize the model1
modelO7 = create_base_model(X_train.shape[1])

# Compile the model with the current optimizer
optimizer = op(learning_rate=0.004)
modelO7.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model1
historyO7 = modelO7.fit(X_train, y_train, epochs=159, batch_size=32, validation_split=0.2, verbose=1)

# Evaluate the model1
loss, accuracy = modelO7.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with epochs {op.__name__}: {accuracy:.4f}")

# Print all the results
```

```
# Print all the results
print("\nResults:")

print(f"Optimizer: {op.__name__}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
4/4 [==============================] - 0s 20ms/step - loss: 0.1264 - accuracy: 0.9758 - val_loss: 0.2123 - val_accuracy: 0.8750
Epoch 134/159
4/4 [==============================] - 0s 12ms/step - loss: 0.1259 - accuracy: 0.9758 - val_loss: 0.2130 - val_accuracy: 0.8750
Epoch 135/159
4/4 [==============================] - 0s 12ms/step - loss: 0.1246 - accuracy: 0.9758 - val_loss: 0.2218 - val_accuracy: 0.8750
Epoch 136/159
4/4 [==============================] - 0s 12ms/step - loss: 0.1232 - accuracy: 0.9758 - val_loss: 0.2141 - val_accuracy: 0.8750
Epoch 137/159
4/4 [==============================] - 0s 11ms/step - loss: 0.1213 - accuracy: 0.9758 - val_loss: 0.2106 - val_accuracy: 0.8750
Epoch 138/159
4/4 [==============================] - 0s 18ms/step - loss: 0.1204 - accuracy: 0.9758 - val_loss: 0.2072 - val_accuracy: 0.9062
Epoch 139/159
4/4 [==============================] - 0s 19ms/step - loss: 0.1189 - accuracy: 0.9758 - val_loss: 0.2077 - val_accuracy: 0.8750
Epoch 140/159
4/4 [==============================] - 0s 12ms/step - loss: 0.1188 - accuracy: 0.9758 - val_loss: 0.1988 - val_accuracy: 0.9062
Epoch 141/159
4/4 [==============================] - 0s 12ms/step - loss: 0.1168 - accuracy: 0.9758 - val_loss: 0.2051 - val_accuracy: 0.8750
Epoch 142/159
4/4 [==============================] - 0s 13ms/step - loss: 0.1152 - accuracy: 0.9758 - val_loss: 0.2119 - val_accuracy: 0.8750
Epoch 143/159
4/4 [==============================] - 0s 16ms/step - loss: 0.1147 - accuracy: 0.9758 - val_loss: 0.2112 - val_accuracy: 0.8750
Epoch 144/159
4/4 [==============================] - 0s 20ms/step - loss: 0.1135 - accuracy: 0.9758 - val_loss: 0.1961 - val_accuracy: 0.9062
Epoch 145/159
4/4 [==============================] - 0s 21ms/step - loss: 0.1120 - accuracy: 0.9758 - val_loss: 0.1947 - val_accuracy: 0.9062
Epoch 146/159
4/4 [==============================] - 0s 21ms/step - loss: 0.1104 - accuracy: 0.9839 - val_loss: 0.2013 - val_accuracy: 0.8750
Epoch 147/159
4/4 [==============================] - 0s 16ms/step - loss: 0.1100 - accuracy: 0.9758 - val_loss: 0.2088 - val_accuracy: 0.8750
Epoch 148/159
4/4 [==============================] - 0s 13ms/step - loss: 0.1084 - accuracy: 0.9758 - val_loss: 0.1956 - val_accuracy: 0.9062
Epoch 149/159
4/4 [==============================] - 0s 12ms/step - loss: 0.1068 - accuracy: 0.9839 - val_loss: 0.1903 - val_accuracy: 0.9062
Epoch 150/159
4/4 [==============================] - 0s 12ms/step - loss: 0.1061 - accuracy: 0.9758 - val_loss: 0.1899 - val_accuracy: 0.9062
Epoch 151/159
4/4 [==============================] - 0s 13ms/step - loss: 0.1046 - accuracy: 0.9839 - val_loss: 0.1883 - val_accuracy: 0.9062
Epoch 152/159
4/4 [==============================] - 0s 19ms/step - loss: 0.1038 - accuracy: 0.9839 - val_loss: 0.1908 - val_accuracy: 0.9062
Epoch 153/159
4/4 [==============================] - 0s 13ms/step - loss: 0.1026 - accuracy: 0.9839 - val_loss: 0.2071 - val_accuracy: 0.8750
Epoch 154/159
4/4 [==============================] - 0s 12ms/step - loss: 0.1026 - accuracy: 0.9758 - val_loss: 0.2056 - val_accuracy: 0.8750
Epoch 155/159
4/4 [==============================] - 0s 13ms/step - loss: 0.0999 - accuracy: 0.9839 - val_loss: 0.1827 - val_accuracy: 0.9062
Epoch 156/159
4/4 [==============================] - 0s 20ms/step - loss: 0.0987 - accuracy: 0.9839 - val_loss: 0.1758 - val_accuracy: 0.9062
Epoch 157/159
4/4 [==============================] - 0s 13ms/step - loss: 0.0989 - accuracy: 0.9758 - val_loss: 0.1755 - val_accuracy: 0.9062
Epoch 158/159
4/4 [==============================] - 0s 12ms/step - loss: 0.0969 - accuracy: 0.9758 - val_loss: 0.1830 - val_accuracy: 0.9062
Epoch 159/159
4/4 [==============================] - 0s 12ms/step - loss: 0.0962 - accuracy: 0.9839 - val_loss: 0.1950 - val_accuracy: 0.8750
Test accuracy with epochs Adamax: 0.9231

Results:
Optimizer: Adamax
Test accuracy: 0.9231
```

```
# Plot the results
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

axes[0].set_title(f'Optimizer: Adamax')
axes[0].set_xlabel('Epochs')
axes[0].set_ylabel('Loss')
axes[0].plot(historyO7.history['loss'], label='Training Loss')
axes[0].plot(historyO7.history['val_loss'], label='Validation Loss')
axes[0].legend()
axes[0].grid(True)

axes[1].plot(historyO7.history['accuracy'], label='Training Accuracy')
axes[1].plot(historyO7.history['val_accuracy'], label='Validation Accuracy')
axes[1].set_title('Model Accuracy')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy')
axes[1].legend()
axes[1].grid(True)
```

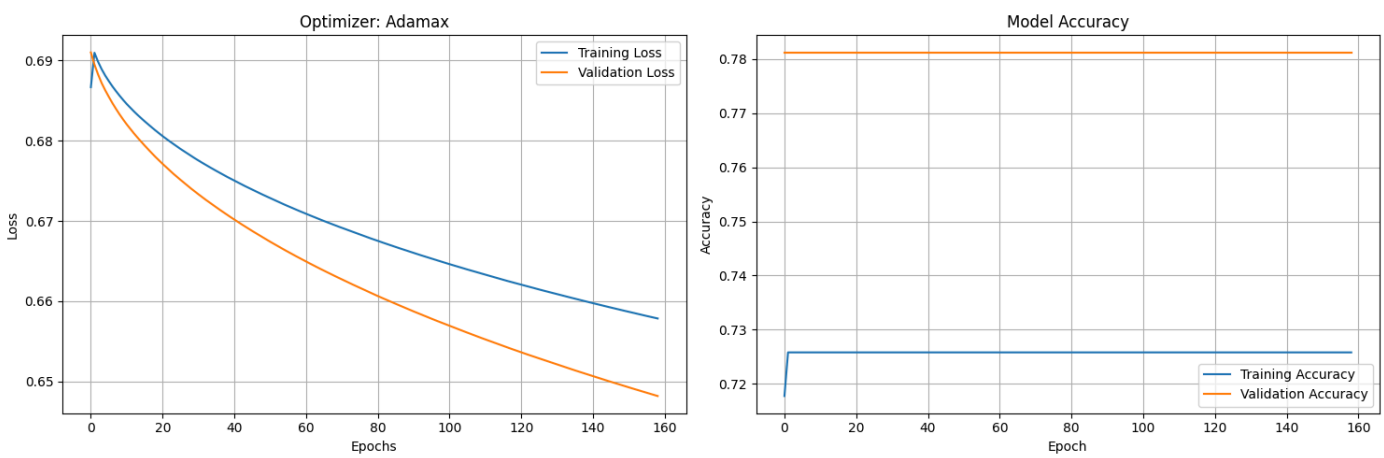
```
axes[1].grid(True)

plt.tight_layout()
plt.show()
```




```
# MODEL 8
# Optimizer = Ftrl
# determine the optimizer to evaluate
optimizers = tf.keras.optimizers.Ftrl
op = optimizers

#for ep in optimizer:
print(f"Training with optimizer: {op.__name__}")

# Initialize the model1
model08 = create_base_model(X_train.shape[1])

# Compile the model with the current optimizer
optimizer = op(learning_rate=0.004)
model08.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model1
history08 = model08.fit(X_train, y_train, epochs=159, batch_size=32, validation_split=0.2, verbose=1)

# Evaluate the model1
loss, accuracy = model08.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with epochs {op.__name__}: {accuracy:.4f}")

# Print all the results
print("\nResults:")

print(f"Optimizer: {op.__name__}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
4/4 [==============================] - 0s 12ms/step - loss: 0.6594 - accuracy: 0.7258 - val_loss: 0.6502 - val_accuracy: 0.7812
Epoch 145/159
4/4 [==============================] - 0s 12ms/step - loss: 0.6593 - accuracy: 0.7258 - val_loss: 0.6501 - val_accuracy: 0.7812
Epoch 146/159
4/4 [==============================] - 0s 18ms/step - loss: 0.6592 - accuracy: 0.7258 - val_loss: 0.6499 - val_accuracy: 0.7812
Epoch 147/159
4/4 [==============================] - 0s 21ms/step - loss: 0.6591 - accuracy: 0.7258 - val_loss: 0.6498 - val_accuracy: 0.7812
Epoch 148/159
4/4 [==============================] - 0s 21ms/step - loss: 0.6590 - accuracy: 0.7258 - val_loss: 0.6497 - val_accuracy: 0.7812
Epoch 149/159
4/4 [==============================] - 0s 21ms/step - loss: 0.6589 - accuracy: 0.7258 - val_loss: 0.6495 - val_accuracy: 0.7812
Epoch 150/159
4/4 [==============================] - 0s 16ms/step - loss: 0.6588 - accuracy: 0.7258 - val_loss: 0.6494 - val_accuracy: 0.7812
Epoch 151/159
4/4 [==============================] - 0s 13ms/step - loss: 0.6587 - accuracy: 0.7258 - val_loss: 0.6493 - val_accuracy: 0.7812
Epoch 152/159
4/4 [==============================] - 0s 12ms/step - loss: 0.6586 - accuracy: 0.7258 - val_loss: 0.6491 - val_accuracy: 0.7812
Epoch 153/159
4/4 [==============================] - 0s 19ms/step - loss: 0.6585 - accuracy: 0.7258 - val_loss: 0.6490 - val_accuracy: 0.7812
Epoch 154/159
4/4 [==============================] - 0s 19ms/step - loss: 0.6584 - accuracy: 0.7258 - val_loss: 0.6488 - val_accuracy: 0.7812
Epoch 155/159
4/4 [==============================] - 0s 12ms/step - loss: 0.6583 - accuracy: 0.7258 - val_loss: 0.6487 - val_accuracy: 0.7812
Epoch 156/159
4/4 [==============================] - 0s 12ms/step - loss: 0.6582 - accuracy: 0.7258 - val_loss: 0.6486 - val_accuracy: 0.7812
Epoch 157/159
4/4 [==============================] - 0s 18ms/step - loss: 0.6580 - accuracy: 0.7258 - val_loss: 0.6484 - val_accuracy: 0.7812
Epoch 158/159
4/4 [==============================] - 0s 18ms/step - loss: 0.6579 - accuracy: 0.7258 - val_loss: 0.6483 - val_accuracy: 0.7812
Epoch 159/159
4/4 [==============================] - 0s 18ms/step - loss: 0.6578 - accuracy: 0.7258 - val_loss: 0.6482 - val_accuracy: 0.7812
Test accuracy with epochs Ftrl: 0.8205

Results:
Optimizer: Ftrl
Test accuracy: 0.8205
```

```
# Plot the results
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

axes[0].set_title(f'Optimizer: Adamax')
axes[0].set_xlabel('Epochs')
axes[0].set_ylabel('Loss')
axes[0].plot(historyO8.history['loss'], label='Training Loss')
axes[0].plot(historyO8.history['val_loss'], label='Validation Loss')
axes[0].legend()
axes[0].grid(True)

axes[1].plot(historyO8.history['accuracy'], label='Training Accuracy')
axes[1].plot(historyO8.history['val_accuracy'], label='Validation Accuracy')
axes[1].set_title('Model Accuracy')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.show()
```



```
# MODEL 9
# Optimizer = Nadam
# Learning rate = 0.02
# determine the optimizer to evaluate
optimizers = tf.keras.optimizers. Nadam
op = optimizers
learning_rates = 0.02
lr = learning_rates

#for ep in optimizer:
print(f"Training with optimizer: {op.__name__}")

# Initialize the model1
model09 = create_base_model(X_train.shape[1])

# Compile the model with the current optimizer
optimizer = op(learning_rate=lr)
model09.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model1
history09 = model09.fit(X_train, y_train, epochs=159, batch_size=32, validation_split=0.2, verbose=1)

# Evaluate the model1
loss, accuracy = model09.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with epochs {op.__name__}: {accuracy:.4f}")

# Print all the results
print("\nResults:")

print(f"Optimizer: {op.__name__}")
print(f"Learning rate: {lr}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Epoch 121/159
4/4 [==============================] - 0s 16ms/step - loss: 0.0011 - accuracy: 1.0000 - val_loss: 0.4582 - val_accuracy: 0.8750
Epoch 122/159
4/4 [==============================] - 0s 16ms/step - loss: 0.0010 - accuracy: 1.0000 - val_loss: 0.4649 - val_accuracy: 0.8750
Epoch 123/159
4/4 [==============================] - 0s 14ms/step - loss: 0.0010 - accuracy: 1.0000 - val_loss: 0.4692 - val_accuracy: 0.8750
Epoch 124/159
4/4 [==============================] - 0s 13ms/step - loss: 0.0011 - accuracy: 1.0000 - val_loss: 0.5018 - val_accuracy: 0.8750
Epoch 125/159
4/4 [==============================] - 0s 20ms/step - loss: 0.0010 - accuracy: 1.0000 - val_loss: 0.4954 - val_accuracy: 0.8750
Epoch 126/159
```

```
# MODEL 10
# Optimzer = Adadelta
# Learning rate = 0.000001
# determine the optimizer to evaluate

optimizers = tf.keras.optimizers.Adadelta
op = optimizers
learning_rates = 0.000001
lr = learning_rates

#for ep in optimizer:
print(f"Training with optimizer: {op.__name__}")

# Initialize the model1
modelO10 = create_base_model(X_train.shape[1])

# Compile the model with the current optimizer
optimizer = op(learning_rate=lr)
modelO10.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model1
historyO10 = modelO10.fit(X_train, y_train, epochs=159, batch_size=32, validation_split=0.2, verbose=1)

# Evaluate the model1
loss, accuracy = modelO10.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy with epochs {op.__name__}: {accuracy:.4f}")

# Print all the results
print("\nResults:")

print(f"Optimizer: {op.__name__}")
print(f"Learning rate: {lr}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Training with optimizer: Adadelta
Epoch 1/159
4/4 [==============================] - 2s 113ms/step - loss: 0.6719 - accuracy: 0.7016 - val_loss: 0.6620 - val_accuracy: 0.7812
Epoch 2/159
4/4 [==============================] - 0s 13ms/step - loss: 0.6719 - accuracy: 0.7016 - val_loss: 0.6620 - val_accuracy: 0.7812
Epoch 3/159
4/4 [==============================] - 0s 12ms/step - loss: 0.6719 - accuracy: 0.7016 - val_loss: 0.6620 - val_accuracy: 0.7812
Epoch 4/159
4/4 [==============================] - 0s 12ms/step - loss: 0.6719 - accuracy: 0.7016 - val_loss: 0.6620 - val_accuracy: 0.7812
Epoch 5/159
4/4 [==============================] - 0s 17ms/step - loss: 0.6719 - accuracy: 0.7016 - val_loss: 0.6620 - val_accuracy: 0.7812
Epoch 6/159
4/4 [==============================] - 0s 20ms/step - loss: 0.6719 - accuracy: 0.7016 - val_loss: 0.6620 - val_accuracy: 0.7812
Epoch 7/159
4/4 [==============================] - 0s 12ms/step - loss: 0.6719 - accuracy: 0.7016 - val_loss: 0.6620 - val_accuracy: 0.7812
Epoch 8/159
4/4 [==============================] - 0s 12ms/step - loss: 0.6719 - accuracy: 0.7016 - val_loss: 0.6620 - val_accuracy: 0.7812
Epoch 9/159
4/4 [==============================] - 0s 21ms/step - loss: 0.6719 - accuracy: 0.7016 - val_loss: 0.6620 - val_accuracy: 0.7812
Epoch 10/159
4/4 [==============================] - 0s 21ms/step - loss: 0.6719 - accuracy: 0.7016 - val_loss: 0.6620 - val_accuracy: 0.7812
Epoch 11/159
4/4 [==============================] - 0s 16ms/step - loss: 0.6719 - accuracy: 0.7016 - val_loss: 0.6620 - val_accuracy: 0.7812
Epoch 12/159
4/4 [==============================] - 0s 16ms/step - loss: 0.6719 - accuracy: 0.7016 - val_loss: 0.6620 - val_accuracy: 0.7812
Epoch 13/159
4/4 [==============================] - 0s 19ms/step - loss: 0.6719 - accuracy: 0.7016 - val_loss: 0.6620 - val_accuracy: 0.7812
Epoch 14/159
4/4 [==============================] - 0s 13ms/step - loss: 0.6719 - accuracy: 0.7016 - val_loss: 0.6620 - val_accuracy: 0.7812
Epoch 15/159
4/4 [==============================] - 0s 27ms/step - loss: 0.6719 - accuracy: 0.7016 - val_loss: 0.6620 - val_accuracy: 0.7812
Epoch 16/159
4/4 [==============================] - 0s 12ms/step - loss: 0.6719 - accuracy: 0.7016 - val_loss: 0.6620 - val_accuracy: 0.7812
Epoch 17/159
```

```
4/4 [==============================] - 0s 11ms/step - loss: 0.6719 - accuracy: 0.7016 - val_loss: 0.6620 - val_accuracy: 0.7812
Epoch 18/159
4/4 [==============================] - 0s 17ms/step - loss: 0.6719 - accuracy: 0.7016 - val_loss: 0.6620 - val_accuracy: 0.7812
Epoch 19/159
4/4 [==============================] - 0s 17ms/step - loss: 0.6719 - accuracy: 0.7016 - val_loss: 0.6620 - val_accuracy: 0.7812
Epoch 20/159
4/4 [==============================] - 0s 21ms/step - loss: 0.6719 - accuracy: 0.7016 - val_loss: 0.6620 - val_accuracy: 0.7812
Epoch 21/159
4/4 [==============================] - 0s 25ms/step - loss: 0.6719 - accuracy: 0.7016 - val_loss: 0.6620 - val_accuracy: 0.7812
Epoch 22/159
4/4 [==============================] - 0s 26ms/step - loss: 0.6719 - accuracy: 0.7016 - val_loss: 0.6620 - val_accuracy: 0.7812
Epoch 23/159
4/4 [==============================] - 0s 23ms/step - loss: 0.6719 - accuracy: 0.7016 - val_loss: 0.6620 - val_accuracy: 0.7812
Epoch 24/159
4/4 [==============================] - 0s 23ms/step - loss: 0.6719 - accuracy: 0.7016 - val_loss: 0.6620 - val_accuracy: 0.7812
Epoch 25/159
4/4 [==============================] - 0s 20ms/step - loss: 0.6719 - accuracy: 0.7016 - val_loss: 0.6620 - val_accuracy: 0.7812
Epoch 26/159
4/4 [==============================] - 0s 22ms/step - loss: 0.6719 - accuracy: 0.7016 - val_loss: 0.6620 - val_accuracy: 0.7812
Epoch 27/159
4/4 [==============================] - 0s 30ms/step - loss: 0.6719 - accuracy: 0.7016 - val_loss: 0.6620 - val_accuracy: 0.7812
Epoch 28/159
4/4 [==============================] - 0s 40ms/step - loss: 0.6719 - accuracy: 0.7016 - val_loss: 0.6620 - val_accuracy: 0.7812
Epoch 29/159
```

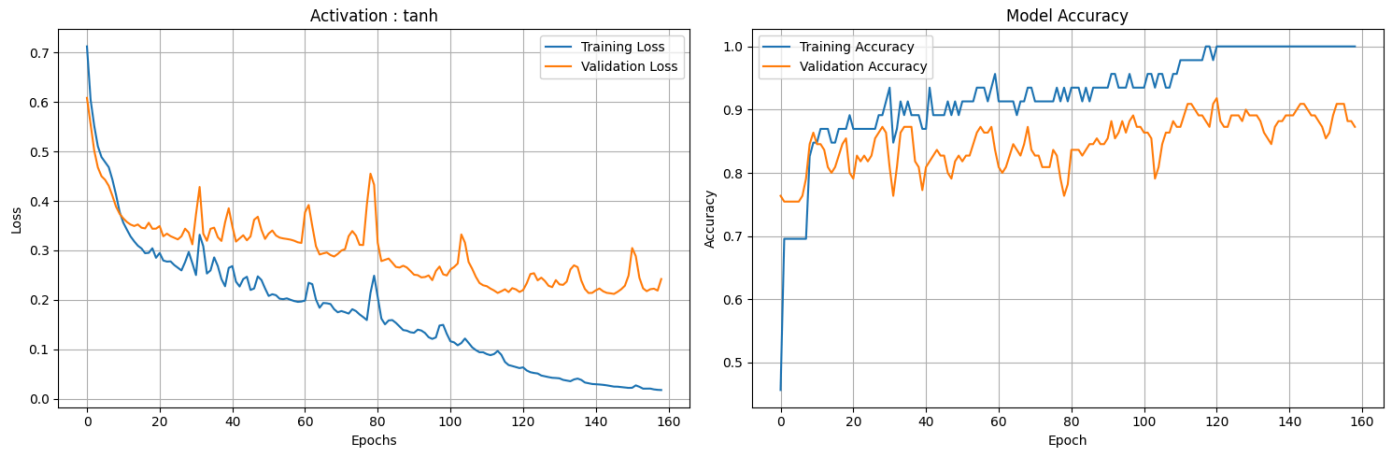
ACTIVATION MODEL

```
act = 'tanh'
# Define the base model function
def create_base_model(input_dim):
    model = Sequential()
    model.add(Dense(32, input_dim=input_dim, activation='relu'))
    model.add(Dense(16, activation=act))
    model.add(Dense(1, activation='sigmoid'))
    return model

# Initialize the model
modeA1 = create_base_model(X_train.shape[1])  # Assuming x_train is defined and has proper shape

# Compile the model
optimizer = Adam(learning_rate=0.004)
modeA1.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model
historyA1 = modeA1.fit(X_train, y_train, epochs=159, batch_size=15, validation_split=0.7, verbose=1)

# Evaluate the model on the test set
loss, accuracy = modeA1.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy: {accuracy*100:.2f} %")


# Print all the results
print("\nResults:")

print(f"Activation : {act}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Epoch 1/159
4/4 [==============================] - 3s 205ms/step - loss: 0.7123 - accuracy: 0.4565 - val_loss: 0.6082 - val_accuracy: 0.7636
Epoch 2/159
4/4 [==============================] - 0s 42ms/step - loss: 0.6046 - accuracy: 0.6957 - val_loss: 0.5547 - val_accuracy: 0.7545
Epoch 3/159
4/4 [==============================] - 0s 40ms/step - loss: 0.5518 - accuracy: 0.6957 - val_loss: 0.5019 - val_accuracy: 0.7545
Epoch 4/159
4/4 [==============================] - 0s 25ms/step - loss: 0.5105 - accuracy: 0.6957 - val_loss: 0.4677 - val_accuracy: 0.7545
Epoch 5/159
4/4 [==============================] - 0s 26ms/step - loss: 0.4887 - accuracy: 0.6957 - val_loss: 0.4499 - val_accuracy: 0.7545
Epoch 6/159
4/4 [==============================] - 0s 37ms/step - loss: 0.4786 - accuracy: 0.6957 - val_loss: 0.4425 - val_accuracy: 0.7545
Epoch 7/159
4/4 [==============================] - 0s 25ms/step - loss: 0.4681 - accuracy: 0.6957 - val_loss: 0.4304 - val_accuracy: 0.7636
Epoch 8/159
4/4 [==============================] - 0s 22ms/step - loss: 0.4432 - accuracy: 0.6957 - val_loss: 0.4105 - val_accuracy: 0.7909
Epoch 9/159
4/4 [==============================] - 0s 50ms/step - loss: 0.4122 - accuracy: 0.8261 - val_loss: 0.3884 - val_accuracy: 0.8455
Epoch 10/159
4/4 [==============================] - 0s 38ms/step - loss: 0.3777 - accuracy: 0.8478 - val_loss: 0.3738 - val_accuracy: 0.8636
Epoch 11/159
4/4 [==============================] - 0s 78ms/step - loss: 0.3560 - accuracy: 0.8478 - val_loss: 0.3648 - val_accuracy: 0.8455
```

```
Epoch 12/159
4/4 [==============================] - 0s 28ms/step - loss: 0.3419 - accuracy: 0.8696 - val_loss: 0.3574 - val_accuracy: 0.8455
Epoch 13/159
4/4 [==============================] - 0s 23ms/step - loss: 0.3282 - accuracy: 0.8696 - val_loss: 0.3522 - val_accuracy: 0.8364
Epoch 14/159
4/4 [==============================] - 0s 28ms/step - loss: 0.3183 - accuracy: 0.8696 - val_loss: 0.3492 - val_accuracy: 0.8091
Epoch 15/159
4/4 [==============================] - 0s 24ms/step - loss: 0.3096 - accuracy: 0.8478 - val_loss: 0.3524 - val_accuracy: 0.8000
Epoch 16/159
4/4 [==============================] - 0s 24ms/step - loss: 0.3040 - accuracy: 0.8478 - val_loss: 0.3459 - val_accuracy: 0.8091
Epoch 17/159
4/4 [==============================] - 0s 28ms/step - loss: 0.2943 - accuracy: 0.8696 - val_loss: 0.3445 - val_accuracy: 0.8273
Epoch 18/159
4/4 [==============================] - 0s 36ms/step - loss: 0.2950 - accuracy: 0.8696 - val_loss: 0.3559 - val_accuracy: 0.8455
Epoch 19/159
4/4 [==============================] - 0s 28ms/step - loss: 0.3043 - accuracy: 0.8696 - val_loss: 0.3438 - val_accuracy: 0.8545
Epoch 20/159
4/4 [==============================] - 0s 54ms/step - loss: 0.2851 - accuracy: 0.8913 - val_loss: 0.3440 - val_accuracy: 0.8000
Epoch 21/159
4/4 [==============================] - 0s 31ms/step - loss: 0.2947 - accuracy: 0.8696 - val_loss: 0.3491 - val_accuracy: 0.7909
Epoch 22/159
4/4 [==============================] - 0s 41ms/step - loss: 0.2796 - accuracy: 0.8696 - val_loss: 0.3285 - val_accuracy: 0.8273
Epoch 23/159
4/4 [==============================] - 0s 25ms/step - loss: 0.2774 - accuracy: 0.8696 - val_loss: 0.3338 - val_accuracy: 0.8182
Epoch 24/159
4/4 [==============================] - 0s 54ms/step - loss: 0.2779 - accuracy: 0.8696 - val_loss: 0.3288 - val_accuracy: 0.8273
Epoch 25/159
4/4 [==============================] - 0s 45ms/step - loss: 0.2705 - accuracy: 0.8696 - val_loss: 0.3255 - val_accuracy: 0.8182
Epoch 26/159
4/4 [==============================] - 0s 37ms/step - loss: 0.2649 - accuracy: 0.8696 - val_loss: 0.3223 - val_accuracy: 0.8273
Epoch 27/159
4/4 [==============================] - 0s 51ms/step - loss: 0.2595 - accuracy: 0.8696 - val_loss: 0.3286 - val_accuracy: 0.8545
Epoch 28/159
4/4 [==============================] - 0s 31ms/step - loss: 0.2764 - accuracy: 0.8913 - val_loss: 0.3439 - val_accuracy: 0.8636
Epoch 29/159
4/4 [==============================] - 0s 23ms/step - loss: 0.2967 - accuracy: 0.8913 - val_loss: 0.3360 - val_accuracy: 0.8727
```

```
# Plot the results
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

axes[0].set_title(f'Activation : {act}')
axes[0].set_xlabel('Epochs')
axes[0].set_ylabel('Loss')
axes[0].plot(historyA1.history['loss'], label='Training Loss')
axes[0].plot(historyA1.history['val_loss'], label='Validation Loss')
axes[0].legend()
axes[0].grid(True)

axes[1].plot(historyA1.history['accuracy'], label='Training Accuracy')
axes[1].plot(historyA1.history['val_accuracy'], label='Validation Accuracy')
axes[1].set_title('Model Accuracy')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.show()
```




```
act = 'tanh'
# Define the base model function
def create_base_model(input_dim):
    model = Sequential()
    model.add(Dense(32, input_dim=input_dim, activation='relu'))
    model.add(Dense(16, activation='relu'))
    model.add(Dense(1, activation=act))
    return model

# Initialize the model
modelA2 = create_base_model(X_train.shape[1])  # Assuming x_train is defined and has proper shape

# Compile the model
optimizer = Adam(learning_rate=0.004)
modelA2.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model
historyA2= modelA2.fit(X_train, y_train, epochs=159, batch_size=15, validation_split=0.7, verbose=1)

# Evaluate the model on the test set
loss, accuracy = modelA2.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy: {accuracy*100:.2f} %")


# Print all the results
print("\nResults:")

print(f"Activation : {act}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Epoch 1/159
4/4 [==============================] - 1s 80ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
Epoch 2/159
4/4 [==============================] - 0s 33ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
Epoch 3/159
4/4 [==============================] - 0s 18ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
Epoch 4/159
4/4 [==============================] - 0s 18ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
Epoch 5/159
4/4 [==============================] - 0s 17ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
Epoch 6/159
4/4 [==============================] - 0s 18ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
Epoch 7/159
4/4 [==============================] - 0s 17ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
Epoch 8/159
4/4 [==============================] - 0s 16ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
Epoch 9/159
4/4 [==============================] - 0s 18ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
Epoch 10/159
4/4 [==============================] - 0s 15ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
Epoch 11/159
4/4 [==============================] - 0s 16ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
Epoch 12/159
4/4 [==============================] - 0s 17ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
Epoch 13/159
```

```
4/4 [==============================] - 0s 15ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
Epoch 14/159
4/4 [==============================] - 0s 18ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
Epoch 15/159
4/4 [==============================] - 0s 15ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
Epoch 16/159
4/4 [==============================] - 0s 18ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
Epoch 17/159
4/4 [==============================] - 0s 14ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
Epoch 18/159
4/4 [==============================] - 0s 33ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
Epoch 19/159
4/4 [==============================] - 0s 19ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
Epoch 20/159
4/4 [==============================] - 0s 22ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
Epoch 21/159
4/4 [==============================] - 0s 22ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
Epoch 22/159
4/4 [==============================] - 0s 21ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
Epoch 23/159
4/4 [==============================] - 0s 22ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
Epoch 24/159
4/4 [==============================] - 0s 16ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
Epoch 25/159
4/4 [==============================] - 0s 15ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
Epoch 26/159
4/4 [==============================] - 0s 16ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
Epoch 27/159
4/4 [==============================] - 0s 20ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
Epoch 28/159
4/4 [==============================] - 0s 18ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
Epoch 29/159
4/4 [==============================] - 0s 20ms/step - loss: 10.7304 - accuracy: 0.3043 - val_loss: 11.6388 - val_accuracy: 0.2455
```

```
act = 'tanh'
# Define the base model function
def create_base_model(input_dim):
    model = Sequential()
    model.add(Dense(32, input_dim=input_dim, activation='relu'))
    model.add(Dense(16, activation='sigmoid'))
    model.add(Dense(1, activation=act))
    return model

# Initialize the model
modelA3 = create_base_model(X_train.shape[1])  # Assuming x_train is defined and has proper shape

# Compile the model
optimizer = Adam(learning_rate=0.004)
modelA3.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model
historyA3 = modelA3.fit(X_train, y_train, epochs=159, batch_size=15, validation_split=0.7, verbose=1)

# Evaluate the model on the test set
loss, accuracy = modelA3.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy: {accuracy*100:.2f} %")


# Print all the results
print("\nResults:")

print(f"Activation : {act}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Epoch 1/159
4/4 [==============================] - 1s 80ms/step - loss: 0.6136 - accuracy: 0.6957 - val_loss: 0.5464 - val_accuracy: 0.7545
Epoch 2/159
4/4 [==============================] - 0s 18ms/step - loss: 0.5924 - accuracy: 0.6957 - val_loss: 0.5231 - val_accuracy: 0.7545
Epoch 3/159
4/4 [==============================] - 0s 16ms/step - loss: 0.5621 - accuracy: 0.6957 - val_loss: 0.5094 - val_accuracy: 0.7545
Epoch 4/159
4/4 [==============================] - 0s 29ms/step - loss: 0.5399 - accuracy: 0.6957 - val_loss: 0.4972 - val_accuracy: 0.7545
Epoch 5/159
4/4 [==============================] - 0s 40ms/step - loss: 0.5240 - accuracy: 0.6957 - val_loss: 0.4942 - val_accuracy: 0.8091
Epoch 6/159
4/4 [==============================] - 0s 33ms/step - loss: 0.5052 - accuracy: 0.7391 - val_loss: 0.4743 - val_accuracy: 0.8364
Epoch 7/159
4/4 [==============================] - 0s 41ms/step - loss: 0.4804 - accuracy: 0.7826 - val_loss: 0.4491 - val_accuracy: 0.8273
Epoch 8/159
4/4 [==============================] - 0s 27ms/step - loss: 0.4620 - accuracy: 0.7826 - val_loss: 0.4308 - val_accuracy: 0.8273
```

```
Epoch 9/159
4/4 [==============================] - 0s 25ms/step - loss: 0.4432 - accuracy: 0.7826 - val_loss: 0.4165 - val_accuracy: 0.8455
Epoch 10/159
4/4 [==============================] - 0s 47ms/step - loss: 0.4168 - accuracy: 0.8261 - val_loss: 0.4100 - val_accuracy: 0.8273
Epoch 11/159
4/4 [==============================] - 0s 39ms/step - loss: 0.3886 - accuracy: 0.8478 - val_loss: 0.4469 - val_accuracy: 0.7273
Epoch 12/159
4/4 [==============================] - 0s 33ms/step - loss: 0.4439 - accuracy: 0.7826 - val_loss: 0.4894 - val_accuracy: 0.7182
Epoch 13/159
4/4 [==============================] - 0s 23ms/step - loss: 0.4143 - accuracy: 0.8043 - val_loss: 0.4125 - val_accuracy: 0.7455
Epoch 14/159
4/4 [==============================] - 0s 23ms/step - loss: 0.3608 - accuracy: 0.8261 - val_loss: 0.3762 - val_accuracy: 0.8182
Epoch 15/159
4/4 [==============================] - 0s 22ms/step - loss: 0.3552 - accuracy: 0.8478 - val_loss: 0.3710 - val_accuracy: 0.8364
Epoch 16/159
4/4 [==============================] - 0s 37ms/step - loss: 0.3493 - accuracy: 0.8478 - val_loss: 0.3697 - val_accuracy: 0.8091
Epoch 17/159
4/4 [==============================] - 0s 25ms/step - loss: 0.3453 - accuracy: 0.8261 - val_loss: 0.3891 - val_accuracy: 0.7545
Epoch 18/159
4/4 [==============================] - 0s 25ms/step - loss: 0.3443 - accuracy: 0.8261 - val_loss: 0.3912 - val_accuracy: 0.7545
Epoch 19/159
4/4 [==============================] - 0s 23ms/step - loss: 0.3375 - accuracy: 0.8261 - val_loss: 0.3722 - val_accuracy: 0.7909
Epoch 20/159
4/4 [==============================] - 0s 40ms/step - loss: 0.3233 - accuracy: 0.8261 - val_loss: 0.3590 - val_accuracy: 0.8182
Epoch 21/159
4/4 [==============================] - 0s 45ms/step - loss: 0.3213 - accuracy: 0.8696 - val_loss: 0.3597 - val_accuracy: 0.8364
Epoch 22/159
4/4 [==============================] - 0s 24ms/step - loss: 0.3363 - accuracy: 0.8696 - val_loss: 0.3643 - val_accuracy: 0.8364
Epoch 23/159
4/4 [==============================] - 0s 25ms/step - loss: 0.3265 - accuracy: 0.8696 - val_loss: 0.3545 - val_accuracy: 0.8364
Epoch 24/159
4/4 [==============================] - 0s 25ms/step - loss: 0.3099 - accuracy: 0.8696 - val_loss: 0.3610 - val_accuracy: 0.8091
Epoch 25/159
4/4 [==============================] - 0s 23ms/step - loss: 0.3082 - accuracy: 0.8478 - val_loss: 0.3710 - val_accuracy: 0.8000
Epoch 26/159
4/4 [==============================] - 0s 37ms/step - loss: 0.3054 - accuracy: 0.8261 - val_loss: 0.3631 - val_accuracy: 0.8000
Epoch 27/159
4/4 [==============================] - 0s 21ms/step - loss: 0.3012 - accuracy: 0.8696 - val_loss: 0.3596 - val_accuracy: 0.8091
Epoch 28/159
4/4 [==============================] - 0s 20ms/step - loss: 0.2992 - accuracy: 0.8478 - val_loss: 0.3792 - val_accuracy: 0.7727
Epoch 29/159
4/4 [==============================] - 0s 22ms/step - loss: 0.3154 - accuracy: 0.8261 - val_loss: 0.3761 - val_accuracy: 0.7727
```

```
act = 'tanh'
# Define the base model function
def create_base_model(input_dim):
    model = Sequential()
    model.add(Dense(32, input_dim=input_dim, activation='relu'))
    model.add(Dense(16, activation='relu'))
    model.add(Dense(1, activation=act))
    return model

# Initialize the model
modelA4 = create_base_model(X_train.shape[1])  # Assuming x_train is defined and has proper shape

# Compile the model
optimizer = Adam(learning_rate=0.004)
modelA4.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model
historyA3 = modelA4.fit(X_train, y_train, epochs=159, batch_size=15, validation_split=0.7, verbose=1)

# Evaluate the model on the test set
loss, accuracy = modelA4.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy: {accuracy*100:.2f} %")


# Print all the results
print("\nResults:")

print(f"Activation : {act}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Epoch 1/159
4/4 [==============================] - 2s 125ms/step - loss: 0.9289 - accuracy: 0.3043 - val_loss: 0.5626 - val_accuracy: 0.7545
Epoch 2/159
4/4 [==============================] - 0s 21ms/step - loss: 0.5802 - accuracy: 0.7174 - val_loss: 0.4940 - val_accuracy: 0.7545
Epoch 3/159
4/4 [==============================] - 0s 19ms/step - loss: 0.5615 - accuracy: 0.6957 - val_loss: 0.4873 - val_accuracy: 0.7545
Epoch 4/159
```

```
4/4 [==============================] - 0s 17ms/step - loss: 0.5563 - accuracy: 0.6957 - val_loss: 0.4693 - val_accuracy: 0.7545
Epoch 5/159
4/4 [==============================] - 0s 19ms/step - loss: 0.5213 - accuracy: 0.7174 - val_loss: 0.4343 - val_accuracy: 0.8091
Epoch 6/159
4/4 [==============================] - 0s 18ms/step - loss: 0.4590 - accuracy: 0.8043 - val_loss: 0.4065 - val_accuracy: 0.8273
Epoch 7/159
4/4 [==============================] - 0s 16ms/step - loss: 0.4243 - accuracy: 0.7609 - val_loss: 0.4257 - val_accuracy: 0.7545
Epoch 8/159
4/4 [==============================] - 0s 19ms/step - loss: 0.4401 - accuracy: 0.7609 - val_loss: 0.4090 - val_accuracy: 0.7636
Epoch 9/159
4/4 [==============================] - 0s 19ms/step - loss: 0.4046 - accuracy: 0.7826 - val_loss: 0.3762 - val_accuracy: 0.8545
Epoch 10/159
4/4 [==============================] - 0s 19ms/step - loss: 0.3812 - accuracy: 0.8261 - val_loss: 0.3781 - val_accuracy: 0.8455
Epoch 11/159
4/4 [==============================] - 0s 36ms/step - loss: 0.3821 - accuracy: 0.8478 - val_loss: 0.3758 - val_accuracy: 0.8455
Epoch 12/159
4/4 [==============================] - 0s 17ms/step - loss: 0.3704 - accuracy: 0.8478 - val_loss: 0.3516 - val_accuracy: 0.8455
Epoch 13/159
4/4 [==============================] - 0s 16ms/step - loss: 0.3385 - accuracy: 0.8696 - val_loss: 0.3455 - val_accuracy: 0.8455
Epoch 14/159
4/4 [==============================] - 0s 14ms/step - loss: 0.3265 - accuracy: 0.8261 - val_loss: 0.3627 - val_accuracy: 0.7636
Epoch 15/159
4/4 [==============================] - 0s 15ms/step - loss: 0.3255 - accuracy: 0.8043 - val_loss: 0.3413 - val_accuracy: 0.8182
Epoch 16/159
4/4 [==============================] - 0s 16ms/step - loss: 0.3052 - accuracy: 0.8696 - val_loss: 0.3363 - val_accuracy: 0.8636
Epoch 17/159
4/4 [==============================] - 0s 20ms/step - loss: 0.3298 - accuracy: 0.8478 - val_loss: 0.3831 - val_accuracy: 0.8455
Epoch 18/159
4/4 [==============================] - 0s 22ms/step - loss: 0.3820 - accuracy: 0.7826 - val_loss: 0.3978 - val_accuracy: 0.8273
Epoch 19/159
4/4 [==============================] - 0s 19ms/step - loss: 0.3774 - accuracy: 0.7826 - val_loss: 0.3675 - val_accuracy: 0.8545
Epoch 20/159
4/4 [==============================] - 0s 18ms/step - loss: 0.3334 - accuracy: 0.8478 - val_loss: 0.3276 - val_accuracy: 0.8636
Epoch 21/159
4/4 [==============================] - 0s 20ms/step - loss: 0.2832 - accuracy: 0.9130 - val_loss: 0.3186 - val_accuracy: 0.8636
Epoch 22/159
4/4 [==============================] - 0s 18ms/step - loss: 0.2575 - accuracy: 0.8913 - val_loss: 0.3560 - val_accuracy: 0.7545
Epoch 23/159
4/4 [==============================] - 0s 19ms/step - loss: 0.2969 - accuracy: 0.8261 - val_loss: 0.3937 - val_accuracy: 0.7455
Epoch 24/159
4/4 [==============================] - 0s 19ms/step - loss: 0.2937 - accuracy: 0.8261 - val_loss: 0.3010 - val_accuracy: 0.8455
Epoch 25/159
4/4 [==============================] - 0s 20ms/step - loss: 0.2338 - accuracy: 0.8913 - val_loss: 0.2912 - val_accuracy: 0.8636
Epoch 26/159
4/4 [==============================] - 0s 23ms/step - loss: 0.2340 - accuracy: 0.9130 - val_loss: 0.3060 - val_accuracy: 0.8636
Epoch 27/159
4/4 [==============================] - 0s 22ms/step - loss: 0.2491 - accuracy: 0.9130 - val_loss: 0.3017 - val_accuracy: 0.8636
Epoch 28/159
4/4 [==============================] - 0s 19ms/step - loss: 0.2391 - accuracy: 0.9130 - val_loss: 0.2894 - val_accuracy: 0.8727
Epoch 29/159
```

```
#ACTIVATION = TANH
#EPOCH = 100
act = 'tanh'
epochs = 159
# Define the base model function
def create_base_model(input_dim):
    model = Sequential()
    model.add(Dense(32, input_dim=input_dim, activation='relu'))
    model.add(Dense(16, activation='relu'))
    model.add(Dense(1, activation=act))
    return model

# Initialize the model
modelA4 = create_base_model(X_train.shape[1])  # Assuming x_train is defined and has proper shape

# Compile the model
optimizer = Adam(learning_rate=0.01)
modelA4.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model
historyA4 = modelA4.fit(X_train, y_train, epochs=epochs, batch_size=15, validation_split=0.7, verbose=1)

# Evaluate the model on the test set
loss, accuracy = modelA4.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy: {accuracy*100:.2f} %")


print('Info :- ')
print(f'Activation : {act}')
print(f'Epochs : {epochs}')
print('')
# Print all the results
print("\nResults:")

print(f"Activation : {act}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Epoch 1/159
4/4 [==============================] - 3s 330ms/step - loss: 0.8700 - accuracy: 0.4565 - val_loss: 0.4834 - val_accuracy: 0.8000
Epoch 2/159
4/4 [==============================] - 0s 76ms/step - loss: 0.4860 - accuracy: 0.7609 - val_loss: 0.4751 - val_accuracy: 0.7909
Epoch 3/159
4/4 [==============================] - 0s 66ms/step - loss: 0.5193 - accuracy: 0.7609 - val_loss: 0.4221 - val_accuracy: 0.8364
Epoch 4/159
4/4 [==============================] - 0s 47ms/step - loss: 0.4091 - accuracy: 0.8478 - val_loss: 0.3971 - val_accuracy: 0.8000
Epoch 5/159
4/4 [==============================] - 0s 43ms/step - loss: 0.7087 - accuracy: 0.8043 - val_loss: 0.4291 - val_accuracy: 0.7545
Epoch 6/159
4/4 [==============================] - 0s 30ms/step - loss: 0.4125 - accuracy: 0.8043 - val_loss: 0.3598 - val_accuracy: 0.8273
Epoch 7/159
4/4 [==============================] - 0s 56ms/step - loss: 0.3418 - accuracy: 0.8478 - val_loss: 0.3582 - val_accuracy: 0.8364
Epoch 8/159
4/4 [==============================] - 0s 40ms/step - loss: 0.3221 - accuracy: 0.8478 - val_loss: 0.3489 - val_accuracy: 0.8545
Epoch 9/159
4/4 [==============================] - 0s 35ms/step - loss: 0.3137 - accuracy: 0.8696 - val_loss: 0.3461 - val_accuracy: 0.8545
Epoch 10/159
4/4 [==============================] - 0s 24ms/step - loss: 0.3062 - accuracy: 0.8913 - val_loss: 0.3438 - val_accuracy: 0.8545
Epoch 11/159
4/4 [==============================] - 0s 25ms/step - loss: 0.3015 - accuracy: 0.8913 - val_loss: 0.3429 - val_accuracy: 0.8636
Epoch 12/159
4/4 [==============================] - 0s 23ms/step - loss: 0.2919 - accuracy: 0.8696 - val_loss: 0.3922 - val_accuracy: 0.7818
Epoch 13/159
4/4 [==============================] - 0s 41ms/step - loss: 0.3215 - accuracy: 0.8913 - val_loss: 0.3417 - val_accuracy: 0.8636
Epoch 14/159
4/4 [==============================] - 0s 24ms/step - loss: 0.2641 - accuracy: 0.8913 - val_loss: 0.3433 - val_accuracy: 0.8727
Epoch 15/159
4/4 [==============================] - 0s 43ms/step - loss: 0.2961 - accuracy: 0.8913 - val_loss: 0.3471 - val_accuracy: 0.8727
Epoch 16/159
4/4 [==============================] - 0s 26ms/step - loss: 0.2817 - accuracy: 0.8913 - val_loss: 0.3291 - val_accuracy: 0.8636
Epoch 17/159
4/4 [==============================] - 0s 27ms/step - loss: 0.2501 - accuracy: 0.8696 - val_loss: 0.3491 - val_accuracy: 0.8273
Epoch 18/159
4/4 [==============================] - 0s 34ms/step - loss: 0.2802 - accuracy: 0.8913 - val_loss: 0.3262 - val_accuracy: 0.8727
Epoch 19/159
4/4 [==============================] - 0s 38ms/step - loss: 0.2523 - accuracy: 0.9130 - val_loss: 0.4101 - val_accuracy: 0.8545
Epoch 20/159
4/4 [==============================] - 0s 39ms/step - loss: 0.3656 - accuracy: 0.8478 - val_loss: 0.3746 - val_accuracy: 0.8545
Epoch 21/159
4/4 [==============================] - 0s 25ms/step - loss: 0.2965 - accuracy: 0.8696 - val_loss: 0.3333 - val_accuracy: 0.8455
Epoch 22/159
4/4 [==============================] - 0s 23ms/step - loss: 0.6027 - accuracy: 0.8478 - val_loss: 1.3713 - val_accuracy: 0.7364
```

```
Epoch 23/159
4/4 [==============================] - 0s 24ms/step - loss: 0.6732 - accuracy: 0.7826 - val_loss: 0.3319 - val_accuracy: 0.8364
Epoch 24/159
4/4 [==============================] - 0s 33ms/step - loss: 0.2413 - accuracy: 0.9130 - val_loss: 0.3629 - val_accuracy: 0.8727
Epoch 25/159
4/4 [==============================] - 0s 38ms/step - loss: 0.3126 - accuracy: 0.8696 - val_loss: 0.3936 - val_accuracy: 0.8545
Epoch 26/159
4/4 [==============================] - 0s 38ms/step - loss: 0.3273 - accuracy: 0.8696 - val_loss: 0.3450 - val_accuracy: 0.8727
Epoch 27/159
4/4 [==============================] - 0s 26ms/step - loss: 0.2614 - accuracy: 0.8913 - val_loss: 0.3185 - val_accuracy: 0.8545
Epoch 28/159
4/4 [==============================] - 0s 27ms/step - loss: 0.2257 - accuracy: 0.8913 - val_loss: 0.3929 - val_accuracy: 0.7909
Epoch 29/159
4/4 [==============================] - 0s 24ms/step - loss: 0.6002 - accuracy: 0.8478 - val_loss: 0.8083 - val_accuracy: 0.7545
```

```
#ACTIVATION = TANH
#EPOCH = 100
actA5 = 'tanh'
lrA5 = 0.001
# Define the base model function
def create_base_model(input_dim):
    model = Sequential()
    model.add(Dense(32, input_dim=input_dim, activation='relu'))
    model.add(Dense(16, activation='relu'))
    model.add(Dense(1, activation=actA5))
    return model

# Initialize the model
modelA5 = create_base_model(X_train.shape[1])  # Assuming x_train is defined and has proper shape

# Compile the model
optimizer = Adam(learning_rate=lrA5)
modelA5.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model
historyA5 = modelA5.fit(X_train, y_train, epochs=epochs, batch_size=15, validation_split=0.7, verbose=1)

# Evaluate the model on the test set
loss, accuracy = modelA5.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy: {accuracy*100:.2f} %")


print('Info :- ')
print(f'Activation : {act}')
print(f'Learning Rate : {lrA5}')
print('')
# Print all the results
print("\nResults:")
print(f"Activation : {act}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Epoch 1/159
4/4 [==============================] - 1s 90ms/step - loss: 1.9040 - accuracy: 0.3261 - val_loss: 1.0558 - val_accuracy: 0.2818
Epoch 2/159
4/4 [==============================] - 0s 21ms/step - loss: 0.9835 - accuracy: 0.3261 - val_loss: 0.8528 - val_accuracy: 0.3455
Epoch 3/159
4/4 [==============================] - 0s 25ms/step - loss: 0.8122 - accuracy: 0.3913 - val_loss: 0.7172 - val_accuracy: 0.5091
Epoch 4/159
4/4 [==============================] - 0s 19ms/step - loss: 0.6927 - accuracy: 0.6087 - val_loss: 0.6271 - val_accuracy: 0.6545
Epoch 5/159
4/4 [==============================] - 0s 18ms/step - loss: 0.6259 - accuracy: 0.6522 - val_loss: 0.5637 - val_accuracy: 0.6909
Epoch 6/159
4/4 [==============================] - 0s 20ms/step - loss: 0.5767 - accuracy: 0.6522 - val_loss: 0.5240 - val_accuracy: 0.7182
Epoch 7/159
4/4 [==============================] - 0s 19ms/step - loss: 0.5427 - accuracy: 0.6957 - val_loss: 0.5027 - val_accuracy: 0.7545
Epoch 8/159
4/4 [==============================] - 0s 24ms/step - loss: 0.5235 - accuracy: 0.7174 - val_loss: 0.4874 - val_accuracy: 0.7636
Epoch 9/159
4/4 [==============================] - 0s 15ms/step - loss: 0.5073 - accuracy: 0.7391 - val_loss: 0.4751 - val_accuracy: 0.7636
Epoch 10/159
4/4 [==============================] - 0s 15ms/step - loss: 0.4957 - accuracy: 0.7609 - val_loss: 0.4647 - val_accuracy: 0.7636
Epoch 11/159
4/4 [==============================] - 0s 17ms/step - loss: 0.4860 - accuracy: 0.7609 - val_loss: 0.4593 - val_accuracy: 0.7727
Epoch 12/159
4/4 [==============================] - 0s 18ms/step - loss: 0.4789 - accuracy: 0.7826 - val_loss: 0.4526 - val_accuracy: 0.7818
Epoch 13/159
4/4 [==============================] - 0s 17ms/step - loss: 0.4711 - accuracy: 0.7826 - val_loss: 0.4459 - val_accuracy: 0.7818
Epoch 14/159
4/4 [==============================] - 0s 20ms/step - loss: 0.4638 - accuracy: 0.7826 - val_loss: 0.4409 - val_accuracy: 0.7818
Epoch 15/159
4/4 [==============================] - 0s 24ms/step - loss: 0.4575 - accuracy: 0.7826 - val_loss: 0.4360 - val_accuracy: 0.7818
```

```
Epoch 16/159
4/4 [==============================] - 0s 22ms/step - loss: 0.4509 - accuracy: 0.7609 - val_loss: 0.4345 - val_accuracy: 0.7909
Epoch 17/159
4/4 [==============================] - 0s 38ms/step - loss: 0.4466 - accuracy: 0.7609 - val_loss: 0.4362 - val_accuracy: 0.7909
Epoch 18/159
4/4 [==============================] - 0s 18ms/step - loss: 0.4483 - accuracy: 0.7609 - val_loss: 0.4379 - val_accuracy: 0.7727
Epoch 19/159
4/4 [==============================] - 0s 16ms/step - loss: 0.4465 - accuracy: 0.7609 - val_loss: 0.4326 - val_accuracy: 0.7909
Epoch 20/159
4/4 [==============================] - 0s 17ms/step - loss: 0.4379 - accuracy: 0.7826 - val_loss: 0.4265 - val_accuracy: 0.8091
Epoch 21/159
4/4 [==============================] - 0s 17ms/step - loss: 0.4306 - accuracy: 0.7826 - val_loss: 0.4209 - val_accuracy: 0.8091
Epoch 22/159
4/4 [==============================] - 0s 17ms/step - loss: 0.4247 - accuracy: 0.7826 - val_loss: 0.4146 - val_accuracy: 0.8091
Epoch 23/159
4/4 [==============================] - 0s 16ms/step - loss: 0.4179 - accuracy: 0.7826 - val_loss: 0.4083 - val_accuracy: 0.8091
Epoch 24/159
4/4 [==============================] - 0s 19ms/step - loss: 0.4119 - accuracy: 0.7826 - val_loss: 0.4035 - val_accuracy: 0.8091
Epoch 25/159
4/4 [==============================] - 0s 16ms/step - loss: 0.4071 - accuracy: 0.7826 - val_loss: 0.3996 - val_accuracy: 0.8091
Epoch 26/159
4/4 [==============================] - 0s 15ms/step - loss: 0.4027 - accuracy: 0.7826 - val_loss: 0.3963 - val_accuracy: 0.8182
Epoch 27/159
4/4 [==============================] - 0s 15ms/step - loss: 0.3987 - accuracy: 0.7826 - val_loss: 0.3933 - val_accuracy: 0.8091
Epoch 28/159
4/4 [==============================] - 0s 17ms/step - loss: 0.3955 - accuracy: 0.7826 - val_loss: 0.3912 - val_accuracy: 0.8091
Epoch 29/159
4/4 [==============================] - 0s 22ms/step - loss: 0.3957 - accuracy: 0.7826 - val_loss: 0.3909 - val_accuracy: 0.8000
```

```
#ACTIVATION = TANH
#EPOCH = 100
actA6 = 'tanh'
lrA6 = 0.0012
# Define the base model function
def create_base_model(input_dim):
    model = Sequential()
    model.add(Dense(32, input_dim=input_dim, activation='relu'))
    model.add(Dense(16, activation='relu'))
    model.add(Dense(1, activation=actA6))
    return model

# Initialize the model
modelA6 = create_base_model(X_train.shape[1])  # Assuming x_train is defined and has proper shape

# Compile the model
optimizer = Adam(learning_rate=lrA6)
modelA6.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model
historyA6 = modelA6.fit(X_train, y_train, epochs=epochs, batch_size=15, validation_split=0.7, verbose=1)

# Evaluate the model on the test set
loss, accuracy = modelA6.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy: {accuracy*100:.2f} %")


print('Info :- ')
print(f'Activation : {act}')
print(f'Learning Rate : {lrA6}')
print('')
# Print all the results
print("\nResults:")
print(f"Activation : {act}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Epoch 1/159
4/4 [==============================] - 2s 80ms/step - loss: 3.8002 - accuracy: 0.3043 - val_loss: 1.8635 - val_accuracy: 0.2455
Epoch 2/159
4/4 [==============================] - 0s 18ms/step - loss: 1.5722 - accuracy: 0.3043 - val_loss: 1.2091 - val_accuracy: 0.2455
Epoch 3/159
4/4 [==============================] - 0s 15ms/step - loss: 1.1327 - accuracy: 0.3043 - val_loss: 1.0342 - val_accuracy: 0.2545
Epoch 4/159
4/4 [==============================] - 0s 15ms/step - loss: 0.9604 - accuracy: 0.3261 - val_loss: 0.9187 - val_accuracy: 0.2545
Epoch 5/159
4/4 [==============================] - 0s 16ms/step - loss: 0.8600 - accuracy: 0.3261 - val_loss: 0.8302 - val_accuracy: 0.3182
Epoch 6/159
4/4 [==============================] - 0s 16ms/step - loss: 0.7888 - accuracy: 0.3478 - val_loss: 0.7544 - val_accuracy: 0.3909
Epoch 7/159
4/4 [==============================] - 0s 18ms/step - loss: 0.7278 - accuracy: 0.4565 - val_loss: 0.6889 - val_accuracy: 0.5273
Epoch 8/159
4/4 [==============================] - 0s 18ms/step - loss: 0.6705 - accuracy: 0.6304 - val_loss: 0.6347 - val_accuracy: 0.6364
```

```
Epoch 9/159
4/4 [==============================] - 0s 22ms/step - loss: 0.6263 - accuracy: 0.6957 - val_loss: 0.5906 - val_accuracy: 0.7455
Epoch 10/159
4/4 [==============================] - 0s 17ms/step - loss: 0.5912 - accuracy: 0.7174 - val_loss: 0.5552 - val_accuracy: 0.7455
Epoch 11/159
4/4 [==============================] - 0s 17ms/step - loss: 0.5663 - accuracy: 0.7391 - val_loss: 0.5287 - val_accuracy: 0.8000
Epoch 12/159
4/4 [==============================] - 0s 21ms/step - loss: 0.5432 - accuracy: 0.7609 - val_loss: 0.5112 - val_accuracy: 0.7818
Epoch 13/159
4/4 [==============================] - 0s 38ms/step - loss: 0.5305 - accuracy: 0.7826 - val_loss: 0.5001 - val_accuracy: 0.8091
Epoch 14/159
4/4 [==============================] - 0s 23ms/step - loss: 0.5225 - accuracy: 0.7826 - val_loss: 0.4916 - val_accuracy: 0.8182
Epoch 15/159
4/4 [==============================] - 0s 43ms/step - loss: 0.5159 - accuracy: 0.7826 - val_loss: 0.4858 - val_accuracy: 0.8182
Epoch 16/159
4/4 [==============================] - 0s 24ms/step - loss: 0.5115 - accuracy: 0.7826 - val_loss: 0.4805 - val_accuracy: 0.8182
Epoch 17/159
4/4 [==============================] - 0s 39ms/step - loss: 0.5070 - accuracy: 0.7609 - val_loss: 0.4771 - val_accuracy: 0.8182
Epoch 18/159
4/4 [==============================] - 0s 43ms/step - loss: 0.5022 - accuracy: 0.7826 - val_loss: 0.4746 - val_accuracy: 0.8182
Epoch 19/159
4/4 [==============================] - 0s 23ms/step - loss: 0.4992 - accuracy: 0.7826 - val_loss: 0.4728 - val_accuracy: 0.8182
Epoch 20/159
4/4 [==============================] - 0s 22ms/step - loss: 0.4951 - accuracy: 0.7826 - val_loss: 0.4690 - val_accuracy: 0.8182
Epoch 21/159
4/4 [==============================] - 0s 25ms/step - loss: 0.4901 - accuracy: 0.7826 - val_loss: 0.4646 - val_accuracy: 0.8182
Epoch 22/159
4/4 [==============================] - 0s 26ms/step - loss: 0.4857 - accuracy: 0.7826 - val_loss: 0.4598 - val_accuracy: 0.8182
Epoch 23/159
4/4 [==============================] - 0s 41ms/step - loss: 0.4807 - accuracy: 0.7826 - val_loss: 0.4558 - val_accuracy: 0.8182
Epoch 24/159
4/4 [==============================] - 0s 35ms/step - loss: 0.4762 - accuracy: 0.8043 - val_loss: 0.4535 - val_accuracy: 0.8273
Epoch 25/159
4/4 [==============================] - 0s 35ms/step - loss: 0.4722 - accuracy: 0.8043 - val_loss: 0.4504 - val_accuracy: 0.8273
Epoch 26/159
4/4 [==============================] - 0s 39ms/step - loss: 0.4682 - accuracy: 0.8043 - val_loss: 0.4471 - val_accuracy: 0.8273
Epoch 27/159
4/4 [==============================] - 0s 25ms/step - loss: 0.4648 - accuracy: 0.8043 - val_loss: 0.4439 - val_accuracy: 0.8273
Epoch 28/159
4/4 [==============================] - 0s 25ms/step - loss: 0.4600 - accuracy: 0.8043 - val_loss: 0.4410 - val_accuracy: 0.8273
Epoch 29/159
4/4 [==============================] - 0s 24ms/step - loss: 0.4546 - accuracy: 0.8043 - val_loss: 0.4378 - val_accuracy: 0.8273
```

```
#ACTIVATION = TANH
#EPOCH = 100
actA7 = 'tanh'
ep = 80
# Define the base model function
def create_base_model(input_dim):
    model = Sequential()
    model.add(Dense(32, input_dim=input_dim, activation='relu'))
    model.add(Dense(16, activation='relu'))
    model.add(Dense(1, activation=actA7))
    return model

# Initialize the model
modelA7 = create_base_model(X_train.shape[1])  # Assuming x_train is defined and has proper shape

# Compile the model
optimizer = Adam(learning_rate=0.0004)
modelA7.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model
historyA7 = modelA7.fit(X_train, y_train, epochs=ep, batch_size=15, validation_split=0.7, verbose=1)

# Evaluate the model on the test set
loss, accuracy = modelA7.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy: {accuracy*100:.2f} %")


print('Info :- ')
print(f'Activation : {actA7}')
print(f'Epoch : {ep}')
print('')
# Print all the results
print("\nResults:")
print(f"Activation : {actA7}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Epoch 1/80
4/4 [==============================] - 1s 100ms/step - loss: 1.2347 - accuracy: 0.3261 - val_loss: 1.2352 - val_accuracy: 0.3091
```

```
Epoch 2/80
4/4 [==============================] - 0s 24ms/step - loss: 1.1453 - accuracy: 0.3261 - val_loss: 1.1538 - val_accuracy: 0.3182
Epoch 3/80
4/4 [==============================] - 0s 21ms/step - loss: 1.0695 - accuracy: 0.3478 - val_loss: 1.0834 - val_accuracy: 0.3364
Epoch 4/80
4/4 [==============================] - 0s 22ms/step - loss: 1.0088 - accuracy: 0.3696 - val_loss: 1.0143 - val_accuracy: 0.3545
Epoch 5/80
4/4 [==============================] - 0s 20ms/step - loss: 0.9469 - accuracy: 0.3696 - val_loss: 0.9556 - val_accuracy: 0.3636
Epoch 6/80
4/4 [==============================] - 0s 25ms/step - loss: 0.8990 - accuracy: 0.3696 - val_loss: 0.9099 - val_accuracy: 0.3636
Epoch 7/80
4/4 [==============================] - 0s 17ms/step - loss: 0.8617 - accuracy: 0.3913 - val_loss: 0.8726 - val_accuracy: 0.3909
Epoch 8/80
4/4 [==============================] - 0s 18ms/step - loss: 0.8295 - accuracy: 0.4130 - val_loss: 0.8349 - val_accuracy: 0.4000
Epoch 9/80
4/4 [==============================] - 0s 19ms/step - loss: 0.7968 - accuracy: 0.4348 - val_loss: 0.7999 - val_accuracy: 0.4455
Epoch 10/80
4/4 [==============================] - 0s 19ms/step - loss: 0.7662 - accuracy: 0.4565 - val_loss: 0.7715 - val_accuracy: 0.4545
Epoch 11/80
4/4 [==============================] - 0s 16ms/step - loss: 0.7423 - accuracy: 0.4783 - val_loss: 0.7433 - val_accuracy: 0.4364
Epoch 12/80
4/4 [==============================] - 0s 18ms/step - loss: 0.7202 - accuracy: 0.5435 - val_loss: 0.7162 - val_accuracy: 0.4636
Epoch 13/80
4/4 [==============================] - 0s 20ms/step - loss: 0.6985 - accuracy: 0.5652 - val_loss: 0.6914 - val_accuracy: 0.4727
Epoch 14/80
4/4 [==============================] - 0s 18ms/step - loss: 0.6764 - accuracy: 0.6304 - val_loss: 0.6691 - val_accuracy: 0.5364
Epoch 15/80
4/4 [==============================] - 0s 18ms/step - loss: 0.6584 - accuracy: 0.6522 - val_loss: 0.6495 - val_accuracy: 0.5727
Epoch 16/80
4/4 [==============================] - 0s 21ms/step - loss: 0.6411 - accuracy: 0.6087 - val_loss: 0.6318 - val_accuracy: 0.6091
Epoch 17/80
4/4 [==============================] - 0s 40ms/step - loss: 0.6285 - accuracy: 0.6304 - val_loss: 0.6154 - val_accuracy: 0.6364
Epoch 18/80
4/4 [==============================] - 0s 23ms/step - loss: 0.6150 - accuracy: 0.6739 - val_loss: 0.6004 - val_accuracy: 0.6636
Epoch 19/80
4/4 [==============================] - 0s 23ms/step - loss: 0.6020 - accuracy: 0.6522 - val_loss: 0.5877 - val_accuracy: 0.6636
Epoch 20/80
4/4 [==============================] - 0s 21ms/step - loss: 0.5928 - accuracy: 0.6739 - val_loss: 0.5782 - val_accuracy: 0.6727
Epoch 21/80
4/4 [==============================] - 0s 21ms/step - loss: 0.5846 - accuracy: 0.7174 - val_loss: 0.5697 - val_accuracy: 0.7000
Epoch 22/80
4/4 [==============================] - 0s 21ms/step - loss: 0.5773 - accuracy: 0.7174 - val_loss: 0.5632 - val_accuracy: 0.7000
Epoch 23/80
4/4 [==============================] - 0s 35ms/step - loss: 0.5712 - accuracy: 0.6957 - val_loss: 0.5564 - val_accuracy: 0.7182
Epoch 24/80
4/4 [==============================] - 0s 19ms/step - loss: 0.5652 - accuracy: 0.6957 - val_loss: 0.5504 - val_accuracy: 0.7182
Epoch 25/80
4/4 [==============================] - 0s 21ms/step - loss: 0.5600 - accuracy: 0.6957 - val_loss: 0.5459 - val_accuracy: 0.7182
Epoch 26/80
4/4 [==============================] - 0s 22ms/step - loss: 0.5553 - accuracy: 0.6957 - val_loss: 0.5423 - val_accuracy: 0.7455
Epoch 27/80
4/4 [==============================] - 0s 19ms/step - loss: 0.5510 - accuracy: 0.7174 - val_loss: 0.5367 - val_accuracy: 0.7364
Epoch 28/80
4/4 [==============================] - 0s 25ms/step - loss: 0.5450 - accuracy: 0.7391 - val_loss: 0.5313 - val_accuracy: 0.7364
Epoch 29/80
4/4 [==============================] - 0s 39ms/step - loss: 0.5406 - accuracy: 0.7174 - val_loss: 0.5288 - val_accuracy: 0.7455
```

```
#ACTIVATION = TANH
#EPOCH = 100
actA8 = 'tanh'
ep = 170
# Define the base model function
def create_base_model(input_dim):
    model = Sequential()
    model.add(Dense(32, input_dim=input_dim, activation='relu'))
    model.add(Dense(16, activation='relu'))
    model.add(Dense(1, activation=actA8))
    return model

# Initialize the model
modelA8 = create_base_model(X_train.shape[1])  # Assuming x_train is defined and has proper shape

# Compile the model
optimizer = Adam(learning_rate=0.0004)
modelA8.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model
historyA8 = modelA8.fit(X_train, y_train, epochs=ep, batch_size=15, validation_split=0.7, verbose=1)

# Evaluate the model on the test set
loss, accuracy = modelA8.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy: {accuracy*100:.2f} %")


print('Info :- ')
print(f'Activation : {actA8}')
print(f'Epoch : {ep}')
print('')
# Print all the results
print("\nResults:")
print(f"Activation : {actA8}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Epoch 1/170
4/4 [==============================] - 3s 201ms/step - loss: 1.1808 - accuracy: 0.3043 - val_loss: 1.1401 - val_accuracy: 0.2545
Epoch 2/170
4/4 [==============================] - 0s 24ms/step - loss: 1.0434 - accuracy: 0.3043 - val_loss: 1.0148 - val_accuracy: 0.2545
Epoch 3/170
4/4 [==============================] - 0s 28ms/step - loss: 0.9457 - accuracy: 0.3043 - val_loss: 0.9198 - val_accuracy: 0.2727
Epoch 4/170
4/4 [==============================] - 0s 25ms/step - loss: 0.8623 - accuracy: 0.3043 - val_loss: 0.8532 - val_accuracy: 0.3091
Epoch 5/170
4/4 [==============================] - 0s 24ms/step - loss: 0.8103 - accuracy: 0.3261 - val_loss: 0.7963 - val_accuracy: 0.3455
Epoch 6/170
4/4 [==============================] - 0s 26ms/step - loss: 0.7641 - accuracy: 0.3913 - val_loss: 0.7494 - val_accuracy: 0.4000
Epoch 7/170
4/4 [==============================] - 0s 29ms/step - loss: 0.7238 - accuracy: 0.4565 - val_loss: 0.7103 - val_accuracy: 0.4182
Epoch 8/170
4/4 [==============================] - 0s 24ms/step - loss: 0.6935 - accuracy: 0.4783 - val_loss: 0.6783 - val_accuracy: 0.5091
Epoch 9/170
4/4 [==============================] - 0s 24ms/step - loss: 0.6654 - accuracy: 0.6087 - val_loss: 0.6546 - val_accuracy: 0.5818
Epoch 10/170
4/4 [==============================] - 0s 30ms/step - loss: 0.6480 - accuracy: 0.6522 - val_loss: 0.6325 - val_accuracy: 0.6364
Epoch 11/170
4/4 [==============================] - 0s 18ms/step - loss: 0.6280 - accuracy: 0.6957 - val_loss: 0.6136 - val_accuracy: 0.6727
Epoch 12/170
4/4 [==============================] - 0s 19ms/step - loss: 0.6127 - accuracy: 0.7174 - val_loss: 0.5969 - val_accuracy: 0.7545
Epoch 13/170
4/4 [==============================] - 0s 16ms/step - loss: 0.6009 - accuracy: 0.7174 - val_loss: 0.5812 - val_accuracy: 0.7818
Epoch 14/170
4/4 [==============================] - 0s 19ms/step - loss: 0.5875 - accuracy: 0.7391 - val_loss: 0.5672 - val_accuracy: 0.7909
Epoch 15/170
4/4 [==============================] - 0s 19ms/step - loss: 0.5769 - accuracy: 0.7391 - val_loss: 0.5542 - val_accuracy: 0.8000
Epoch 16/170
4/4 [==============================] - 0s 18ms/step - loss: 0.5681 - accuracy: 0.7609 - val_loss: 0.5423 - val_accuracy: 0.8091
Epoch 17/170
4/4 [==============================] - 0s 18ms/step - loss: 0.5593 - accuracy: 0.7609 - val_loss: 0.5326 - val_accuracy: 0.8000
Epoch 18/170
4/4 [==============================] - 0s 18ms/step - loss: 0.5505 - accuracy: 0.7826 - val_loss: 0.5260 - val_accuracy: 0.7909
Epoch 19/170
4/4 [==============================] - 0s 18ms/step - loss: 0.5451 - accuracy: 0.7826 - val_loss: 0.5196 - val_accuracy: 0.7636
Epoch 20/170
4/4 [==============================] - 0s 22ms/step - loss: 0.5401 - accuracy: 0.7609 - val_loss: 0.5135 - val_accuracy: 0.7545
Epoch 21/170
4/4 [==============================] - 0s 20ms/step - loss: 0.5349 - accuracy: 0.7391 - val_loss: 0.5076 - val_accuracy: 0.7545
Epoch 22/170
4/4 [==============================] - 0s 19ms/step - loss: 0.5306 - accuracy: 0.7174 - val_loss: 0.5021 - val_accuracy: 0.7545
Epoch 23/170
```

```
4/4 [==============================] - 0s 21ms/step - loss: 0.5262 - accuracy: 0.7174 - val_loss: 0.4967 - val_accuracy: 0.7545
Epoch 24/170
4/4 [==============================] - 0s 17ms/step - loss: 0.5221 - accuracy: 0.7174 - val_loss: 0.4915 - val_accuracy: 0.7545
Epoch 25/170
4/4 [==============================] - 0s 17ms/step - loss: 0.5173 - accuracy: 0.6957 - val_loss: 0.4864 - val_accuracy: 0.7636
Epoch 26/170
4/4 [==============================] - 0s 15ms/step - loss: 0.5128 - accuracy: 0.6957 - val_loss: 0.4817 - val_accuracy: 0.7636
Epoch 27/170
4/4 [==============================] - 0s 18ms/step - loss: 0.5075 - accuracy: 0.6957 - val_loss: 0.4776 - val_accuracy: 0.7636
Epoch 28/170
4/4 [==============================] - 0s 15ms/step - loss: 0.5024 - accuracy: 0.7174 - val_loss: 0.4735 - val_accuracy: 0.7727
Epoch 29/170
4/4 [==============================] - 0s 18ms/step - loss: 0.4972 - accuracy: 0.7174 - val_loss: 0.4697 - val_accuracy: 0.7727
```

```
#ACTIVATION = TANH
#EPOCH = 100
actA9 = 'tanh'
ep = 190
# Define the base model function
def create_base_model(input_dim):
    model = Sequential()
    model.add(Dense(32, input_dim=input_dim, activation=actA9))
    model.add(Dense(16, activation='relu'))
    model.add(Dense(1, activation='sigmoid'))
    return model

# Initialize the model
modelA8 = create_base_model(X_train.shape[1])  # Assuming x_train is defined and has proper shape

# Compile the model
optimizer = Adam(learning_rate=0.0004)
modelA8.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model
historyA8 = modelA8.fit(X_train, y_train, epochs=ep, batch_size=15, validation_split=0.7, verbose=1)

# Evaluate the model on the test set
loss, accuracy = modelA8.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy: {accuracy*100:.2f} %")


print('Info :- ')
print(f'Activation : {actA9}')
print(f'Epoch : {ep}')
print('')
# Print all the results
print("\nResults:")
print(f"Activation : {actA9}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
Epoch 1/190
4/4 [==============================] - 2s 180ms/step - loss: 0.6842 - accuracy: 0.6087 - val_loss: 0.6592 - val_accuracy: 0.6727
Epoch 2/190
4/4 [==============================] - 0s 31ms/step - loss: 0.6751 - accuracy: 0.6304 - val_loss: 0.6513 - val_accuracy: 0.7182
Epoch 3/190
4/4 [==============================] - 0s 40ms/step - loss: 0.6678 - accuracy: 0.6522 - val_loss: 0.6411 - val_accuracy: 0.7273
Epoch 4/190
4/4 [==============================] - 0s 53ms/step - loss: 0.6591 - accuracy: 0.6739 - val_loss: 0.6315 - val_accuracy: 0.7455
Epoch 5/190
4/4 [==============================] - 0s 55ms/step - loss: 0.6512 - accuracy: 0.6739 - val_loss: 0.6224 - val_accuracy: 0.7455
Epoch 6/190
4/4 [==============================] - 0s 51ms/step - loss: 0.6440 - accuracy: 0.6739 - val_loss: 0.6140 - val_accuracy: 0.7455
Epoch 7/190
4/4 [==============================] - 0s 38ms/step - loss: 0.6371 - accuracy: 0.6739 - val_loss: 0.6061 - val_accuracy: 0.7545
Epoch 8/190
4/4 [==============================] - 0s 66ms/step - loss: 0.6324 - accuracy: 0.6957 - val_loss: 0.5987 - val_accuracy: 0.7545
Epoch 9/190
4/4 [==============================] - 0s 43ms/step - loss: 0.6273 - accuracy: 0.6957 - val_loss: 0.5921 - val_accuracy: 0.7545
Epoch 10/190
4/4 [==============================] - 0s 46ms/step - loss: 0.6217 - accuracy: 0.6957 - val_loss: 0.5858 - val_accuracy: 0.7545
Epoch 11/190
4/4 [==============================] - 0s 54ms/step - loss: 0.6176 - accuracy: 0.6957 - val_loss: 0.5801 - val_accuracy: 0.7545
Epoch 12/190
4/4 [==============================] - 0s 51ms/step - loss: 0.6126 - accuracy: 0.6957 - val_loss: 0.5763 - val_accuracy: 0.7545
Epoch 13/190
4/4 [==============================] - 0s 56ms/step - loss: 0.6090 - accuracy: 0.6957 - val_loss: 0.5722 - val_accuracy: 0.7545
Epoch 14/190
4/4 [==============================] - 0s 55ms/step - loss: 0.6060 - accuracy: 0.6957 - val_loss: 0.5679 - val_accuracy: 0.7545
Epoch 15/190
4/4 [==============================] - 0s 39ms/step - loss: 0.6026 - accuracy: 0.6957 - val_loss: 0.5640 - val_accuracy: 0.7545
Epoch 16/190
```

```
4/4 [==============================] - 0s 44ms/step - loss: 0.5993 - accuracy: 0.6957 - val_loss: 0.5613 - val_accuracy: 0.7545
Epoch 17/190
4/4 [==============================] - 0s 71ms/step - loss: 0.5967 - accuracy: 0.6957 - val_loss: 0.5595 - val_accuracy: 0.7545
Epoch 18/190
4/4 [==============================] - 0s 101ms/step - loss: 0.5943 - accuracy: 0.6957 - val_loss: 0.5572 - val_accuracy: 0.7545
Epoch 19/190
4/4 [==============================] - 0s 91ms/step - loss: 0.5917 - accuracy: 0.6957 - val_loss: 0.5551 - val_accuracy: 0.7545
Epoch 20/190
4/4 [==============================] - 0s 45ms/step - loss: 0.5891 - accuracy: 0.6957 - val_loss: 0.5521 - val_accuracy: 0.7545
Epoch 21/190
4/4 [==============================] - 0s 50ms/step - loss: 0.5860 - accuracy: 0.6957 - val_loss: 0.5488 - val_accuracy: 0.7545
Epoch 22/190
4/4 [==============================] - 0s 53ms/step - loss: 0.5831 - accuracy: 0.6957 - val_loss: 0.5455 - val_accuracy: 0.7545
Epoch 23/190
4/4 [==============================] - 0s 47ms/step - loss: 0.5803 - accuracy: 0.6957 - val_loss: 0.5420 - val_accuracy: 0.7545
Epoch 24/190
4/4 [==============================] - 0s 49ms/step - loss: 0.5771 - accuracy: 0.6957 - val_loss: 0.5384 - val_accuracy: 0.7545
Epoch 25/190
4/4 [==============================] - 0s 55ms/step - loss: 0.5747 - accuracy: 0.6957 - val_loss: 0.5353 - val_accuracy: 0.7545
Epoch 26/190
4/4 [==============================] - 0s 27ms/step - loss: 0.5717 - accuracy: 0.6957 - val_loss: 0.5332 - val_accuracy: 0.7545
Epoch 27/190
4/4 [==============================] - 0s 32ms/step - loss: 0.5694 - accuracy: 0.6957 - val_loss: 0.5307 - val_accuracy: 0.7545
Epoch 28/190
4/4 [==============================] - 0s 24ms/step - loss: 0.5669 - accuracy: 0.6957 - val_loss: 0.5282 - val_accuracy: 0.7545
Epoch 29/190
4/4 [==============================] - 0s 37ms/step - loss: 0.5641 - accuracy: 0.6957 - val_loss: 0.5263 - val_accuracy: 0.7545
```

BEST MODEL

```
# Model 2
# Optimizer : Nadam

# determine the optimizer to evaluate
optimizers = tf.keras.optimizers.Nadam
op = optimizers

#for ep in optimizer:
print(f"Training with optimizer: {op.__name__}")

# Initialize the model1
modelOB = create_base_model(X_train.shape[1])

# Compile the model with the current optimizer
optimizer = op(learning_rate=0.004)
modelOB.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# Train the model1
historyOB = modelOB.fit(X_train, y_train, epochs=159, batch_size=32, validation_split=0.2, verbose=1)
```